Danmarks Tekniske Universitet



Fullstack Multiplatform Stock Tracking Application

Bachelor of Science Project

Supervisor: Hubert Baumeister

Gustav Onsberg - s194288

Frederik Kjærgaard - s194610

12/06/2023

Contents

1 Abstract **4**

2 Introduction **4**
2.1 Analysis of the Problem . . . 5
2.1.1 Questionnaire Questions . . . 5
2.1.2 Questionnaire Answers Asset Distribution . . . 6
2.1.3 Questionnaire Answers Platform Distribution . . . 7
2.1.4 Questionnaire Answers Platform Features . . . 8
2.2 Overview of Our Solution . . . 9
2.2.1 Advantages of Our Solution . . . 10
2.2.2 Overview of Technical Details . . . 10

3 Requirements **11**
3.1 Third-party API Requirements . . . 11
3.2 User Stories . . . 12
3.3 User Story Map . . . 13

4 Design **15**
4.1 Found third-party APIs . . . 15
4.1.1 Choosing an API . . . 16
4.2 Tech-Stack . . . 16
4.2.1 Backend . . . 16
4.2.2 Frontend . . . 16
4.2.3 Database . . . 17
4.2.4 Version Control . . . 17
4.2.5 Other Technologies . . . 17
4.3 Software Architecture - Backend . . . 17
4.3.1 Interfaces . . . 19
4.3.2 Retrieving and Storing Data . . . 19
4.3.3 Currency Conversion . . . 22
4.3.4 Error Handling . . . 24
4.3.5 Transactions . . . 25
4.3.6 Value History . . . 26
4.3.7 Routing . . . 26
4.4 Business Logic . . . 28
4.4.1 User . . . 30
4.4.2 Portfolio . . . 31
4.4.3 StockTransaction and CashTransaction . . . 32
4.4.4 Stock and Money . . . 32
4.4.5 StockPosition . . . 32
4.5 Software Architecture - Frontend . . . 33
4.5.1 Integration with the Backend . . . 33
4.5.2 Caching . . . 33
4.5.3 Error Handling . . . 34
4.5.4 Application Flow . . . 35
4.6 UI Design . . . 36
4.7 Security . . . 42
4.7.1 Authentication . . . 42
4.7.2 Authorization . . . 46
4.7.3 Password Handling . . . 46

4.7.4 General Data Protection Regulation (GDPR) . . . 47
4.8 The REST API Structure . . . 48
4.9 Technical Risk Analysis . . . 49
4.9.1 User Information Leak . . . 49
4.9.2 Incorrect Market Data . . . 49
4.9.3 Low Data Integrity . . . 50
4.9.4 Server Interruption and Downtime . . . 51
4.10 Technical Analysis . . . 52
4.10.1 Reliability and Error Handling Analysis . . . 52
4.10.2 Security Analysis . . . 52
4.10.3 Performance Analysis . . . 53
4.10.4 Scalability Analysis . . . 55
4.10.5 Quality Assessment . . . 56
4.10.6 Implementation of New Features . . . 56
4.11 Database Design . . . 57
4.11.1 Conceptual Design . . . 57
4.11.2 Logical Design . . . 59
4.11.3 Normalization . . . 61
4.11.4 Database Objects . . . 67
4.11.5 Populating the Database . . . 69

5 Implementation **70**
5.1 Populating Database . . . 70
5.2 Middleware - Authentication . . . 72
5.3 Middleware - Error Handling . . . 77

6 Tests **78**
6.1 Unit Tests . . . 78
6.1.1 Test Results and Code Coverage . . . 85
6.2 Manual Tests . . . 88

7 Discussion **89**

8 Conclusion **90**

9 Glossary **91**

10 References **92**

11 Appendix **94**
11.1 Appendix 1 - Questionnaire . . . 94
11.1.1 Answers . . . 95
11.2 Appendix 2 - JSON schemas of the REST API . . . 96
11.2.1 /users POST . . . 96
11.2.2 /users DELETE . . . 97
11.2.3 /users/email PUT . . . 98
11.2.4 /users/password PUT . . . 99
11.2.5 /users/preferences GET . . . 100
11.2.6 /users/preferences POST . . . 101
11.2.7 /users/preferences DELETE . . . 102
11.2.8 /stock-profiles GET . . . 103
11.2.9 /stock-histories GET . . . 105
11.2.10 /portfolios GET . . . 109

11.2.11 /portfolios POST . . . 111
11.2.12 /portfolios PUT . . . 112
11.2.13 /portfolios DELETE . . . 113
11.2.14 /search-results GET . . . 114
11.2.15 /stock-transactions GET . . . 117
11.2.16 /stock-transactions POST . . . 119
11.2.17 /stock-transactions PUT . . . 121
11.2.18 /stock-transactions DELETE . . . 123
11.2.19 /cash-transactions POST . . . 124
11.2.20 /cash-transactions PUT . . . 125
11.2.21 /cash-transactions DELETE . . . 127
11.2.22 /transactions GET . . . 128
11.2.23 /value-history GET . . . 130
11.2.24 /tokens POST . . . 134
11.2.25 /tokens PUT . . . 135

1 Abstract

This project was focused on developing a multiplatform stock tracking application, constructed upon the requests of potential users, alongside own personal ideas, and a desire to create and implement the shortcomings of existing competitors on the market. The project used a questionnaire to gather user input, which influenced the features of the application and confirmed the existence of potential users. Despite a limited amount of answers from potential users, the responses from the questionnaire provided a solid foundation for features to include within the application. Agile development principles were utilized, with user stories and a user story map guiding the distribution of features across five releases. The architectural design centered on the frontend, backend, and database was aimed at providing a solid foundation for the application for future implementations and improvements. We chose to use Flutter entirely for frontend development due to its multiplatform integration, despite the restrictions this posed on the web-application part compared to Angular. Dependency inversion was implemented to minimize the impact of potential API changes, while unit testing ensured the stability and reliability of existing and new implementation of features. The security of the application was not the primary focus, which is why we refrained from integrating pre-existing and complex technologies for authentication and password handling. The project had some challenges, such as time constraints and limitations from the API. Despite not being able to fully implement releases four and five, the creation of a solid foundation was prioritized, which proved beneficial for the overall quality and simplicity of future development. Future development should focus on implementing the missing features to enhance the user experience. The project provided valuable learning experiences with new technologies and good development practices in general. Moving forward, considerations for switching to a different API due to commercial use restrictions, addressing scalability and security were discussed. Reflecting on the time invested in the project, including application development and report writing, we are satisfied with the outcome, especially with the solid foundation, the alignment with user feedback, and the complete and general development flow.

2 Introduction

After years of owning stocks, ETFs, Index Funds, etc., we have yet to find a platform with the features and qualities we are interested in. In our search for an adequate system, we were constantly met by missing or flawed features. A few examples are Yahoo Finance, Google Finance, self-made spreadsheets, and Nordnet's Shareville. A non-exhausting list of missing or flawed features includes dividend tracking, tracking of sale of a stock, tracking non-American stocks, limited to only either website or mobile app, limitation on number of positions, and inadequate historical overview. Due to these lacking we decided to create our own multi-platform stock tracking application tailored to fit all our personal preferences and needs for such an application.

The features we perceived missing in the different systems, will be combined into our project. This is to get an application which is satisfactory for most people with everything they may want to track their investments' development. Please observe for now this will **only** be a tracking platform. The "money" a user spends to "buy" a stock on the application, is not real money. It is a simulation/representation, so you, as a user, will be able to track every aspect of your investment, test out new strategies, etc. All without risking real life money.

2.1 Analysis of the Problem

2.1.1 Questionnaire Questions

To motivate and map desired features for the application, we created a questionnaire and shared it among three different stock enthusiast groups on Facebook. The questionnaire itself and the answers can be found in the appendix 11.1 Appendix 1 - Questionnaire. The questionnaire contained questions about the amount of assets the person owns, what platforms they are using, and what features these platforms are missing, but also what they do well. With a total of 15 questionnaire responses, we were able to mold our application after our own personal preferences, while keeping in mind what our potential users might want in a tracking platform. The questionnaire was written in a casual way to meet people on a friendly and relaxing level. We wanted people to be comfortable while answering our questionnaire. We wanted them to answer as honestly and elaborated as much as possible. This was also why we in the very start of the questionnaire mentioned it was completely anonymous and non of the questions were mandatory to answer.

To get an overview of the questionnaire, we sat up the different questions in the following list.

- How many stocks/index-funds/cryptocurrency currencies/other do you have in your complete portfolio? Write them individually, such as: Stocks: 5, Crypto: 10, etc.
  - We wanted to find out, if users were investing in something that we have not thought of, but could implement.
- Which platform do you use to track these? (Yahoo Finance, Nordnet, Saxo, Google Finance, other). Please write all the different ones you use
  - We wanted to find additional platforms to get inspiration from, and to get a general idea of what users were using. To know if some platforms were more popular than others.
- Which features do you find genius, that these platforms have?
  - We wanted to know what features users found to be essential for their choice of platforms. Both to find out what features to prioritize and to find inspiration for new features.
- Which features do you think, if any, that the platforms are missing?
  - We wanted to know, if users too were finding their platforms to be lacking essential features. Again also to find inspiration for features and to correctly prioritize them.
- Which features would your perfect platform have, which others do not? This can be features that are split between different platforms, that you would like to have gathered in one place.
  - This was a more summarizing question, where users could write whatever they wanted as features - big and small. This again to find inspiration for new features.
- How big of a probability would there be for you to switch your current platforms, with something new?
  - This was a slider, where the users could pick a number between 1 and 10. Where 1 being there is no possibility and 10 being there is a huge probability for them to switch.

- Why does your probability look like this?
  - This was a summarizing question, we mostly wanted to figure out why people are not likely to change their platforms. This would indicate how big of a market there is for a new platform.

2.1.2 Questionnaire Answers Asset Distribution

Reviewing the 15 questionnaire responses we indeed found new features and functionalities to incorporate into our solution. We will in this sections point out some of the more important answers. We explain why or why not they would be implemented. According to their first answers to what types of assets and how many they own, responders' combined had three different types of assets: Stocks, funds (both index and ETF), and crypto/forex (crypto/forex will forthwith be refereed to as the general therm "currencies"). One user also reported bonds, but as a single mention it was not be included in the analysis.

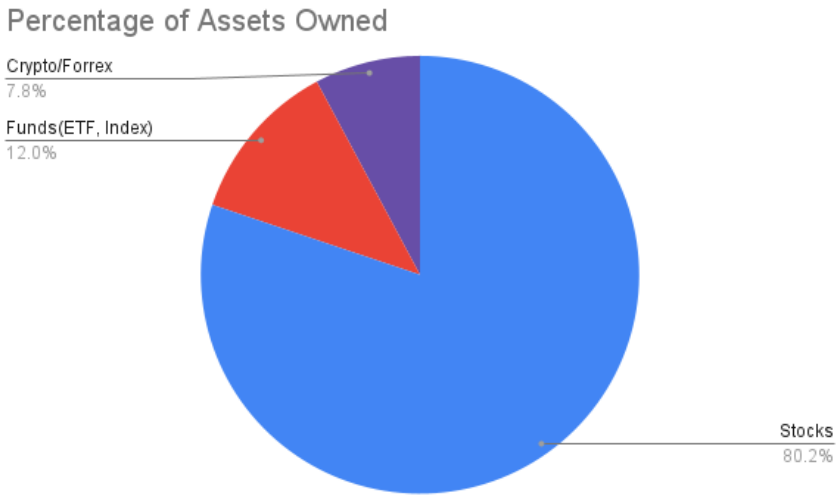


Figure 1: Pie chart showcasing the percentage distribution of users' assets.

Figure 1 shows percentage distribution of the responders' assets. Around 80% stocks, 12% funds, and 8% currencies. This clear picture with stock assets popularity among potential users of the new application meant this would be the biggest priority in the development. Almost all responders owned stocks. However, they also owned funds and currencies hence these could not be ignored.

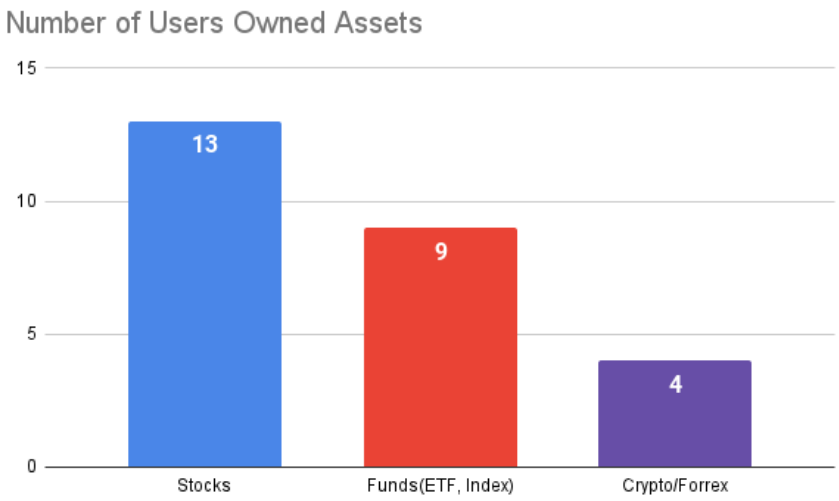


Figure 2: Bar chart showcasing the number of people owning the different assets.

Figure 2 shows what the 15 different responders owned of different assets. Almost all owned stocks and nine people also owned funds. Arguably this means even though stocks might be the biggest percentage of owned assets, almost all also owned some funds or currencies. This aligns well with the answers given regarding features.

2.1.3 Questionnaire Answers Platform Distribution

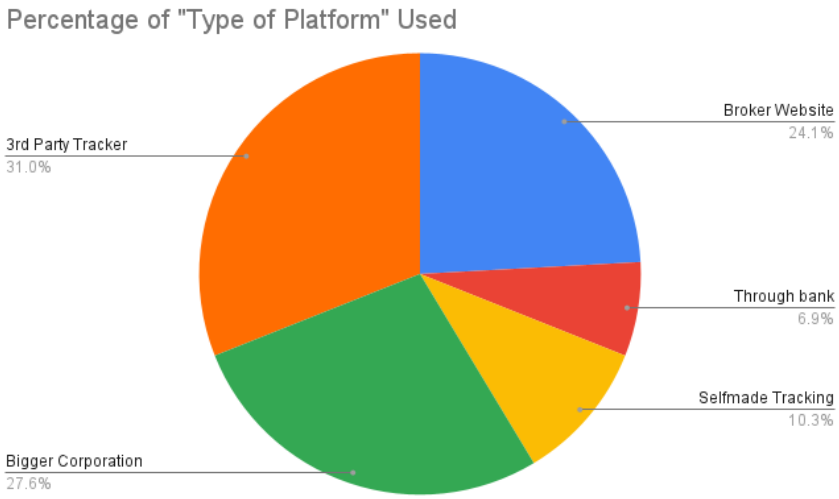


Figure 3: Pie chart showcasing the percentage of different type of platforms used.

Figure 3, shows the percentage distribution of different platforms the reponders used. The different types refers to:

- Broker Website - Nordnet, Saxo, Shareville, Etoro
- Through Bank - Danske Bank, Jyske Bank
- Self-made Tracking - A self-made tracking application with Python, a website or Google Sheets
- Bigger Corporation - Google and Yahoo
- 3rd Party Tracker - Any third party tracker, such as Simply Wall Street, Track Your Dividend, etc.

The figure visualizes how people are very split on what they use. The biggest grouping were 3rd party trackers, broker websites, and bigger corporations. This indicates people are not set on using one kind of tracking as just four of the 15 used one platform. 11 used at least two and up to seven different platforms. This showed potential users will not be turned away by the idea of an additional third party tracking application. Most are already using one. In fact, 10 of the 15 people used some sort of third party tracking application, so they are not locked to the notion it has to be provided by a bigger corporation or a broker. However, just because they are already using a third party tracker, does not indicate the possibility of them changing platform.

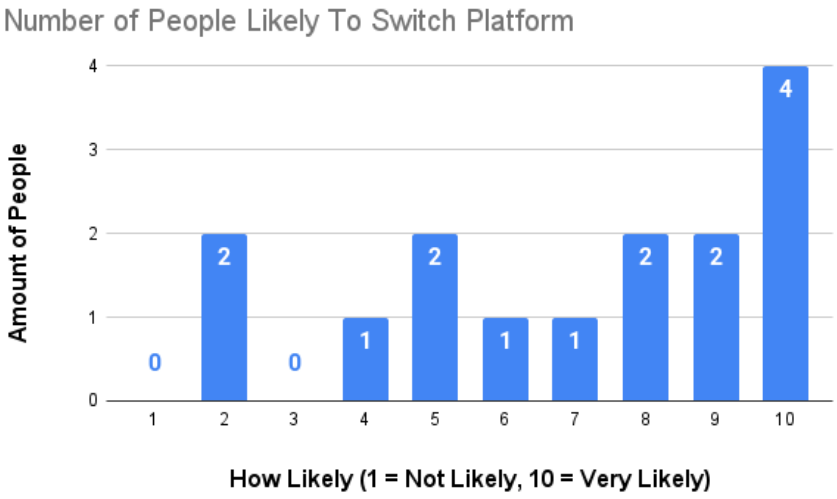


Figure 4: Bar chart showcasing how likely the people are to switch platform.

Figure 4 showcases how likely responders said they were to change their platforms to something new. Responds were very spread out. Ten seemed more likely than not to change and eight people were 8 or above. This generally showed people are open for changing their platforms. Most reasoned by missing the ability to contain all tracking in one place or just by not being set on a particular platform. Those who were less likely to change platform were skeptical if a new platform would have what they needed or they found their current platform to be "more than enough". In general it seems there is a market. People are willing to change if a new platform have the features others are missing.

2.1.4 Questionnaire Answers Platform Features

Responders were asked which features they found to be genius and what was missing on the platforms they were using. One third mentioned being able to track the historical development

of a stock. This is fundamental for a tracking application, and was of course implemented. A different mention was the ability to export portfolio data, to be used and processed in a different application. This was taken into consideration as it holds a lot of potential by allowing users the freedom to export their own data. This also opened up for another feature. The ability to interact directly with the API, instead of only interacting with it through our application. One mentioned the ability to set more advance alarms for the price of an asset. With a notification to be sent when a price meets the alarms criteria. This holds a lot of potential and was taken into consideration for the list of features. However, there may be some foreseeable problems, with limitations from the API. A last mention by a few responders was the ability to get news about a company; document / press releases, etc. However great a feature this would be, it had to remain out of scope. It would require finding news sources uniquely for every stock, which this project simply did not have the resources to implement.

Missing Features

To make a new application relevant it must deliver what others are missing. The responses received aligned well with the notions behind this project. For example missing dividend payout registration and overview and the ability to see all assets in one place.

The last question on what features a perfect platform would have? One optimistic responder wrote an idea for their perfect platform; artificial intelligence (AI) technical analysis. This project is not focused on any AI technologies and this will not be implemented. The remaining answers focuses on generally getting a better overview of their assets' movements, tracking dividend, and the ability to get it in different currencies - all planned features for the new application.

2.2 Overview of Our Solution

As mentioned in the introduction, we were going to create a multi-platform stock tracking application. The multi-platform here refers to a mobile and web application. Both are connected via a backend, that has its data stored in a database. The application itself, should be able to store and track a user's stocks in a portfolio and to display the historical overview. The user should be able to have an unlimited number of portfolios, that they can view separately or together in one combined overview. To do this and to correctly display the historical development the application also had to take into consideration stock split and reverse split. As stocks are traded in different currencies it is also necessary for the application to handle currency conversion. Thus every stock price is converted into the user's currency choice. The user might also want to invest in forex, meaning it should also be possible to create a currency transaction, and hold the bought currency as an investment in a portfolio. To do this, it is required to register historical exchange rates for covered currencies. This will also provide a truer picture of the historic development of stocks, as it takes the currency development into account. A stock holding can also pay dividend, meaning that the stock will pay out a sum of money after a given amount of time. The application should be able to track this, to give a historical overview of dividend received, alongside giving an idea of expected upcoming dividend payouts.

The application would have to feature an integrated search function. Users shall be able to search for a stock, and see different kind of information about it, such as historical development, the company name company, etc. This way, the user would not have to leave our application to review a stock, it would all be contained within the same application.

In its final release everything should of course be connected within a frontend, making it possible for a user to navigate the application.

2.2.1 Advantages of Our Solution

Looking at other solutions, they are close to having all the necessary and "nice-to-have" features, required for a stock tracking application. However, they are often lacking in certain areas. "Sell" your stock and view earnings or losses. Not having correct historical currency exchange rates but instead relay on the current exchange rate instead. I.e. providing a flawed historic development overview as it does not take currency development into account. Not being able to see dividend, etc. The advantage of our solution is it takes what is lacking across most other solutions and place it inside one single application. One important feature is having correct historical exchange rates for currencies. It is not just the asset's price that changes, the currency exchange rate can fluctuate massively and impact as well.

Another feature missing from a lot of other applications, is the ability to see dividend payout. A lot of applications simply just do not showcase the dividend of a stock, giving no insight into it. Missing from many applications is the ability to when "buy" and "sell" stocks, to keep full track of all your transactions, both when buying and selling. This would give a great overview of what you have gained and/or lost through your investments.

2.2.2 Overview of Technical Details

This section gives a more technical description of the planned application, with a general overview of the technical details. As mentioned the platform will use a mobile and web application. Both will be connected via a common backend, that has its data stored in a database. The frontend formation was initially planned to being split into two; one for the web application, created in Angular, and one for the mobile application, created in Flutter. However, to focus on implementing features both were created in Flutter.

The backend would be setup as a REST API, for the frontend to be integrated with. A REST API is a protocol of rules and conventions for interacting with web services. This architecture utilizes HTTP as its communication protocol. It enables the possibility of different software systems to communicate over the web. Unlike a general API, a REST API specifically follows the architecture rules of REST. This includes, when a user sends a request to the REST API, it will send a representation of the state of the resource, to the endpoint. This state is represented in the body of the request and will in our project be represented through JSON. It is the more commonly used format and can be read by multiple different frameworks. Another aspect when using REST API, is the use of headers and parameters. These act as identifier for the endpoint, the request is sent to, for what should happen with the data at the receiving endpoint and what information should be returned. With this, the HTTP methods at which the resources are sent, is also important. The HTTP methods, GET, POST, PUT, DELETE, all depicts a way that the user expects the data to be handled and should follow conventions of these. The REST API was build in C#, using ASP.NET Core. C# was selected as it was planned to host the application on Azure. This made integration easier as both C# and Azure are created by Microsoft. Similarly the database would also be hosted on Azure, meaning that it would be written in T-SQL.

The backend would be the operative connection between the frontend and database. Needed data on different stocks was obtained from Yahoo Finance API. It was possible to collect all registered information about a stock, including historical stock prices, dividend payout, etc. To limit the number of future calls to this API, and to prevent any problems arising if it experiences any outages, all received information was saved in our database on Azure. This means every time a user searches for a stock, it will first be searched in our database. If the information does not exists in our database, the application will then call the Yahoo Finance API instead. When the Yahoo Finance API is called, the additional data is saved to the database. This means next time a user searches for this stock, it will retrieve the information from the database instead.

3 Requirements

3.1 Third-party API Requirements

A third-party API had to be identified for the project. It was the logic first step due to the selected API could have restrictions rendering some desired features impossible to include in the new application. To identify the correct API in the design section, certain requirements were set:

- Get stock data
  - The API must be able to retrieve basic stock data: Price, company name, country of registration and currency.
- Get stock data from minimum 70 percent of Google supported exchanges [4].
  - Google supported exchanges refers to the extensive list of supported exchanges, that covers a large segment of the international market, not only USA.
  - A more subjective reason for this requirement was experiences with the system and its abbreviations.
- Get currency data
  - The API must, as minimum, be able to retrieve exchange rates between a currency and US Dollar
  - The API must cover exchange rates of all currencies used on the exchanges the API supports
  - This criteria could be fulfilled by another API, but one API would be preferable.
- Get historic data (minimum of 5 years)
  - The API must be able to retrieve historic data; stock price and currency to USD exchange rate.
  - Data must cover minimum five year on a day by day resolution .
  - Data must support display of significant stock development
  - Information spanning five years is pivotal for a realistic view of a stock's development. The COVID-19 crisis illustrates the importance. The enormous changes in company values seen from 2020 to 2021 had more to do with the world crisis than actual company performances.
- Get dividend data
  - The API should be able to retrieve information on the dividend of a stock, frequency and amounts.
  - Dividend information would benefit complete correct stock data, but it is not required.
- Be free
  - This is a bachelor project, we therefore do not wish to spend money on it.

3.2 User Stories

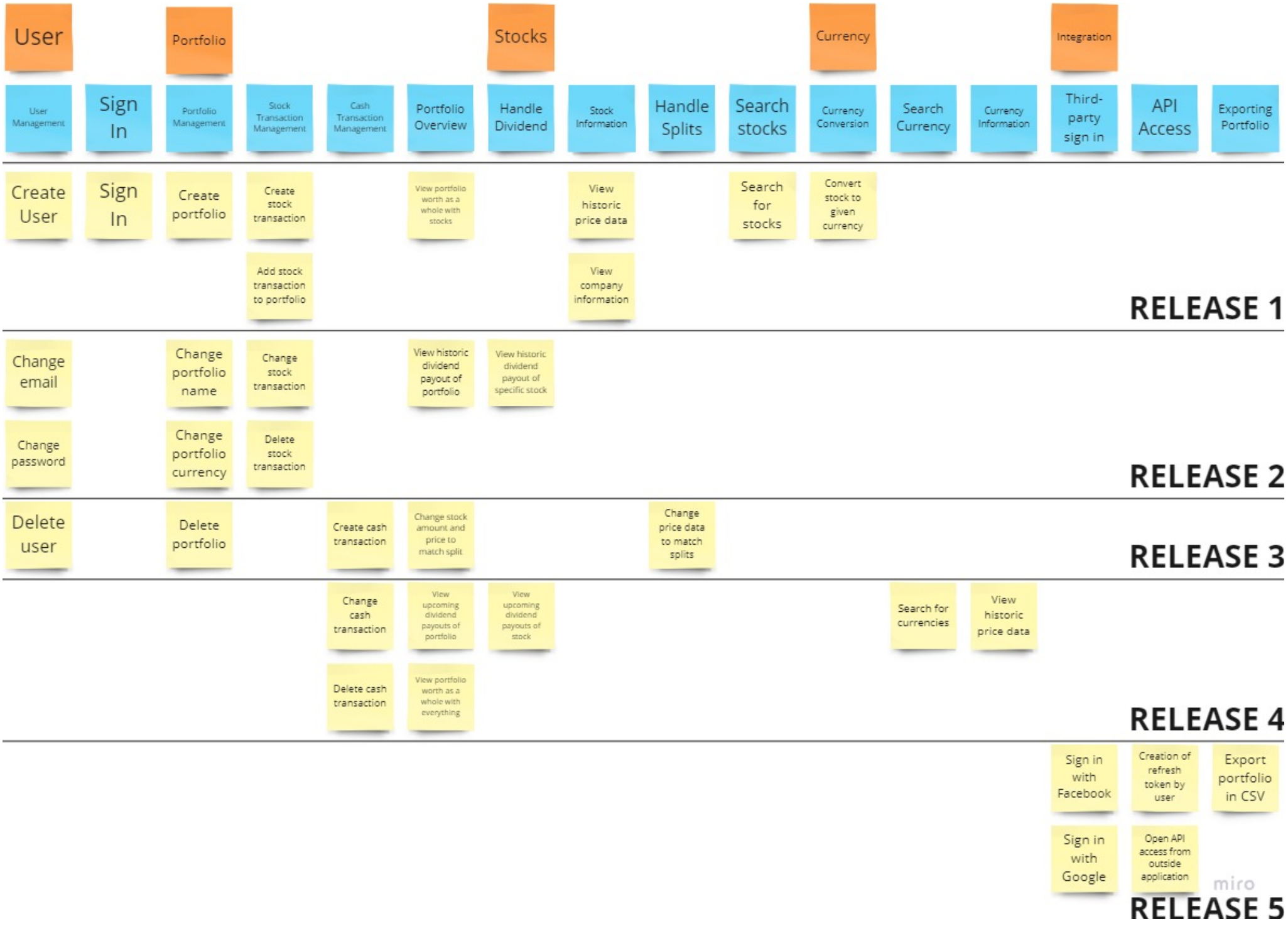
Based on the questionnaire responds and the original frustration with existing platforms user stories were formulated. These were instrumental in planning and developing the new application and its features with an agile development environment. Below are general user stories for features and implementations of the application.

- As a user, I want to be able to create a user profile, so my information gets saved.
- As a user, I want to create a portfolio of stocks, so I have a place to add and store information about all the stocks I want to track.
- As a user, I want to be able to search for different stocks, to find inspiration for new or known stocks.
- As a user, I want to be able to view the historic development in stock value, to get a historic overview.
- As a user, I want to add a stock to my portfolio, so that I will be able to keep track of it.
- As a user, I want an overview of all stocks in my portfolio, so that I can view the historic development of my portfolio as a whole.
- As a user, I want to be able to change my e-mail, as I might want to use a different one for my account.
- As a user, I want to be able to change my password, as I might have forgotten mine and need it changed.
- As a user, I want to be able to change for example portfolio name and default currency.
- As a user, I want to be able to delete any individual portfolio, for example a redundant tracking portfolio.
- As a user, I want to be able to delete any abandon experimental transaction setup, to have less clutter in my overview.
- As a user, I want to be able to change any stock transaction on my portfolio, so that I can change any potential mistakes I might have made.
- As a user, I want to view historic information about dividend payout of a stock, so that I can get an overview of the increase or decrease of the payout.
- As a user, I want to view the historic dividend payouts of the stocks in my portfolio, to get an overview of the development as a whole.
- As a user, I want to see the upcoming expected dividend payouts of different stocks, so that I can see if the payout has increased, decreased, or if it is the same.
- As a user, I want to be able to delete my user, if I do not want to use the application anymore.
- As a user, I want to be sure that stock splits are handled correctly on my portfolio, so that historic values of my portfolio is correct.
- As a user, I want to be able to search for different currencies, as to get inspiration for known or new currencies.
- As a user, I want to be able to view historic data of a currency, so I can get an overview of the historic development of the price.

- As a user, I want to be able to create a fictional cash transaction with a currency to my portfolio, so that I can keep track of the different transactions I have made to my portfolio.
- As a user, I want to be able to change any cash transactions, should I have put in wrong information.
- As a user, I want to be able to delete a cash transaction, as I might have the possibility to create a wrong one, that I do not need.
- As a user, I want to be able to have an overview of the different currencies in my portfolio, so that I can view the historic development of my portfolio as a whole.
- As a user, I want to be able to sign in using third party application, such as Google, Facebook, etc, to make signing in easier and quicker.
- As a user, I want to be able to create a refresh token that gives me access to the API, so that I can retrieve data about my portfolio outside of the application.
- As a user, I want to be able to export my portfolio to different file formats, so that I can run the file through another application.

3.3 User Story Map

On the next page, we have the figure for our user story maps, created from the user stories in the previous section. Here it is also displayed in different releases, corresponding to the priorities of the features - 1 being the highest and 5 being lowest.


User
Portfolio
Stocks
Currency
Integration
User Management
Sign In
Portfolio Management
Stock Transaction Management
Cash Transaction Management
Portfolio Overview
Handle Dividend
Stock Information
Handle Splits
Search stocks
Currency Conversion
Search Currency
Currency Information
Third-party sign in
API Access
Exporting Portfolio
Create User
Sign In
Create portfolio
Create stock transaction
View portfolio worth as a whole with stocks
View historic price data
Search for stocks
Convert stock to given currency
Add stock transaction to portfolio
View company information
RELEASE 1
Change email
Change portfolio name
Change stock transaction
View historic dividend payout of portfolio
View historic dividend payout of specific stock
Change password
Change portfolio currency
Delete stock transaction
RELEASE 2
Delete user
Delete portfolio
Create cash transaction
Change stock amount and price to match split
Change price data to match splits
RELEASE 3
Change cash transaction
View upcoming dividend payouts of portfolio
View upcoming dividend payouts of stock
Search for currencies
View historic price data
Delete cash transaction
View portfolio worth as a whole with everything
RELEASE 4
Sign in with Facebook
Creation of refresh token by user
Export portfolio in CSV
Sign in with Google
Open API access from outside application
RELEASE 5
miro

4 Design

The design phase sought to ensure the application would meet its functional requirements in an efficient manner. It should be maintainable and allow for expansion withe future features. The design phase addressed application features, the tech-stack, how data flows through the application, how different components connects - how the frontend, backend, and database interact. Their individual architectures are crucially linked.

4.1 Found third-party APIs

Identified were different APIs, believed to meet the majority of the set criteria.

- Polygon [15]:
  - The Polygon API. The free version supports live and historical data, currency data, dividend, and other investment options to expand the application in a further point in time. It also supports dividend data and have no hard-limit.
  - The paid version, starting at 29$ a month, provides data five years back, but it still only covers US exchanges, completely cutting off international coverage.
- Finnhub [3]:
  - The Finnhub API. The free version fulfills almost no criteria. It only covers the US, only has stock data from a year back, no dividend and no currency. It does have a hard-limit of 60 calls a minute.
  - The paid version fulfills all criteria and a whole lot more. However, at the price of 50$ a month, this API is more for professional than personal usage.
- Yahoo Finance API [21]:
  - This API fulfills all criteria.
  - Yahoo Finance API supports daily data from the day that a company is listed, and up-till today. This includes currency exchange rates and historic dividend payouts.
  - The drawback for this API is, it technically deprecated back in 2017. It is not maintained nor updated anymore. However, it is still one of the most used stock API's worldwide, almost always being amongst multiple websites top 10 stock API's.
- marketstack [10]:
  - The marketstack API. The free version supports the Google supported exchanges and more, giving significant international market support. It also supports dividend information, but no currency.
  - The API is, however, limited to 100 calls a month and only a year back of history.
  - A paid basic version (10$ a month) gives 10 years worth of data and 10000 requests a month. Due to the lack of currency data however, this API needs to be paired with another, to get exchange rates.
- Aplha Vantage [19]:
  - The Alpha Vantage API supports 500 calls a day, stock data back 20+ years, currency exchange rates, and support of multiple international exchanges. However specifics are not stated.
  - To work around the 500 calls a day, it provides a paid version for 50$ a month. It does not support dividend either.

4.1.1 Choosing an API

First out-ruled were Polygon and Finnhub API. The former only covers US exchanges cutting international support off completely. The latter is simply not an API suited for personal use. The free version was insufficient and the least expensive paid option started at 50$ monthly.

Neither AlphaVantage nor Marketstack were found to be sufficiently proficient in their free versions.

Yahoo Finance was chosen for this project. It was the only evaluated API that, in a free version, lives up to all set requirements. One important aspect about Yahoo Finance API must be observed. It is only for internal and non-commercial use. While utilizing this API, the application cannot legally be published for commercial use.

4.2 Tech-Stack

For this project the application was going to have a backend, a frontend, and a database. This is explained in depth in the sections 4.3 Software Architecture - Backend, 4.5 Software Architecture - Frontend and 4.11 Database Design. This section explains the used tech-stack for the architecture and why.

4.2.1 Backend

The choice was either Azure and C# or AWS (Amazon Web Services) and Java. For the backend C# was used to enabled it to work with Azure which should host the database and web-application. Azure and C# were selected simply because, unlike AWS, Azure gives free credits for students. These credits sums up to $100, which were used to run the project on Azure.

C# is tightly integrated with the .NET framework. The ASP.NET Core is integrated into C#, it greatly enhances the compatibility with other Microsoft technologies. C# has a strong community behind it, with multiple packages available through the NuGet package manager.

Blade or Spring-Boot could have been used for Java, however these are open-source projects, and not developed by Java. It could therefore lead to insufficient compatibility, however the community behind Java is very strong, posing no problem. C# was then chosen, simply for the integration with Azure.

MSTest was planned to be used for unit-testing the backend. MSTest has a comprehensive set of features for unit-testing. This is for example features for initialization and cleanup. MSTest is integrated into the .NET framework ensuring significant compatibility with other Microsoft technologies.

4.2.2 Frontend

For the frontend Flutter was used. It was used for both the mobile- and web-, application due to its simplicity. It was able to create the UI for all applications in a singular project. It also has a good cross-platform compatibility between Android and IOS. Initially the web-application was thought to use Angular. However, instead of creating two identical designs for both web and mobile, one could be created in Flutter and then compiled to web, Android, and IOS usage.

Flutter is a Google open-source framework, and has a strong community behind it with multiple packages available through the pub package manager. This aids the designing process significantly, by the use of pre-made packages and elements. On top of this, Flutter supports the Google Material Design, Material 3 [5], again simplifying the design process.

4.2.3 Database

Using Azure gave some constraints for the database structure - the SQL relational database, running on T-SQL (Transact-SQL). Relational database being a database management system, that stores and organizes data in a structured manner. In this case in a tabular structure.

4.2.4 Version Control

Apart from the aforementioned parts, the project would be using Git for version control. The backend, frontend, and database would be separated into three different repositories, to easier keep them organized and structured. Apart from this, there was created a deployment branch and a main branch for the backend. The deployment branch has a direct connection with Azure, meaning each pull request made to the deployment branch, would be deployed automatically to the web application on Azure. This ensures the deployment branch is always in a working state, as it should never actively be worked on, but simply pulled to, when main is in a working and finished state.

4.2.5 Other Technologies

The backend would have a separated folder to take care of populating the database. This is described in detail in the section 4.11.5 Populating the Database. To populate the database it was necessary to scrape the Yahoo Finance website. To do this Python was used with the web-scraping module "Selenium". It mimics a normal user on a website, which makes it possible to scrape sites such as Yahoo Finance, as it waits for everything to be loaded, before scraping. Using something like "BeautifulSoup", takes the source code of a link, but if the HTML is not loaded for the page at the startup immediately, BeautifulSoup would not get the correct source code.

For testing and having a centralized place to call the API directly, instead of only doing it from the frontend or through a web browser, the API platform Postman [16] was used. This browser based or locally installed software, allows users to set up endpoints for any local or global API. It is possible to specify the HTTP method, any parameters, headers, body, etc., and share it with other people. Every time an endpoint was created, it could be set up in Postman too, not just test it, but to document the endpoint as well, by saving requests and the structure of what they returned. It was also possible to set up example queries of the Yahoo Finance API, to have it documented how to call it, and what it would return. This is an essential tool when building an API, as it makes testing, debugging and documenting each endpoint significantly easier.

4.3 Software Architecture - Backend

The backend architecture is the vital foundation accounting for the efficiency and reliability of the application. It is critical for the application as a whole. The backend serves as a centralized mechanism for communication between the frontend and database.

In addition to the BackendService, which is the backend itself, there are two other sub-projects. The BackendService.tests, which is the project's test environment, and the PopulateDB, the independent Phyton and C# tool used to populate the database. These will be referenced throughout the Design section.

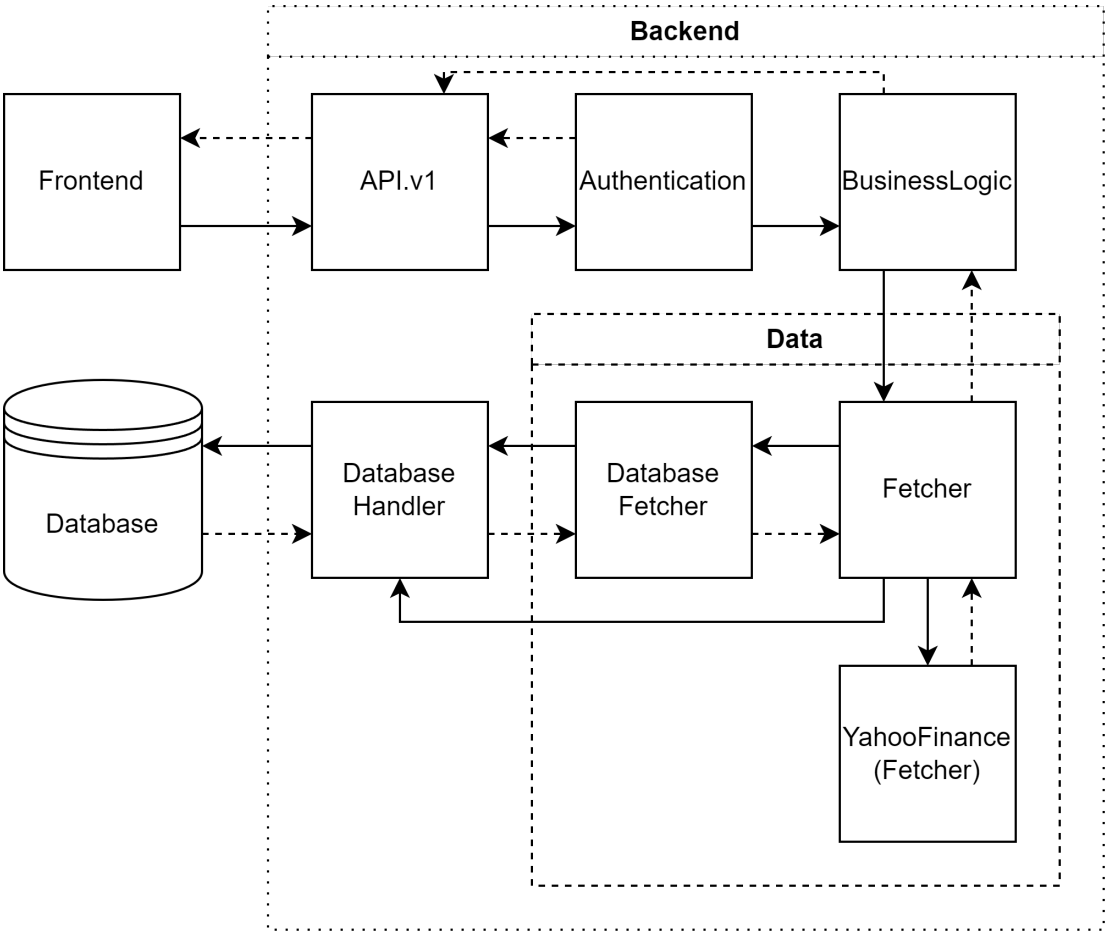


Figure 5: Diagram showing the system architecture

Before diving into the specific parts of the architecture, we will first be looking at the overall flow of the application. For a starter, the backend utilizes the use of interfaces for its implementation on the different components/namespaces. The use of this promotes the implementation of dependency inversion, centralization and flexibility of the application. The different namespaces are displayed on figure 5. Here we see a representation of the individual namespaces of the backend and how they interact. The frontend calls the API.v1 namespace, which controls the routing of the entire backend. It is the only communication between the backend and the frontend, and handles both incomming requests and the return of data. It is here where every individual endpoint is being set up, alongside the connection of the middleware to the rest of the application. The middleware is part of the authentication namespace, while also being built up by the error handling. The middleware authenticates any request, before it is routed to the correct endpoint. If the authentication fails, then it sends back either 401 or 403 HTTP status, unauthorized or forbidden, to the API.v1, which sends it back to the frontend. If the authentication is granted, then the request is routed to the correct endpoint, from where it then calls the business logic of the application. The business logic is responsible for everything that has to do with data handling. When it has received the data it needs, it then returns that data back to the API.v1 namespace, that then gets run through the second part of the middleware, the error handling. The error handling middleware takes care of any exceptions that has been thrown throughout the process of the data, to then handle the message and HTTP status code

of it. It then returns either the data, or the message and status code back to the frontend.

As seen on the figure, the business logic has a single connection forward, which is the fetcher. The fetcher acts as an overall parent of all fetching of data, being that from the database or the Yahoo Finance API. The fetcher takes care of deciding which of the two it should call. Here, the Yahoo Finance fetcher takes care of retrieving data from the Yahoo Finance API and returns it back to the parent fetcher. It can also call the Database Fetcher, which takes care of retrieving existing data from the database. It does this by calling the Database Handler, which is the only direct connection to the database outside of the backend. The Fetcher also has a direct connection to the database handler, as it takes care of saving data it gets from the Yahoo Finance fetcher in the database. This is done to prevent the Yahoo Finance namespace to have a minimum of interaction with the rest of the application, to enforce dependency inversion on the specific API.

4.3.1 Interfaces

The use of interfaces was used for the Fetcher namespace, as shown on figure 5. The parent fetcher acts as the centralized functionality of fetching anything that has to do with the database and the Yahoo Finance API. As it utilizes the Yahoo Finance API, it vital for the namespace to implement interfaces, to enforce dependency inversion. Dependency inversion is necessary to enforce, in case of the need of changing the API. By using interfaces, it creates a complete structure of the different functions that are used by the fetcher namespace, meaning that switching out the API only requires changing on the Yahoo Finance namespace. It creates rule sets for what should be given as parameters, how it should be formatted and what the function itself returns. Having all of this defined in interfaces, makes the possibility of changing the API much simpler.

4.3.2 Retrieving and Storing Data

When a user have requested any historical data, that request is sent to the Fetcher class corresponding to the type of asset the information was requested about. The database handler is implemented using the same interface as the fetcher. This allows the fetcher to forward the request to the database handler, which retrieve any data from the database between the two requested dates. When the data is returned the fetcher looks at the result. If there is no data the original request is sent to the Yahoo Finance fetcher. If there is data a request is sent for missing data before if there is any missing data. The same is the case for missing data after. The Yahoo Finance fetcher query the Yahoo Finance API and saves the result by sending it the database handler before returning the result. This flow can see visualized in the below sequence diagram.

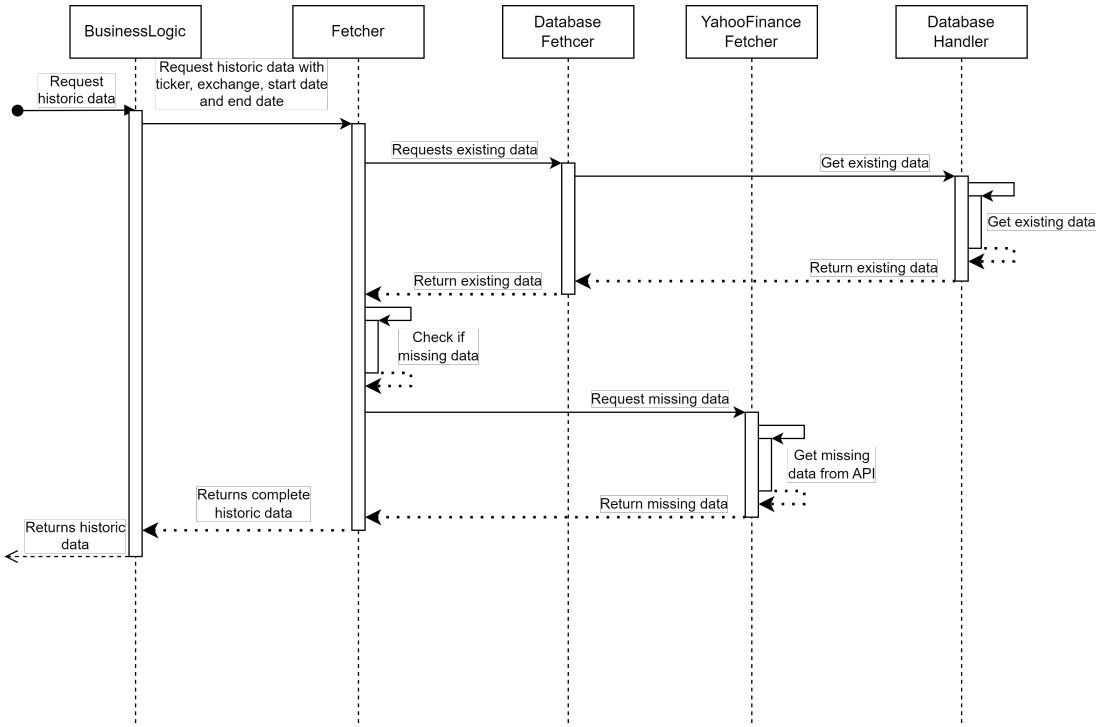


Figure 6: Sequence diagram showcasing the flow of requesting and retrieving missing data

Figure 6 depicts the flow, of the business logic getting requested for historic data of an asset, the business logic requesting this data from the fetcher, that then figures out whether or not this data exist in the database. It will then get any missing data from the Yahoo Finance API, where the complete data would then be returned to the business logic.

As seen on the figure 6, we first start by checking if the data exists in the database. The way this works, is showcased in figure 7 below. It does this, by requesting the data from the database fetcher, that then gets the existing data from the database handler and returns it back to the database fetcher, that then returns it back to the overall fetcher. The fetcher then determines if any data is missing, either before or after the requested dates, to determine if newer or older data is missing. If any of these are missing, it then calls the Yahoo Finance fetcher with the intervals of data that is missing. The Yahoo Finance fetcher then calls the API, and retrieves whatever data is missing. When it has retrieved this data, it then returns the data back to the fetcher. The reason behind this flow and not just letting the Yahoo Finance fetcher call the database for existing data and save the missing data, is to secure dependency inversion. Everything that utilizes the Yahoo Finance API is kept to a minimum, as to make it easier to switch it with another API, if this is needed. By returning the data to the fetcher, the fetcher can handle any refactoring of the data and storing it in the database. It is important to note, that the data here, is not just the price. The Yahoo Finance fetcher will also check for dividends and splits that it then saves in the database. These fit into the overall flow alongside the price, and is gotten and saved in the exact same way, together when the price is gotten.

By doing this flow, first checking if the data exist, getting it from Yahoo Finance, and then returning the complete list of data, is to secure the integrity of the data and the act of dependency inversion. It secures the integrity of the data, as it is possible to catch any errors of fetching the data from the API. This allows for the detection of any errors, and prevents retrieving any incomplete or defect data from the database. The dependency inversion of the flow, by separating

the Yahoo Finance fetcher and the database handler, enables a great level of flexibility of the backend, making it severely easier to switch to a different API if needed. The isolation of everything to do with the API and the use of implementing interfaces, increases this flexibility. On top of this, the flow also adds to the efficiency of the retrieval of data. This is done by excluding unnecessary calls to the API, as it first checks if the data already exists in the database.

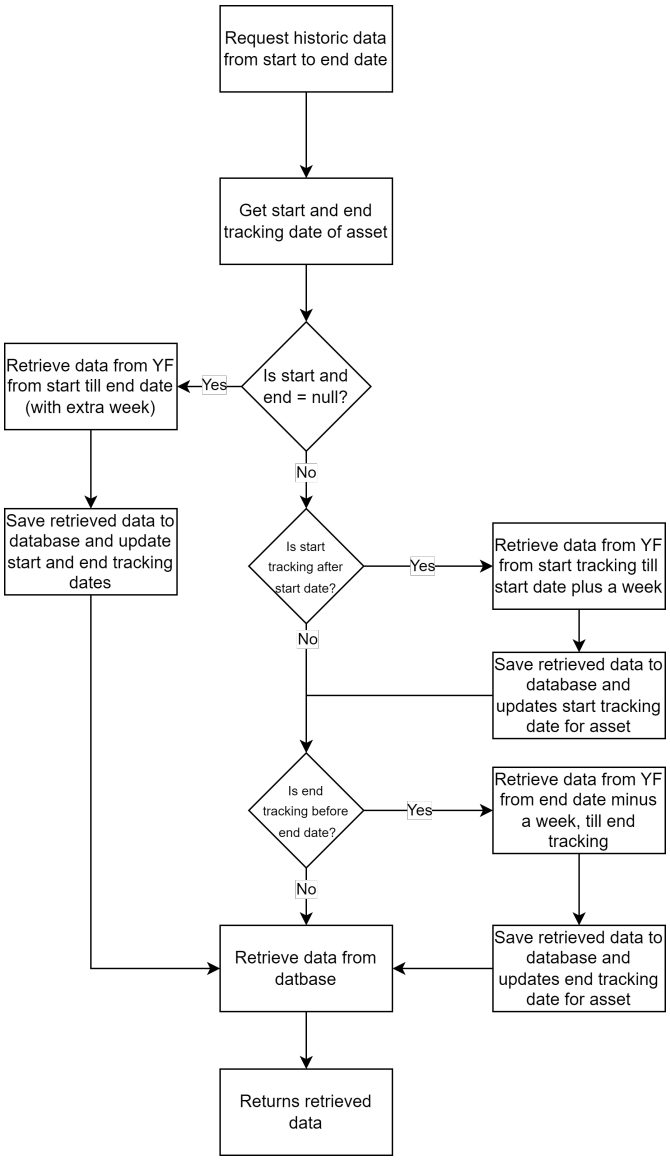


Figure 7: Activity diagram showcasing the flow of identifying missing data

On figure 7, we see a an activity diagram that showcases the flow of identifying missing data, to identify the need to call the Yahoo Finance API. It all starts out with a request of historic data of an asset. The fetcher is here called, that then calls the database fetcher, that gets a start and end tracking date from the database. When an asset is created in the database, say for example a stock, the stock is created with a start and end tracking date. These two values shows from which dates the database has coverage of the price of the stock. This can then be used in the

next step. Both of these values will either be null, if the database does not contain any historic data of the asset, or they are both set to a date each, as they will always be initialized together. Why this is certain, can be seen through the rest of the flow.

Moving further through the diagram, we first check if the start and end tracking dates are null, meaning the database contains no data. If this is the case, we retrieve the data from Yahoo Finance, by calling the Yahoo Finance fetcher. It will here retrieve from the start and end date that was specified, with the start date minus a week. By getting it a week behind the date that was actually specified, we can get around any errors that could rise from asking of data on a weekend, a holiday, etc. This way, we makes sure that Yahoo Finance will always return data, increasing the integrity of the data returned to the user. After this data has been received, the data is then stored in the database by the database handler, and the start and end tracking date for the asset is updated, to the dates that data has been retrieved from and to. After this has been done, it jump straight down to retrieving and returning the data from the database.

If however start and tracking date is not null, it first checks if the start tracking date is after the start date specified. If this is the case, we know that we are missing historic data and therefore calls the Yahoo Finance fetcher, to retrieve the missing data. Here, the flow is the same as specified before. If the start tracking date is not after the specified start date, then it checks if the end tracking date is before the specified end date. If this is the case, we know that we are missing newer data. Similar to the start tracking date, it here gets data from the end tracking date, to the specified end date and stores it in the database. It here ends with, whether the end tracking date is before the specified end date or not, retrieving the data from the database and returning it back to the fetcher, that then returns it back to the business logic.

4.3.3 Currency Conversion

Now that the flow of storing and retrieving data has been explained in section 4.3.2 Retrieving and Storing Data, we can now take a look at the added currency conversion to this flow. When storing and retrieving price data of different assets, that for example being the stock price or dividend payout, the price data will be in different currencies. For stocks, the currency is native to the exchange that the stock is registered on. This means, that a stock registered on NYSE (New York Stock Exchange) will always be listed in US dollars, where a stock registered on CPH (Copenhagen Stock Exchange), will always be listed in the Danish krone.

To simplify the process of converting currencies and storing exchange rate data in the database, it was chosen to always have the price data on a single currency - that being the US dollar. The reason behind always having the prices on the same currency is further elaborated on in section 4.10.3 Performance Analysis.

For the conversion itself, we can take a look at the section 4.3.2 Retrieving and Storing Data. Here, the flow of retrieving and storing data was explained and it is in this flow, that the conversion also comes into play. We can now take figure 6 and update it to also display the flow of conversion.

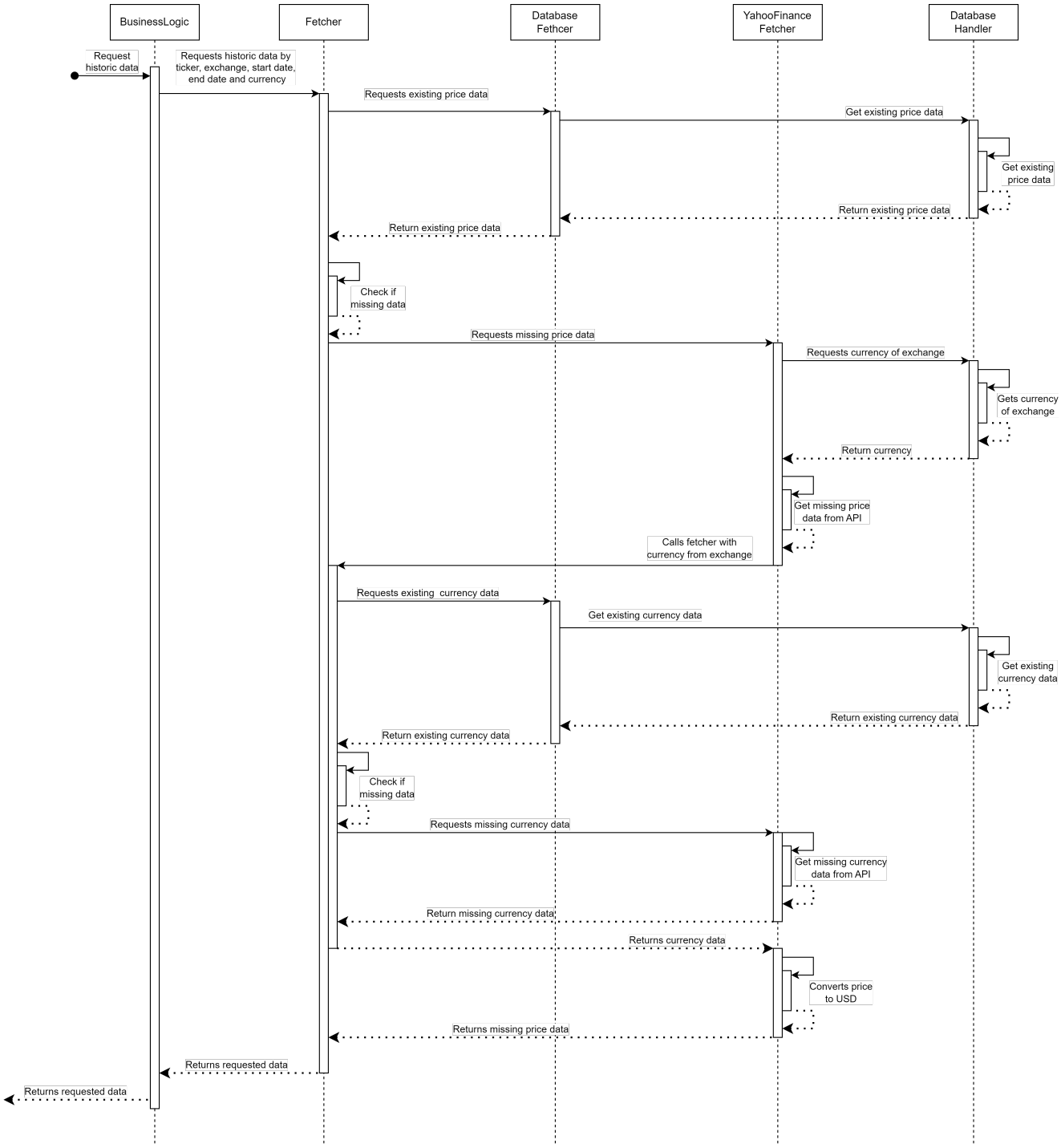


Figure 8: Sequence diagram showcasing the updated flow with currency conversion

As seen on figure 8, we have the updated flow from figure 6, where the added currency conversion is present. This here depicts the flow from getting a stock, that is in another native price than USD. The changes from the previous flow happens, where the Yahoo Finance fetcher calls the database handler, to find the currency of the exchange that the stock is registered on. When

gotten, it then gets the historic price data from the Yahoo Finance API, but then calls back to the Fetcher. The fetcher then goes through the same flow as getting the price, where it first gets the existing data, finds what is missing and requests what is missing from the Yahoo Finance API. The way it finds out what is missing is the same as the stock, as a currency is also fitted with a start and end tracking date. When gotten from either the database or the Yahoo Finance API, it then converts the prices and returns the missing price data to the fetcher, that then takes care of returning everything back to the business logic.

Every endpoint in the application follows this of retrieving currency exchange rate data and then converting the values. How the values are converted can differ depending on the object, if it is a price history, a single price of a transactions, etc. To follow this flow, every endpoint that gets value data to display for the user, is equipped with a query parameter "currency". The endpoints equipped with this, uses the parameter to get the currency exchange rates from either Yahoo Finance or the database. To make sure that the user chooses to convert to a currency that is able to fetch from Yahoo Finance, the currencies of Yahoo Finance was scraped from the website. Each of these currencies was then manually added to the "Currencies" table in the database, that stores all the different currencies that the application supports. If a user therefore gives a currency as a parameter that is not supported, the application sends an error to the user.

4.3.4 Error Handling

With the communication between the frontend and backend, it is vital that the backend always sends some kind of response to the frontend, as to make sure the frontend always knows if there is a problem and what kind of problem happened. To do this, it is needed by the backend to always return something. This way, the backend can return a meaningful exception to the frontend, with information about what went wrong. To do this, the backend utilizes a second middleware, being the error handling. What is different from this one to the authentication one, is that the authentication one handles any incoming requests, where the error handling handles any responses. The way it works is, that when an endpoint is done processing its data and it has to return it to the frontend, it is first processed by the error handling middleware. The middleware is build by try-catch statements, which try to catch any exceptions that has been thrown.

As to how any errors are found and communicated to the frontend, the backend utilizes a custom exception called "StatusCodeException". This exception is in charge for all exception communication with the frontend. StatusCodeException is created with a message and a status code. The status code is whatever HTTP status should be send to the frontend. The message is what should be send with the HTTP status. This means, that whenever an error or exception happens in the program, this can both be a database error 500, or a wrong password HTTP 401, a StatusCodeException is thrown. This exception is then handled in the error handling middleware. This middleware catches any StatusCodeExceptions and forwards them to the frontend. As a safety net, the middleware also listens for a common "Exception", which returns a status 500 to the frontend. This is to ensure, that if anything happens that is not controlled by the StatusCodeException, the frontend still gets noticed about an error has happen.

This whole flow with the StatusCodeException can see illustrated in the below sequence diagram, on figure 9, that depicts the flow of a user, trying to sign up with an email that already exists.

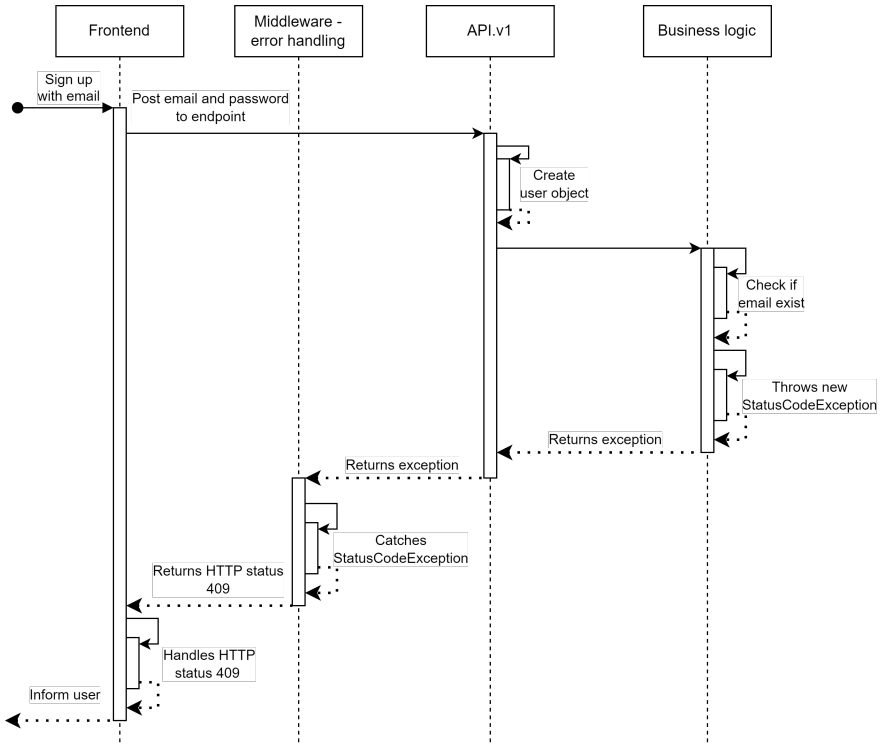


Figure 9: Sequence diagram depicting the flow of the "StatusCodeException" exception

The StatusCodeException itself is created by inheriting the base class "Exception", which implements all the needed constructors for an exception, to for example store a message.

By implementing the use of having a centralized error handling middleware and a single custom exception to forward errors to the frontend, we get a much more readable flow. We get this as we now know that every time a StatusCodeException is thrown, it is supposed to forward an error to the frontend. Any other exception would then be for the application to handle in try-catches. By utilizing this flow, we makes sure that the frontend is always getting a response of some kind.

4.3.5 Transactions

Transactions in the application, refers to entries in the database, that contribute to the overall balance of a portfolio, and a total of everything a user owns, combined. The application has three different transactions: a stock transaction, cash transaction and a dividend payout.

Stock transaction
A stock transaction is when a user "buys" or "sells" a stock. This is identified if the amount of stocks is either positive or negative. The stock transaction is also given in a type of currency, as a user can have chosen to "buy" a stock in a different currency, than the exchange specific currency.

Cash transaction
A cash transaction is when a user either "deposits" or "withdraws" cash to their portfolio. This is defined by the amount of cash being either positive or negative. A cash transaction is also identified by the currency, as a transaction can happen for all types of supported currencies.

Dividend payout
A dividend payout, is different from a stock dividend. A dividend payout refers to the transaction of a stock dividend. Say for example, that a user buys a stock that pays monthly dividend. The months after when the user is then owning the stock, whenever dividend is registered and added to the database, a dividend payout transaction is then added to the portfolio, in which the user owns the stock. This keeps track of what dividends a user should have gotten, adding to their overall cash balance.

The stock and cash transactions is both equipped with information about the transaction in the native currency and in US dollars. The reason behind keeping the amount in the native currency, is for the ability to display it correctly on the frontend, as this will translate to the exact amount a user bought, sold, deposited or withdraw with.

By having all of this information about the individual transaction, it makes it easier to find out the complete balance at any given point for a portfolio or totally for a user. By utilizing a view on the database that combines all the transactions into a single list, described in detail in 4.11.4 Database Objects, we can significantly easier sort them by date and portfolio, to get the total balance of cash for a given portfolio, at any date.

4.3.6 Value History

Value history is a centralized endpoint, that combines all the assets, portfolios and cash balance that a user owns and gives a complete view of a users total owned assets and their historical development. This includes the individual dividend payouts of a stock too, by both showing the dividend, but also including the dividend payout transaction in the cash balance flow. The endpoint can be called with a start date and end date, and then what currency the user will like returned. The way it does this, is by looking through every single portfolio of a user and then every single asset that they own. Each of the assets will then go through the flow, explained in section 4.3.2 Retrieving and Storing Data and section 4.3.3 Currency Conversion, and be added to separate list under the portfolio, or portfolios, that they are part of. Apart from these, a total cash balance is also added. By utilizing the view on the database, a total list of transactions are gone over for every portfolio. To make sure that the values can be translated straight into a graph, cash balance entries are made for every single date, where it then increments or decrements every time it gets to a new transaction. Another list at the bottom of the the response is a sum of all the assets and cash balances of the specific dates. This gives the total historic development of the sum of all assets, alongside their cash, of a user.

The reason behind creating this centralized endpoint, is to make the process of adding more assets easier and to make the flow of retrieving this data more straightforward. To have a centralized endpoint that takes care of every single asset all at once, it is only on singular files it is required to add the retrieving on new data, where the value history endpoint then takes care of summing everything. The flow here becomes much more straightforward to understand, as the value history takes care of calling the necessary methods that are needed to retrieve the data of all owned assets.

4.3.7 Routing

The backend is a REST API, meaning that there needs to be a centralized location, where all the different endpoints and the routes to them are handled.

When the application is started, it runs the method ”Main”. This methods only purpose, is to call the setup of the application. The ”Setup” method of the application, is a centralized method that handles everything that got to do with setting up the application. This includes creating the web application itself and setting up each of the endpoints and their routes. Here,

every single endpoint is equipped with an individual "Setup" method, that refers to the HTTP method of the endpoint, GET, POST, PUT or DELETE and the route of the endpoint, so anything after the "/". The full list of endpoints will be discussed in the section 4.8 The REST API Structure. To have each of the endpoints handle their own paths and methods, it simplifies the process of changing any of them, as they will be located in each of the respective classes, that refers to the specific endpoint. Before an endpoint can be called though, it has to get through multiple layers of the routing, depicted in figure 10.

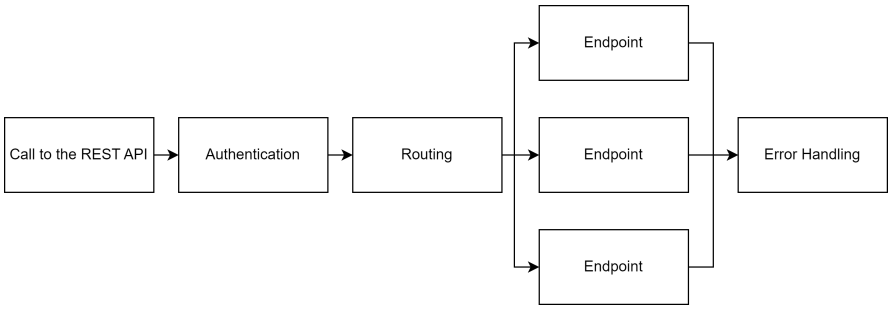


Figure 10: Flow diagram showcasing the layers of routing

On figure 10 we see the visualization of the layers of the routing of the application. As seen here, a call is created from the frontend. It then has to get through a layer of authentication, before the actual routing through the application happens. When each endpoint is then done processing, it has to go through another part of the middleware, which is the error handling.

In the "Setup" method of the application, it is also here the middleware is added, adding to the security and stability of the application and its routing. Now, the middleware authentication makes sure that an endpoint can not be called by just anyone. Being setup in the "Setup" function of the application as well, the middleware authentication is added to the web application through the build-in functionality of the ASP.NET Core framework. The middleware takes care of the authentication of the application. The middleware authentication will be explained in depth the section 4.7.1 Authentication. If a call got through the authentication of the middleware, it can then start the routing of the specific call. This is seen on the figure, as the middleware is the last piece the call has to get through before the routing.

If a call gets through the authentication middleware, the endpoints can then do their own individual processing of the data. Before it is returned to the frontend however, it goes through the second middleware, that being the error handling. The error handling is explained in section 4.3.4 Error Handling. What this does for the routing is, that makes sure that something is returned to the frontend. Even if an error happens, the frontend will get notified about it. If nothing happens and the data processes correctly, it can then get through the error handling middleware and be returned to the frontend.

4.4 Business Logic

The business logic is the core foundation and functionality of the application itself. It is therefore a vital part, as it provides the ability to do data processing when calling an endpoint. The business logic is responsible for ensuring data accuracy and consistency. There are multiple component that makes that happen across all the different data types handled in the application.

The business logic uses a general way of checking and handling wrong or missing inputs. Each method is equipped with individual checks if any of its required values are missing and then throwing a StatusCodeException HTTP status 400 back to the frontend. The reason behind them individually checking for missing values is both for making them independent from all other functions but also to check that parameters and variables for objects are being set correctly. By having each method checking for missing required values it can be handled accordingly and will give more information for troubleshooting the issue. Another error handling method is the class ValidCurrency with the method Check() in the Tools namespace. This namespace contains any generic utility functions that is used throughout the application. Specific for the ValidCurrency class, the method Check() is used to check if a currency is a supported for the application. Any method that has a currency as a parameter is also included with a validity check of this currency. If it is not a supported currency a StatusCodeException with the message "Invalid currency: 'currency'" is thrown back to the frontend with the HTTP status code 400 for bad request.

Figure 11 below, showcases the class diagram for the business logic for our application. It showcases the different classes, their variables and methods, and how they all are connected together. We will now be going over each of the classes, and discuss their functionalities and how they play into the rest of the application.

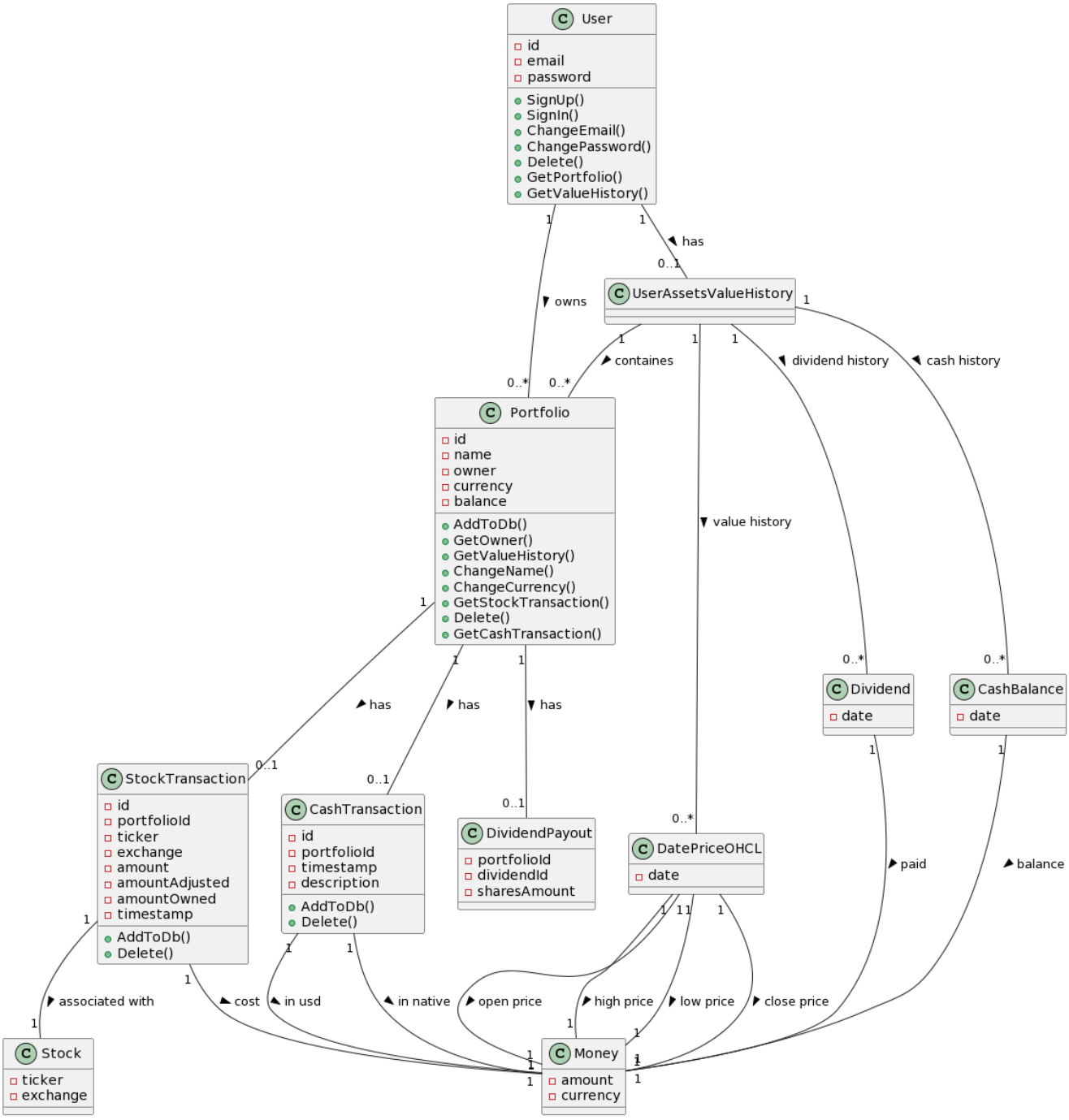


Figure 11: Class diagram for the business logic

4.4.1 User

The user contains three variables; id, email, and password. When **SignIn()** is called on the user, email and password must have a value, while the id is allowed not to be instantiated. This function attempts to retrieve the user from the database, if successful it checks if the password is correct and if so, the id is set to the one found in the database and finally returns a self reference. If any of this fails it throws a StatusCodeException with either "No user with the email: 'email' was found" and a HTTP status of 404, or 'The password is incorrect' and a HTTP status 401.

From inputs and outputs **SignUp()** looks like SignIn(). This function generates a unique user id after which id, email, and password is inserted into the database and returns a self reference. If a user already exist with the same email a StatusCodeException is thrown, with the message "User with email: 'email' already exist" with a HTTP status 409. It is important to mention, that the password is not inserted in plain text, see section 4.7.3 Password Handling

ChangeEmail() updates the user's e-mail in the database. Since emails has to be unique this can result in an error with a StatusCodeException, if it already exists.

ChangePassword() takes a new password and the old password. If the old password is correct, it is changed to the new one. If it is not, a StatusCodeException is thrown with the password being incorrect, and a status 401.

Delete() removes the user from the database which cascades to all portfolios, tokens and user preferences linked to that user.

GetPortfolio() takes a portfolio id and returns an instance of Portfolio if the portfolio with that id belongs to the user. If not, a StatusCodeException status 404 is thrown.

UpdatePortfolios() fetches all portfolios owned by the user and store them in a list on the user object. It does this by retrieving all portfolios with the user as its owner, looping them over and adding a portfolio object for each of them to the list.

GetTransactions() gets all transactions (stock transactions, cash transactions and dividend payouts) associated with all portfolios, that the user owns. This is therefore a complete list of all transactions, across all portfolios of the user. These different transaction types are standardized from the database. Once fetched from the database, a list of standardized "Transaction" objects is created. This objects holds information that is common for all the transactions, to put them into one complete list. Each of these transactions, is added with three different balances. The value, balance and combinedBalance. The value refers to the price of the transaction itself. The balance is taking into account all of the transactions of a portfolio, adding them together in chronological order. This is to showcase, what the total balance of the portfolio was at a certain transaction. The combinedBalance is then accross all portfolios, to showcase the complete value of a user at a given point in time.

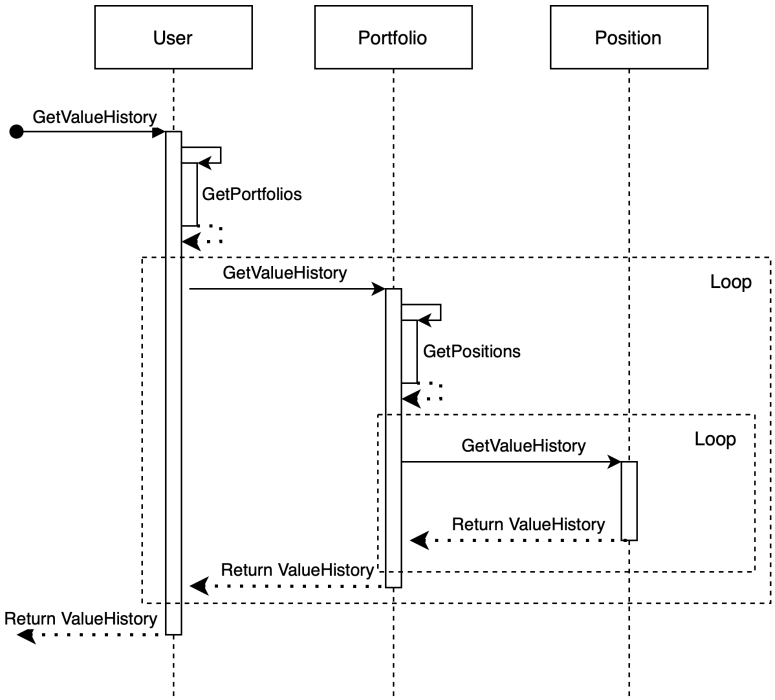


Figure 12: GetValueHistory sequence diagram

GetValueHistory() gets a list with an entry for each day between two dates provided to the function. It also takes a currency parameter, for which currency the value history should be given in. The value history refers to the centralized flow explained in the section 4.3.6 Value History. First UpdatePortfolios() and GetTransactions() is called. A for-loop goes though the list of portfolios and calls a function of the same name (GetValueHistory) on each of those. Each time the result is added to an ongoing "sum" which is returned once done. The data is saved in the UserAssetValueHistory class, that holds information about the total price history of all stock positions, the dividend history of the stocks, the total list of portfolios and the cash balance history, which is the information about the cash a user has at a given point in time. Each of the GetValueHistory functions, gets the value history for the individual portfolio, but also stock positions. This way, we have both the total history of a singular stock, but also across all of a portfolio and across all of the user. This is used to display historic development on the frontend, for a user to get an overview of their total value history development. Figure 12 show this flow.

4.4.2 Portfolio

A portfolio is made up of an id, name, owner, and currency. **AddToDb()** mirros SignUp from user. It takes all values from the portfolio except id, which is generated by a random string function, for then to insert all the values into the database.

GetOwner(), ChangeName(), ChangeCurrency(), and Delete() are all simple functions that all do simple SELECT, UPDATE, or DELETE queries to the database. They work the same as the changing functions from the user, by simply updating in the database, where the portfolio id is equal to the id.

GetValueHistory() is the function mentioned earlier that is called from the user. As explained, the function gets the value history for the user as a whole, but also for the individual parts, such as the portfolio here. The portfolio calls the functions UpdateStockPositions() and UpdateStockTransactions(), to get an overview of what is currently owned on this portfolio. It does this by simple select statements to the database, and getting all the stock transactions for a portfolio, and then also the distinct tickers and exchanges, to get every singular position the portfolio has. Utilizing the same flow as before, it loops over each of the stock positions, that gets each of their value histories, and combines them into one single list for the portfolio, to have a complete overview of the historical devlopment of assets for a specific portfolio.

4.4.3 StockTransaction and CashTransaction

These two classes serve as data structures. They hold information such as id, timestamp and different price values (The exact values can be seen in figure 11). They both have AddToDb() and Delete() functions which do a INSERT and DELETE query to the database, respectably. The StockTransaction class has a more complex AddToDb() function however, as this has different errors and parameters to take into account. While the cash transaction is rather straightforward, by simply adding the price to the total balance, the stock transaction has to keep track of the total amount owned of this particular stock, while taking stock splits into account and adjusting the amount accordingly. It does this by getting the newest stock transaction of this specific stock, on this specific portfolio. It gets the amount owned from this transaction, to then add its own amount to. However, it also has to account for splits. It therefore selects every split that has happened for this stock, by getting the ratio_in and ratio_out values from the stock splits table, where the timestamp of the split, is newer than the inserted stock transaction timestamp. By doing this, we get every split that has happened after the transaction. The ratio_in (the amount of stocks that is split from) and the ratio_out (the amount of stocks after the split), is then looped over. Every ratio_in is then multiplied with every other ratio_in, and the same with ratio_out. They are then dividend with each other, which gives us the total ratio to multiply the amount adjusted of stocks with.

With the amount adjusted gotten, it is then added to the amount owned gotten from the database, before inserting the new stock transaction, with the correct amount owned, adjusted to the splits. Before adding it to the database however, a check is made to make sure, that the now new amount owned, is not less than 0. If that is the case, the user is owned negative amount of stocks, which is not possible. It will therefore throw a StatusCodeException with the error "Not enough owned stocks", and a HTTP status code 400, for a bad request.

4.4.4 Stock and Money

These classes are exclusively data structure classes. Stock hold two values, a ticker and an exchange in the form of a code for the exchange it is referring to. Money also holds two values, one for the amount and another for the currency code, so for example DKK or USD. The money object is used throughout the whole application, for an easy way to always hold information about the value itself, and what currency the value is referred in.

4.4.5 StockPosition

This class is mostly used for just storing the information about a position of a stock, on a specific portfolio. It has two methods, UpdateStockTransactions() and GetValueHistory(). **UpdateStockTransactions()** retrieves all stock transactions associated with the stock position from the database and saves them in an ordered list with dates in ascending order.

GetValueHistory() will get the historical value of the stock position for every day between two dates. First it calls the UpdateStockTransactions() function since it needs the stock transactions to calculate the value for each day. This is done by a for loop going from the start date to the end date, incriminating by one day each time. Before the loop start StockFetcher.GetHistory() is called to get the historical stock prices. Going through the loop the date is compared to the dates of the stock price history and the stock transactions. If the date for the stock transaction matches the amount owned after that transaction is updates and will be used until the next transaction match with the loop date. This is done with the stock prices as well. This means on closed days the value is unchanged from last open day. Aside from weekends, exchanges have closed days differently around the world therefore a solution that would fill in closed days was the simplest way to implement it.

How the StockFetcher retrives data from the database and the API is described above in section 4.3.2 Retrieving and Storing Data.

4.5 Software Architecture - Frontend

The frontend must be a user friendly design, so easy to understand and navigate. Its architecture plays a vital role in the overall user experience, as it allows for the user to interact with the backend through a visual design. It is therefore vital that the frontend implements the different endpoints of the backend. This was achieved by following several different UI concepts, alongside creating a frontend could fundamentally call and act on the responses from the backend.

4.5.1 Integration with the Backend

The frontend to backend integration uses simple https calls to access the API giving data access. The implementation also affects performance, security, and general usability. Data is cached to reduce the backend workload and response time. The authentication the backend requires is also handled here as well as any error the backend might return.

4.5.2 Caching

When the frontend requires a backend resource; the function called getUri() on the Backend-Handler class is called. First a cache key is generated from the URL and any parameters/body there is set for the request. Once the key is made a check happens to see if any data is stored locally for that key. If so data is returned. If no data is found the request is sent to the backend. Assuming the request was successful the BackendHandler proceeds to store that data under a key generated based on the URL including any parameters. The variable cacheCounter, that helps with creating a unique key for the cache, takes its value from memory as well, if it has been stored previously, otherwise it is 0. Then the key for the request to the backend will be stored under the key ”cache_{cacheCounter}”. Finally to limit the number of stored requests the oldest cached request is deleted if the number of cached request exceed the max cache size. The key for that information is found by reading the key stored under ”cache_{cacheCounter - maxCacheSize}” if the cache counter is greater than or equal to the cacheCounter for the key.

This system ensures a low response time for resources and data the user will see often as well as lighten the load of the backend. The maxCacheSize ensures that all data will be updated from time to time since data requested is not moved up in the cache if it is already there.

When doing a POST, PUT or DELETE request for a resource the cache for the corresponding GET request will be deleted. One example is when the user adds a new portfolio to their account using the POST/portfolios endpoint. This means the current cache for the GET/portfolios request is invalid since the new portfolio is not in that list. Therefore the cache is deleted and will be fetched from the backend again when needed.

4.5.3 Error Handling

When the backend returns an error it will be handled by the BackendHandler. If the request fails and no information is given the request will be sent two more times, three in total, and if that fails the user is served a pop up notifying them about the situation.

There are four error related to authentication, described in section 4.7.1 Authentication. When getting the "access token expired" error a request for a new token set is sent with the refresh token. With the new access token the original request is sent again.

When getting either "access token invalid", "refresh token invalid" or "refresh token expired" as the error the user will be logged out. This is because an invalid token, either the access or the refresh token, means the whole token family is invalid and the user will need to provide their login credentials to receive a new token set. This is also the case for when the refresh token expires since logging in is the only way to get a new refresh token without an existing valid refresh token.

Sign up and Sign in
In the process of signing up and signing in a range of errors can occur. Some of these error is returned from the backend when attempting the sign up/in. Others is defined by the frontend. The errors coming from the backend will be in a form of any other status code than 200, alongside an error message.

Errors that can be returned from the backend is as follow

- User already exist - status 409
- User does not exist - status 401
- Incorrect password - status 401

Each of these error will be shown as a pop up for the user to dismiss and fix the mistake in the form.

The sign up implements errors for the purpose of letting the user know the requirements for e-mail and password complexity. The password must be between 8 and 32 characters and contain at least one uppercase, one lowercase, and one numerical character. Violations of these guidelines results in an error text shown in red under the input field with the error. If the user should click the sign up button with an error still in the input fields a pop up will let the user know about this.

Besides the errors returned from the backend, the only errors that can happen on the sign in page is for empty fields which will be a pop up as described above.

4.5.4 Application Flow

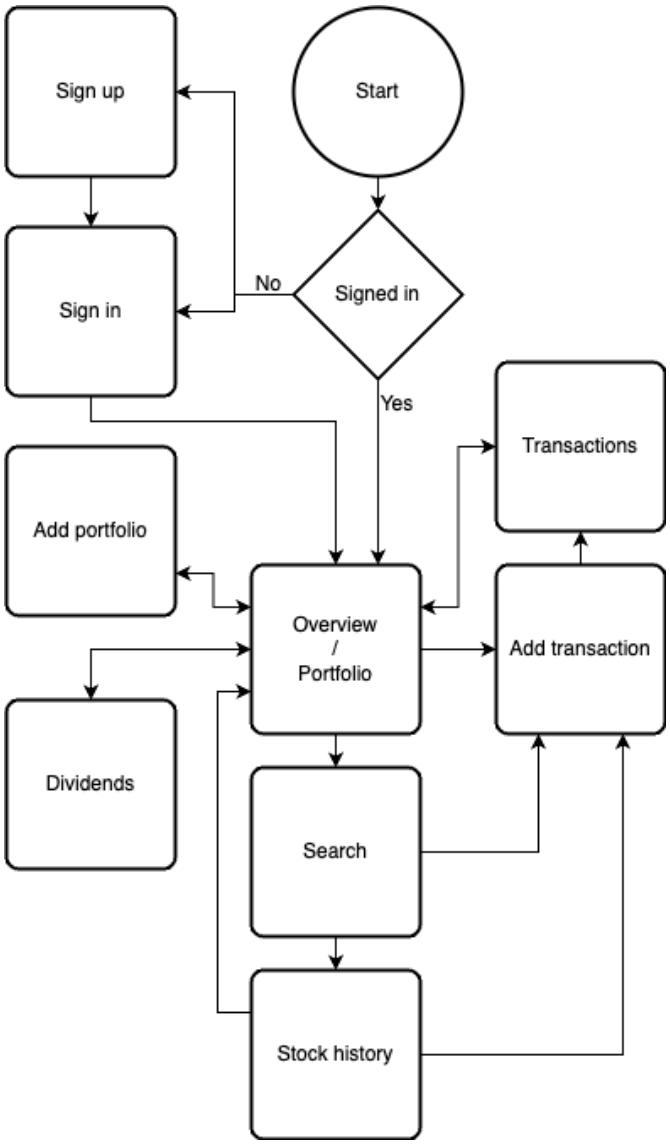


Figure 13: Application flow diagram for the frontend

When starting the application if the user is already signed in from an earlier session, the user is navigated to the overview page. Otherwise to the sign up or sign in page. If the user signs up, the user will be navigated to the sign in page. Once signed in the user is navigated to the overview page. From here the user can get historical information about their portfolios as well as a basic overview of their dividends. It is also possible to search for stocks and from those result go to the stock history page. The add transaction page can be access from multiple places. It can be done from either search or the stock history which result in the "add transaction" form will already be filled with the information specific to that stock. It can also be accessed from the overview page from where the form will initially be empty. Once a stock is added the user will land on the transactions page giving an overview of all transactions the user have done. From here the user can go back to the overview.

4.6 UI Design

The design of the application was aimed to be as user friendly as possible. This would help in the overall quality of the user experience, if the design is easy to understand and navigate. To do this, we implemented two concepts of UI design. The first being the concept of Steve Krug "Don't Make Me Think" [8]. This concept is the basics of UI designing, by having in mind that the user should never think when using your design. This is often referred to "A button should look like a button". The second concept being the concept of "Cognitive Load Theory" [1]. Cognitive Load Theory is a UX concept, that takes into account the mental effort it takes for a user to process information and decision making. This is achieved by minimizing the cognitive load, by using familiar visual cues and objects in your design, that is generally used across websites. It can also be minimizing by organizing information in a manner, that is understandable, so for example to enclose information in a box, if it is information that should be taking in together. This all minimizes the mental load when using your design, meaning the user will get a generally better experience, as they do not have to put a strain on themselves mentally to navigate and understand your design.



Figure 14: Sign in page

The first figure 14 is the sign in page. This is the first page you see, when starting up the application. It has a vibrant and colourful background, with a clear input box that takes the necessary information to sign in. Underneath it is a sign up instead button, separated from the rest of the box, to clearly indicate that it does something different than the sign in box.

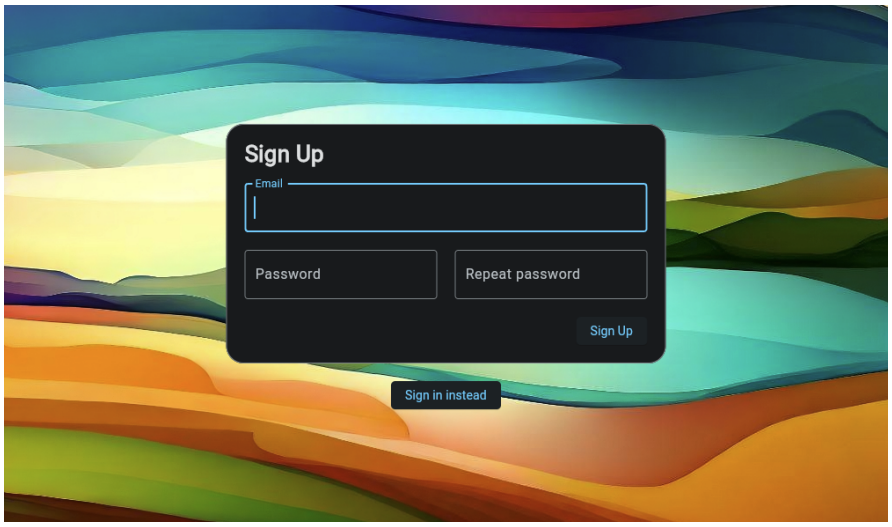


Figure 15: Sign up page

Figure 15 showcases the sign up page, if you click the sign up button on the previous page. The design is similar to the sign in page, but here the highlight of selecting an input field is also shown.



Figure 16: Overview of a stock

Figure 16 is the overview of a particular stock. We get here, by searching up a stock in the top right corner and selecting one of the stocks from the least beneath it. Here we have selected the stock Realty Income. We then have the graph in the top, that showcases a year worth of historical data. The buttons as the top gives the user the ability to switch between one week, one month, half a year, from the start of the year to this day, one year and five years back. To the right of the graph, is the ability to change the currency. The "USD" here refers to the currency of the exchange the stock is registered on, while the "DKK" is referring to the user preferred currency, chosen on the user preference page. Beneath the graph is the bar-chart, that showcases the dividend payouts of the stock, alongside the date it was paid. Realty Income pays the same dividend every month, which is why every bar is the same here.

On the bottom we have the company information. This is all the information we have about the company of the stock, such as the name, website, country, financial currency, etc.

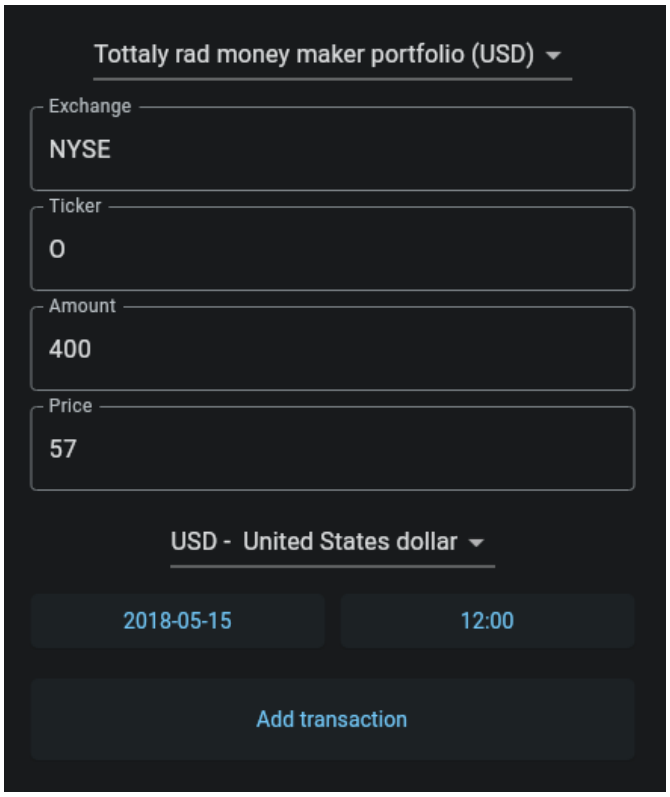


Figure 17: Page to create a stock transaction

On figure 17 we have the page for creating a stock transaction. The creating a cash transaction page is very similar. Here we have a dropdown to select what portfolio to add the transaction to, the exchange, ticker, amount of stocks and what the user paid. Another dropdown at the bottom then showcases the supported currencies, that the user can choose for the price. The user can then choose a date and a time, for when they created the transaction.

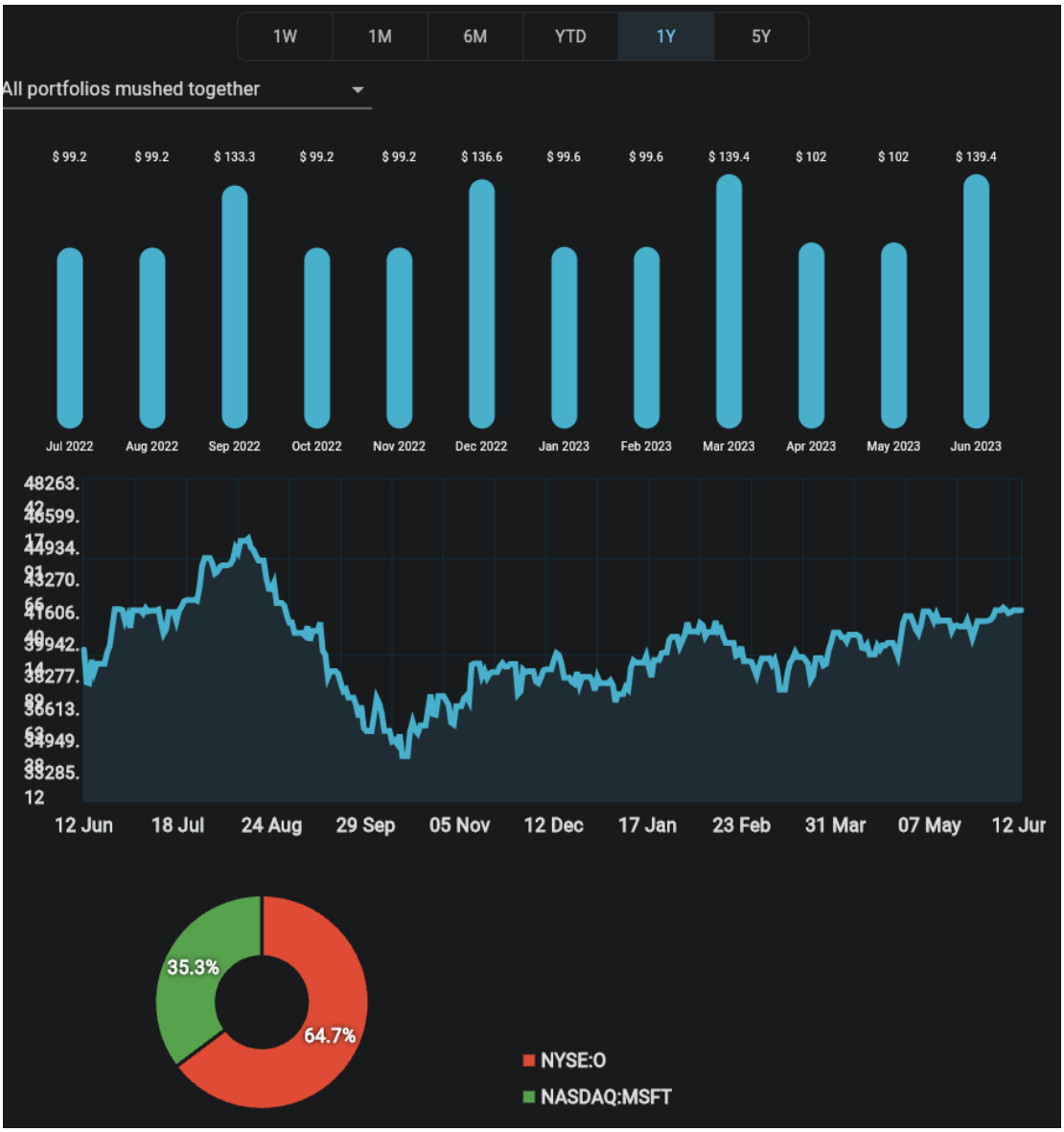


Figure 18: Overview of all portfolios of the user

On figure 18 we have the total overview for the user, that showcases all the portfolios and transactions together. It is very similar to the individual overview of a stock. This particular user has bought two different stocks, which is showcased on the pie-chart. Here we see the total percentage, of how much of the total asset value, each of the stocks are taking up. Both stocks pays dividend, which gives us a more realistic looking bar chart, that changes every month.

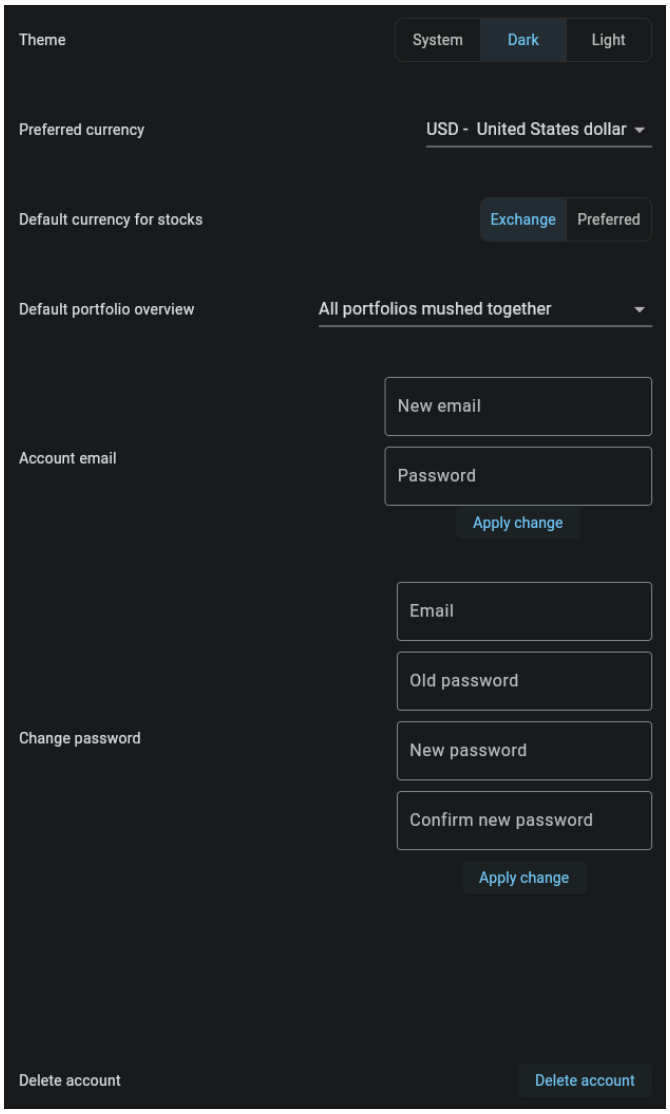


Figure 19: User preferences page

On the figure 19, we have the last page of the application. This is the user preference page, where the user can choose what theme they want, the default currency for the application, change email and password, etc. This is the general user hub, where a user can take define their preferences.

4.7 Security

The security measures are a crucial application part designed to make the user feel safe, while upholding the reliability of the application. The security of the application should follow a centralized path, so it easily can be implemented across the application. This ensures and enforces new application implementations follow the same security measures.

4.7.1 Authentication

An authentication layer ensures users can only call an endpoint when logged in. The authentication is introduced in the middleware. Every time an endpoint is called, it needs to be authenticated before it can reach the business logic part of the application. See figure 5.

Built on ASP.NET Core, the application uses the build-in functionality of middleware. The middleware was added at the setup of the application, where the web-application, the REST-API of the ASP.NET Core framework, was specified to use middleware. The middleware here refers to a separate class that implements the IMiddleware interface. This interface is part of the ASP.NET Core framework, and implements the necessary functionality for the class to become middleware for the application. The middleware here, expects an access token in its header, under the name "Authorization". This access token is then authenticated. If the authentication fails it will then either return a HTTP status of 401; "Unauthorized", if the access token is invalid, or a 403; "Forbidden" if it is expired. The 403 response lets the frontend know, it has to refresh the tokens, as this can only happen if the access token has expired.

Using middleware authentication creates a centralized functionality, reusable across the entire API. It has only to be implemented once, instead of at each endpoint.

The authentication system was created specific for this application. Practice and flow were inspired by OAut [13]. OAuth is one of the top authentication systems with a ton of complexity not needed for the application. Authentication and security were generally not the focus of this project, so an authentication system was created instead, with less complexity.

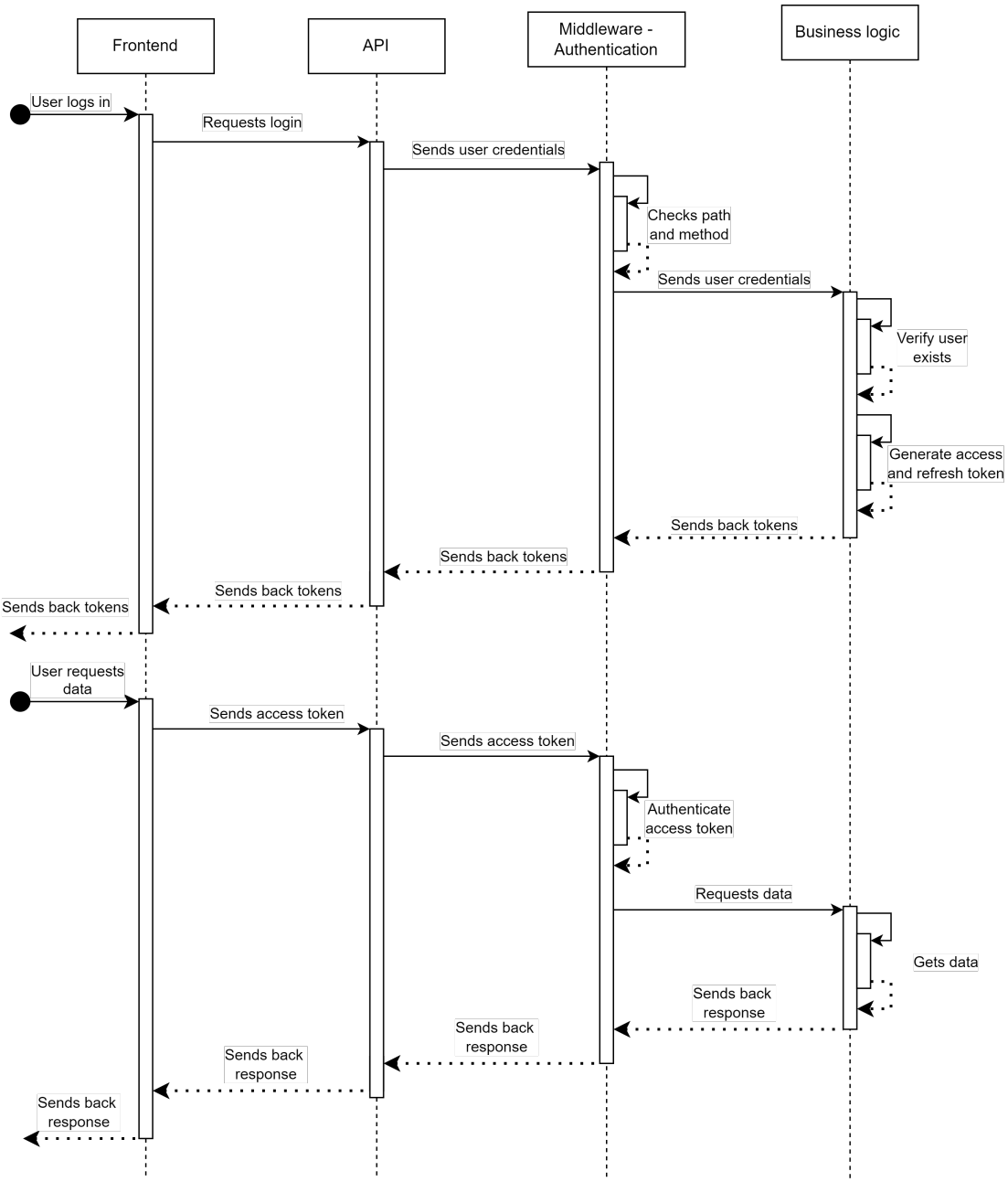


Figure 20: Sequence diagram of authentication flow

Figure 20 shows the authentication flow. The frontend calls the API for a user logging in, which sends the user credentials to the middleware. The middleware then checks if the path and method correspond to the path of a user logging in, meaning it has to ignore the authentication, as the user does not have any tokens. If so, the user credentials are send to the business logic, where the user is authorized. It then generates an access and refresh token, and sends these back to the frontend, through the middleware and the API. The user can then use the access token to call an endpoint and request some data. The access token in the "Authorization" header, is then send to the middleware. The middleware then authenticates the token, to make sure it is

valid. If it is, the request of the HttpContext is then send to the business logic, and returns the requested data back to the user. With the flow explained, lets now quickly go over what an access and a refresh token is.

An access token is a unique string of characters, that upon authentication, serves as a user credential, allowing the user to access the endpoints of the API. Access tokens are short-lived and when expired are no longer able to authenticate the user. To obtain a new valid access token, without the user having to login again, a refresh token is used instead. A refresh token is, as the access token, a unique string of characters. The refresh token has a longer lifespan and is used to generate a new set of tokens, when the access token has expired. If the refresh token expires or becomes invalid for some other reason, the user will be required to login again. The two token combination minimizes the risk of unauthorized access and exposing sensitive user data.

Token authentication uses two techniques, "Refresh Rotation" and "Reuse Detection" [14]. When a set of tokens is generated on login, a token family is created too. This refers to 4.11 Database Design, which mentions the table of TokenFamily. The token family table stores the id of the token family, associated user and valid access token. All tokens are stored in the Tokens table, with an associated family. When someone tries to use an access token, it is first checked if it has expired, but also if the family is valid. If the access token has expired, the frontend is prompted to refresh the tokens, using the refresh token, which creates a new refreshed set of tokens. This flow is done automatically on the frontend with the refresh token, where the tokens are refreshed. However, if the access token is not the valid token of a family or if the family is invalid, which requires the user to login again.

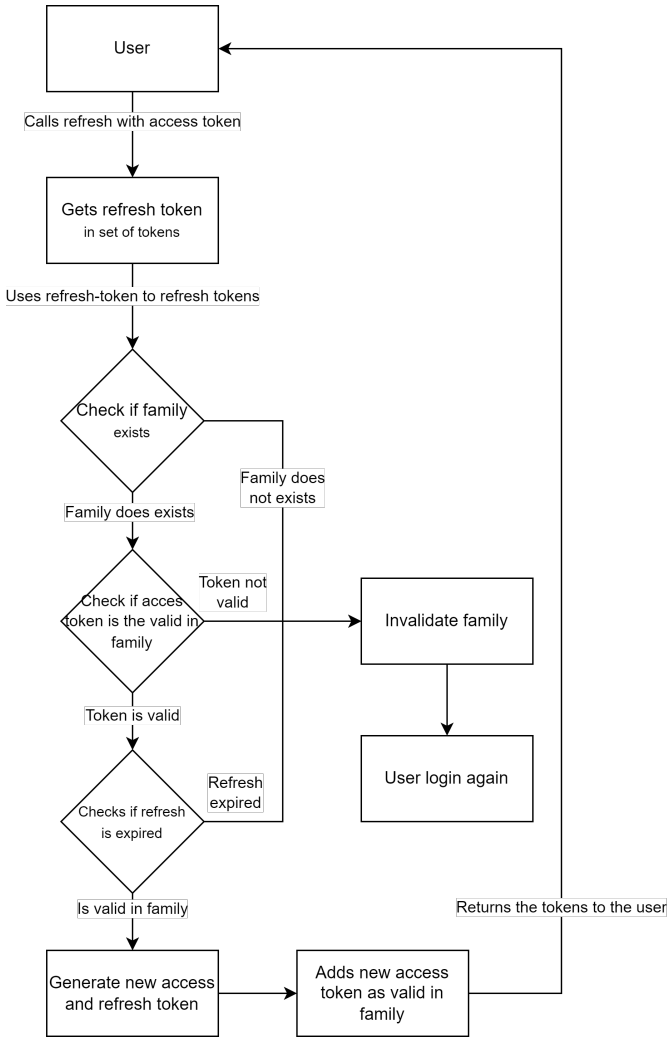


Figure 21: Flow diagram of the flow of the refreshing of tokens

Figure 21 outlines the flow of refreshing of tokens. The flow incorporates the reuse detection and refresh rotation for extra security. When the authentication middleware is called to refresh a token, it first checks a family's validity by checking if it exists in the TokenFamily table. Then the valid refresh token from the family is compared to the refresh the user provided. Last is a check if the refresh token has expired. If all checks pass a new access and refresh token are generated via the refresh rotation.

When a refresh token is used it becomes invalid. This ensures if another user would try to use it either before or after the legitimate user have refreshed it, that entire token family is invalidated which would require the user to sign in again. This would quickly block out a bad actor who might have gotten a hold of a refresh token. When the bad actor uses the refresh token to get a new access token, the refresh and access token the real user holds, becomes invalid. However, the real user will try to use the access token, but as the access token is not valid anymore, the family becomes invalidated, rendering the tokens the bad actor is holding invalid and prompting the user to login again, generating a new set of tokens.

The authentication can forward five different errors to the frontend as part of its 401 or 403 unauthorized or forbidden status:

- Access token is expired - 403
- Access token is invalid - 401
- Refresh token is expired - 401
- Refresh token is invalid - 401
- No token was provided - 401

These five errors helps the frontend determine whether it should seek to refresh the token or if the user should login again. The expired access token is the only error where a refresh can be done. The next three errors require renewed user login. The no token was provided error, simply checks for a token provided in the header.

4.7.2 Authorization

Authorization follows an approach similar to authentication covered in section 4.7.1 Authentication. Once the access token has been successfully authenticated, the token is passed to the individual endpoints through a HttpContext, as explained in section 5.2 Middleware - Authentication.

Within the individual endpoint methods, where authorization is required, the application attempts to create a user based on the access token. This is done by utilizing the "User" object and querying the "Tokens" table on the database, to find the corresponding access token. Upon successful retrieval, the user id associated with the token is returned and used to create the "User" object.

The "User" object is then utilized to perform the requested action, such as retrieving a specific portfolio owned by the user. If the requested id does not exist, an exception of status 401 is thrown to the user. This indicates either an incorrect id or an attempt to access a portfolio that does not belong to the user.

4.7.3 Password Handling

Passwords might be the most sensitive information we will handle for the user. For that reason it is vital that we're careful about how we send and store this information. Our goal is to limit when any information about the password is exposed and opportunities to abuse how it is sent and stored. There is four points to consider where the password passes through. These are the frontend, transfer to the backend, the backend and finally the database.

By hashing the password both on the frontend and the backend there will be nowhere outside the client program that can see the actual password. By hashing the password on the backend the original password is protected against a database breach. Hashing on the frontend mainly protects against mistakes that could happen on the backend [9]. Two examples of possible mistakes could be logging the password in plain text [11] or exposing the password in the URL [18].

Before the frontend sends the password to the backend, the password will be hashed with a salt applied. The frontend uses the user's email as the salt so the function looks like Hash(email+password). This protects against reverse hash lookup where the hash of common passwords are compared to the users hashed password. The password is sent to the backend using the HTTPS protocol which hinders a third party from sniffing the data or act as a "man in the middle" to read data that way. The backend also hash the password and also adds salt.

This time using the unique user id making the function look like hash(userId + password). If the user is signing up the hash password is stored in the database, if the user is signing in it is compared to the saved one in the database. Doing it this way ensures even a database breach or a leak of stored passwords would effectively be useless information and therefore not present a risk.

4.7.4 General Data Protection Regulation (GDPR)

The General Data Protection Regulation (GDPR) is a comprehensive data protection and privacy regulation implemented by the EU. It sets out a strict set of guidelines and rules for applications collecting, processing, and storing personal data of the people within EU. For the application, data regarding GDPR is user's e-mail, and password. However, the user information is regarded non-sensitive, hence no extra security measures are needed. The application still has to follow the guidelines. Sensitive information refers to health-related data, genetic data, racial or ethnic data, etc. This is explained on the GDPR website [2].

An official template on the GDPR website [17] lists questions to different topics. Following these guidelines the application should follow GDPR rules.

What data do we collect?
User's e-mail and password. For the individual using the tracking application for assets they own in real life, the individual's financial data will also be stored.

How do we collect your data?
Only voluntarily user data is stored. These are given through signing up, creating portfolios, and registering different transactions. The application will not be collecting user data from any third-party companies/systems.

How will we use your data?
Data is used purely by the application itself for the tracking of financial assets data, and not by any third-party systems. The application will not use the data for any marketing, or forward-selling to any third-party companies.

How do we store your data?
The application securely stores the data at the cloud platform Microsoft Azure. Microsoft Azure takes care of securely storing the data, as they follow their own regulations and precautions. The application stores the user provided data as long the user account is active.

What are your data protection rights?
The individual always has the ability and right to delete any portfolio, transaction, or user they may have registered. On deletion data will be removed completely from the database, giving the individual complete control of their own data. This is on the condition, that the application needs data to process and work adequately. The application does therefore not take any responsibility, on deleting user data and the application not working correctly or as intended. As the application does not use the data for marketing or forward-selling, the individual does not need rights for choosing not to port, sell or forward their data. The individual will always have the right to access their own data, to revoke it from the database if so chooses.

What types of cookies do we use?
The application uses caching and keeps everything that can improve the quality of the user experience, for example the latest searches or latest shown stock prices. Caching does not save any direct user information, apart from the access and refresh token.

4.8 The REST API Structure

In this section, we will be looking over all the endpoints of the REST API. This will cover the path of the endpoint, the method and a short explanation of what they do. For a view of the full REST API schema with body structure, query parameters, responses, etc., for each endpoint, we will refer to the appendix 11.2 Appendix 2 - JSON schemas of the REST API.

To follow the rules of REST, each endpoint corresponds to a resource of the application that when a client requests a resource, the application sends back the current state of the resource in a standardized representation, which in this case is JSON format. For all of our endpoints, except for /v1/users POST and /v1/tokens POST, it is required to include a header of "Authorization" in which the access token is supplied.

Path	**Method**	**What it does**
/v1/users	POST	Creates a user on sign up
/v1/users	DELETE	Deletes a user and everything associated with it
/v1/users/email	PUT	Changes a users email
/v1/users/password	PUT	Changes a users password
/v1/users/preferences	GET	Get all of a users preferences/settings
/v1/users/preferences	POST	Creates or updates a users preference/setting
/v1/users/preferences	DELETE	Deletes a specified preference/setting of the user
/v1/stock-profiles	GET	Gets a stock profile, like company information
/v1/stock-histories	GET	Gets a list of historic prices and dividends
/v1/portfolios	GET	Gets all portfolios that is owned by a user
/v1/portfolios	POST	Creates a new portfolio with a user as its owner
/v1/portfolios	PUT	Changes a portfolios name or currency
/v1/portfolios	DELETE	Deletes a portfolio that is owned by the user
/v1/search-results	GET	Gets list of stock profiles, based on a given search query
/v1/stock-transactions	GET	Get all stock transactions of a user
/v1/stock-transactions	POST	Creates a stock transaction for a user in a specified portfolio
/v1/stock-transactions	PUT	Changes certain values of a stock transaction
/v1/stock-transactions	DELETE	Deletes a specified stock transaction in a portfolio owned by the user
/v1/cash-transactions	POST	Creates a cash transaction for a user in a specified portfolio
/v1/cash-transactions	PUT	Changes certain values of a cash transaction
/v1/cash-transactions	DELETE	Deletes a specified cash transaction for a user in a specified portfolio
/v1/transactions	GET	Gets a complete list of all types of transactions for a user
/v1/value-history	GET	Gets value history of a user
/v1/tokens	POST	Creates a new token set for a user, corresponding to signing in
/v1/tokens	PUT	Changes a users token set, corresponding to refreshing tokens

4.9 Technical Risk Analysis

For this section, we will be focusing on the risks that a user have, when using our application. This will be put against the impact of the risk itself, and at what likelihood the risk is to happen. These two values combined will let us know how to handle the risk, if there is safety procedures to be implemented to reduce them, or to completely avoid them. The likelihood can be rated 1 to 3, 1 being lowest and 3 highest, and impact can be 1 to 5, 1 being lowest, 5 being highest.

4.9.1 User Information Leak

As our application utilizes an online database, there will always be the potential of data possibly being leaked. We therefore has to assess the possibility of data being leaked, and what kind of data it is. When looking at the data that can possibly be leaked from the database, most of it is not important. The different price data, stock information, etc., is all publicly known data, which mean that a data leak will not pose a security risk. If a users stock information about their portfolio is leaked, the security impact will not be significant either, as the owned stocks for a portfolio may or may not reflect the correct owned stocks, of a user in the real world. This renders the data useless for any leak.

However, we do raise concerns of the Accounts table in the database. This table stores a users email and a password, which is sensitive user data and must be protected, from any unauthorized access. If this information were to be leaked, it could lead to unauthorized access to user accounts, and at worst leak email and password for a user, they also use on other platforms. Looking at a study from Google in 2019 [6], it is stated that 52% of the people taking the survey, reused their password for multiple, but not all, accounts, and 13% reuses the same password for every account. A leak of the password and email could therefore lead to multiple unauthorized accesses across multiple platforms.

With this we can say that the impact of this would be a 5, as it could potentially be critical for the user on, not just our application, but also others. The likelihood of it would be a 2. The reason for this likelihood, is the applications lacking of implementations of expert technologies. This refers to the self-made tokens system and password hashing, which could have used a professional system instead. With that being said, measures has been taking into account, if such a breach should happen. The act of hashing the password, means that a leak of the password of a user, could lead to useless data, as the password is in a non reusable form. By also using HTTPS to make secure connections between the frontend and backend, it also severely lowers the risks of the data being leaked. By implementing these measures, we can significantly reduce the risk of user information leakage, and enhance the general overall security of the application.

4.9.2 Incorrect Market Data

As our application relies on market data to provide correct, daily information to users, there is a potential risk of incorrect data being presented. We need to asses the impact and likelihood of such inaccuracy in the data, to consider appropriate measures to mitigate the risk.

When examining the data that could potentially be incorrect, we focus on the stock price data, currency exchange rates, dividend payouts and other financial information about the company itself. As we are using the Yahoo Finance API, it is always important to note, that the data is subject to potential errors or delays, that are beyond our application. With that being said, we have to take a look at the accuracy of the data, that we get from Yahoo Finance. Yahoo Finance is one of the biggest platforms for getting stock information. They get the information straight from the exchanges and other financial markets, as stated on their website [20]. This means, that we can rely on the accuracy gotten from Yahoo Finance. With that being said, lets take a look at the impact and likelihood of incorrect data.

The impact of incorrect data would be around a 4, where the likelihood sits on a 2. The reason the impact is, that even though a user should always do research and not take our application as investment advice, it is possible for a user to make incorrect investment choices, based in inaccurate data. This can also give inaccurate portfolio valuation, which can represent the historical development of the total portfolio worth incorrectly. This combined can also result in loss of user credibility for the application. For the likelihood sitting on a 2, is not because of Yahoo Finance giving incorrect data, but more missing data. This is a data integrity risk, which is covered in the next sub section. Yahoo Finance however, as they get information straight from the sources, it has a high accuracy in financial data. Where the inaccuracy is happening, is when converting from one currency to another. The currency data is still from Yahoo Finance, meaning they are accurate, but converting them can impose some inaccuracy. This can happen if not enough decimals is used, if converting from two currencies that has a massive difference.

To mitigate the risks, its important to make sure that the conversion of the currency is correct and is displayed correctly, with enough decimals to showcase a correct accuracy. It is also important to take into account development of currencies as well as the stock price itself. This is mitigated however, by using historical currency data, to correctly showcase the historic development of a stock, in a given currency.

4.9.3 Low Data Integrity

Integrity of the data is important for the application, to make sure that the user actually gets the requested data, in the correct format. As mentioned in the subsection before, the problem with Yahoo Finance is not inaccuracy of the data, but rather missing. This can lead to potential lowering of the integrity of the data. By lowering the integrity of the data, it is not so much a problem for the user, rather an annoyance. If a user is requesting a week of data, but a date is missing, the data for the existing dates are accurate, but will not show as many points as the user needs,

The impact of lowering the data integrity will be a 2, with a likelihood of 2. The reason for the low impact is, that a tracking platform is mostly used for looking at the historical development of stocks and portfolios. When looking at historical data, to get an accurate view of the data, it is often needed to go back at least 6 months, if not more, as less will not give an accurate view of the development. Take the COVID-19 pandemic for example. If, a week after the world shutdown, you look a week back, you will see a massive decrease in every, single stock. However, this is not an accurate depiction of the stock, as this was a special event, which can always occur for singular stocks. The likelihood is a two, but is dependent on the amount of data that is requested. For a week the likelihood is a 1 to miss data, but for over a year, the likelihood is a 3 for missing some dates.

When looking back more than 6 months, it is possible for Yahoo Finance to miss data from a few dates, either for the stock price or the currency exchange rate. To ensure correct data rather than the integrity, if any dates are missing from either the exchange rates or price, that date is excluded. We could have done it in such a way, to ensure integrity, that it would take the date before or after, and insert that price on the missing date. However, this would result in incorrect data, which could have a much higher impact. We can not do anything about missing data from the Yahoo Finance API, as this is beyond the application. We can therefore look at the trade-offs of producing the data ourselves or excluding it, where excluding it will give better accuracy, lowering the overall impact.

4.9.4 Server Interruption and Downtime

Server interruption and downtime pose a potential risk to the availability and reliability of our application. In this section, we will analyze the impact and likelihood of server interruption and downtime events, as well as outline measures to mitigate these risks.

Server interruption refers to the temporary unavailability of the server hosting our backend application. This could be due to various factors such as hardware failure, software issues, or network disruptions. Downtime, on the other hand, refers to the duration during which the server is inaccessible or not functioning as expected.

The impact of server interruption and downtime can range from minor inconveniences to significant disruptions for our users. Potential impacts include loss of user confidence and credibility in our application, financial losses for users relying on the application for trading or portfolio management, etc. As the application is not reliant on live data, any downtime or server interruption throughout the day does impose a larger impact, as it is simply more an annoyance to users. However, prolonged downtime of the application, will have an increasing impact on any financial decisions the users could make.

The likelihood of server interruption and downtime events can depend on various things, including the quality of the hosting infrastructure from Azure, the robustness of our application, and the availability of backup systems at Azure. While it is challenging to assign a precise value to likelihood, we can assign it as a 2 for the likelihood, to say it is moderate.

To mitigate the risk, we have implemented different measures. Our backend application is hosted on Azure Web App, which provides built-in features for load balancing, automatic scaling, and fault tolerance. This infrastructure helps minimize the impact of hardware failures or network disruptions by distributing the workload across multiple instances and regions. This is of course limited on the free tier, but can be purchased with a growing user base for the application. With this, we could also implement a backup system on Azure, that regularly backs up the data of the database.

4.10 Technical Analysis

Technical analysis is a necessary tool for analyzing and evaluating the project's different technical aspects. Does the new application perform correctly and as intended - does it fulfill set requirements? Is it efficient, reliable, and can it be scaled up? The process involves looking at the different technologies, the software architecture, design patterns, and overall the tools used to create the application.

4.10.1 Reliability and Error Handling Analysis

The goal of reliability and error handling analysis, is to identify potential sources of failures and assert the probability of them causing a system failure. Reliability and error handling are crucial aspects for an application. Error handling is important for the robustness of the application, making sure it can recover from any error it encounters.

The error handling of the application was explained in section 4.3.4 Error Handling. By utilizing the flow, it secures the reliability of the application, as it can listen for all the different possible errors. Unexpected errors are, of course, always a possibility. These errors will likely relate to data fetching where massive amounts of data are retrieved at once, either from Yahoo Finance API or the database. Missing data is a possibility. If the currency is not USD, the exchange rates is needed too, requiring more data. The way to deal with this particular issue, is to only select the asset prices, for which there exists an exchange rate. It will then only return prices in the correct currency and not display incorrect data. It is worth noting that this has only been experienced when retrieving a large amount of data, for example an asset's price spanning multiple years. A few dates not presented, is therefore an error with no impact, as these are not noticeable in a graphic presentation. This was talked about in the section 4.9 Technical Risk Analysis, where this flow had been chosen to have accuracy, over the integrity of the data.

For reliability of the application, the flow of the error handling can capture everything, to ensure the backend always communicates with the frontend. This is done by having the middleware for error handling, catch the base "Exception", that is any exceptions, that might not have been covered by the StatusCodeException. This will not be able to give a meaningful error message, but it will still prevent the backend from ceasing communication to the frontend. If something should happen though, the frontend is equipped with the functionality of retrying. If it does not receive an answer or an error that is not identifiable, the frontend will simply call the endpoint again. It will do this a specified number of times before alerting the user that something has gone wrong. As there will always exists some unexpected errors in the application, it is important that the frontend can handle these on its own, should the backend fail.

4.10.2 Security Analysis

Analysing the security of the application, is a crucial step to identifying any vulnerabilities. The analysis steps through the process of every aspect of security measures, that has been implemented and identifies any potential vulnerability.

The authentication and authorization flows were explained in the sections 4.7.1 Authentication and 4.7.2 Authorization. Implementing GDPR guidelines was mentioned in section 4.7.4 General Data Protection Regulation (GDPR). These three security measures created a solid foundation of the security of the application.

The flow that starts by calling the API with an endpoint. It then starts with the act of authentication through the middleware. The authentication, with the use of refresh rotation and reuse detection, creates a great and solid wall of protection, that requires the user to have a valid access and refresh token. If the user has this, but is interested in making changes to another users portfolio, there is the act of authorization. By using authorization as the last part of the

flow, it makes sure that the user has the right to change the resource, that they are requesting to change. This flow creates a great and solid foundation for the security of the application, as it utilizes the authentication in the middleware which can stop any request before it gets to actual business logic and data of the application. Here the authorization then makes sure, that users can not make changes to data of other users. The act of implementing the guidelines of GDPR, then gives the users the control and reliability of their data, by completely deleting data from the database, that the user chose to create, but then chose to delete afterwards.

One security vulnerability is the foundation of the token system, though it is solid, it was made from scratch. An existing authentication system could have been used. This could for example be OAuth which is made by professionals and has been tested by thousands of users. It could have provided better security, but was not used due to its complexity, alongside security not being a focus of the project. It was chosen to create a token based system, rather than use OAuth. The trade-offs of having to understand the complexity and implement OAuth rather than creating our own, the time saved on this was not a reliable choice.

4.10.3 Performance Analysis

Performance and resource costs must be considered when storing historical valuation data. The amount of computation needed to process the data, when dealing with the size of all the necessary data.

The two types of historical data to consider were stock prices and exchange rates. It was considered storing the prices in the listed currency or in a universal currency, as for example US dollars. Exchange rates was considered to be stored in a X=USD format where X is any currency or X=Y format where X and Y is any currency.

The data the frontend can request was considered here is historical stock prices in the listed currency or any other currency and the historical value of a portfolio in any currency.

How to request the price history for a stock.

Store stocks prices in listed currency and and exchange rates as X=Y			
Request	**Compute time**	**API time**	**Description**
Stock price history in listed currency	O(1)	O(1)	No conversion is ever needed there is nothing to compute and at most two API calls are necessary.
Stock price history in any currency	O(1)	O(1)	There will be one conversion and at most four API calls. Since the API calls takes much longer - will this take about twice as long as the listed currency even though they have the same O-notation.
Portfolio value history in any currency	O(n) where n is the number of stock positions	O(n) where n is the number of stock positions	There will be up to n conversions and at most $n \cdot 4$ API calls. This takes up to n times longer than a single stock price history

Store stocks prices in listed currency and and exchange rates as X=USD			
Request	**Compute time**	**API time**	**Description**
Stock price history in listed currency	$O(1)$	$O(1)$	No conversion is ever needed there is nothing to compute and at most two API calls are necessary.
Stock price history in any currency	$O(1)$	$O(1)$	There will be two conversion and at most six API calls. Since the API calls takes much longer will this take about three thrice as long as the listed currency even though they have the same O-notation.
Portfolio value history in any currency	$O(n)$ where n is the number of stock positions	$O(n)$ where n is the number of stock positions	There will be up to $n \cdot 2$ conversion and at most n*6 API calls. This takes up to n times longer than a single stock price history

Store stocks prices in USD and and exchange rates as X=USD			
Request	**Compute time**	**API time**	**Description**
Stock price history in listed currency	$O(1)$	$O(1)$	One conversion is needed and at most six API calls is necessary.
Stock price history in any currency	$O(1)$	$O(1)$	One conversion is needed and at most six API calls is necessary. This is the same as above since the listed currency and any other (except USD) is handled the same way
Portfolio value history in any currency	$O(1)$ where n is the number of stock positions	$O(n)$ where n is the number of stock positions	One conversion is needed and at most $n \cdot 2 + 2$ API calls. This is because everything is already in the same currency.

Storing all prices in USD and saving X=Y exchange rates is not considered since we would only ever use USD exchange rates since that would be what all prices is stored in.

It is also worth taking a look at much storage each configuration is taking up.

Store stocks prices in USD and and exchange rates as X=USD			
Stock prices	**Exchange rates**	**Storage**	**Description**
Listed	X=Y	$O(n^2)$ where n = number of currencies	One conversion is needed and at most six API calls is necessary.
Listed	X=USD	$O(n)$ where n = number of currencies	One conversion is needed and at most six API calls is necessary. This is the same as above since the listed currency and any other (except USD) is handled the same way
USD	X=USD	$O(n)$ where n = number of currencies	One conversion is needed and at most $n \cdot 2+2$ API calls. This is because every thing is already in the same currency.

Arguably n is limited since the number of supported currencies is 26 which mean $O(n) = O(26)$ which is the same as $O(1)$. Same thing goes for $O(n^2) = O(26^2)$ which is the same as $O(1)$.

The storage needed for X=USD for ten years worth of data for all 26 stocks is roughly 8MB and 200MB for X=Y.

The solution ended up storing stock prices with their values denominated in USD and only have X=USD. At first glance it might seem like the time O(1) for the computation is the reason for this. This is not the case since the data set will usually not be large enough that this will make a big difference between a O(n) time and a O(1) time. It is actually due to the API time. Even though all combination resulted in a time of O(n) this option is up to two and three times faster than the other options. The storage requirement was also not a determining factor. Even 50 years of data would at most be around one gigabyte.

As described earlier the frontend also caches the results to significantly optimize response times.

4.10.4 Scalability Analysis

Analysing the scalability of the application, focuses on how it can handle increasing numbers of users, that may all use the application at the same time, or stretched out over the entire day. Will they get the same quality of service. How does the application handles data processing and can it be scaled up?

The issue of scalability lies almost entirely, with the server power of Azure. The web service has different kinds of tiers. They all increase the CPU, memory, etc. These are of course more expensive, which introduces the need for generating income from the growing user base. By looking at the following article [12] by Microsoft, it states how many concurrent requests can happen and what the maximum number of sessions/connections are. For the basic tier, at which the database is currently running for free, the maximum number of concurrent sessions is 300. This gives the possibility of 300 concurrent connections to the database at the same time. This is more than enough for a long time, as a user does not get a dedicated connection to the database. This means, that whenever a connection is free, a user can use one of them. This allows for massive upscaling, as it would be possible to create a pool of 300 different connections, that the users can simply switch between and use, whenever a connection becomes available. The resources for the upscaling is available, and as the connection to the database is handled in a singular function on the backend, it would also be simple to change it and upscale it, for something like a connection pool. If at some point in the future 300 connections is not enough, more resources can be bought from Azure by buying the next tier.

Another scalability issue with Azure, is the database size. The current tier allows for a total of 250 GB of storage. As of now, when submitting this report, there is just about 72 MB of data stored. This gives plenty of possibility for upscaling the application. Additional users, new registered stocks that require historic data, or new features added that all require more space in the database, will not pose any problems.

4.10.5 Quality Assessment

The goal of quality assessment is to ensure implemented and future implementations have the same quality. It allows for a definition of a successful feature implementation, while following a certain standard of quality. The use of quality assessment will enable the enforcement of having a feature correctly implemented, before marking it as being finished.

As explained in section 3.2 User Stories, the agile development environment uses four different layers that a feature needs to be implemented in, to secure a foundation for features to be implemented after it, but also to ensure a continuously working application. These are only working requirements however, as the quality of an implementation holds more requirements than the four layers. These extra requirements are error handling and tests. For an implementation to be marked as successfully implemented, it has to work through all the four layers, but it also has to implement the flow of tests and error handling, as described in sections 4.3.4 Error Handling and 6.1 Unit Tests. This means, that it has to implement the use of the exception StatusCodeException, to correctly communicate errors to the frontend. Alongside of the implementation of error handling, it also has to implement the use of unit testing. The unit tests has to cover the functionality of the feature by both the top layer, but also by the individual parts, that was not already implemented into the application. It has to tests the different branches of the feature, to make sure that the error handling is also working as expected.

If a new feature follows and implements through all these different layers and all the tests succeeds, then the feature can be marked as successfully implemented.

4.10.6 Implementation of New Features

The implementation of new features will take a look at what kind of ground work that has been created, to ensure the ability of implementing new features, without the need of changing the foundation work. The section of implementation of new features, will discuss the agile environment of the project and the application itself, to test how easily a new feature can be implemented.

The foundation of the application has been explained throughout the whole section of the design. With the groundwork of the database, the centralized functionality of the value history and middleware, and the general agile development environment that the project and application has followed, has laid its way to a solid foundation of the application. With every feature being implemented into the four different layers as explained in section 3.2 User Stories, every feature creates its own foundation in the application and ensures a continuously working application. By implementing a feature on all four layers, the application should never suffer from completely crashing and breaking, as all features should have their foundations in the application, without changing any core mechanics.

As this is the way the implementation of features has followed, the implementation of new features is no different. The solid groundwork of the database allows for expansion on it, as the entities and attributes currently in the database, are not subject to change. For the authentication part, it is centralized through the use of middleware, meaning that any new endpoint will automatically have authentication implemented. For the centralized functionality of error handling through the middleware too, any new feature will automatically have the

catching and handling of any errors. It is only required to throw meaningful errors for new features, through the use of the StatusCodeException. As for the backend in general, all the endpoints are separated into their own files and logic, meaning that a new endpoint will not interfere with the logic of any existing ones. If the implementation contains the implementation of a new asset, the centralized value history comes into play. The value history centralizes all the different assets that a user owns, and returns it in one list, all together, in the specified currency. Using this, any asset can easily be added to the value history object, and be returned together with the rest.

Using the foundation of having a parent fetcher that takes care of choosing whether data should be gotten from Yahoo Finance or the database, makes it simple to implement any new asset. It simply needs to be implemented on on the parent fetcher, the database and the Yahoo Finance fetcher. From here, currency conversion and everything is already predefined functions, that can easily be implemented with new forms of assets.

The foundation of the application, by using the agile development environment, lays ground to simple implementation on any new features. It simply just need to follow the implementation on all four of the layers, so that the application will stay in a continuously working state.

4.11 Database Design

Database design was done in a conceptual design and a logical design. Then follows an explanation of the normalization of the database as a whole, for the individual tables.

4.11.1 Conceptual Design

The database conceptual design was, at a high-level, an abstract representation of the database, to capture the requirements for the structure of data of the database.

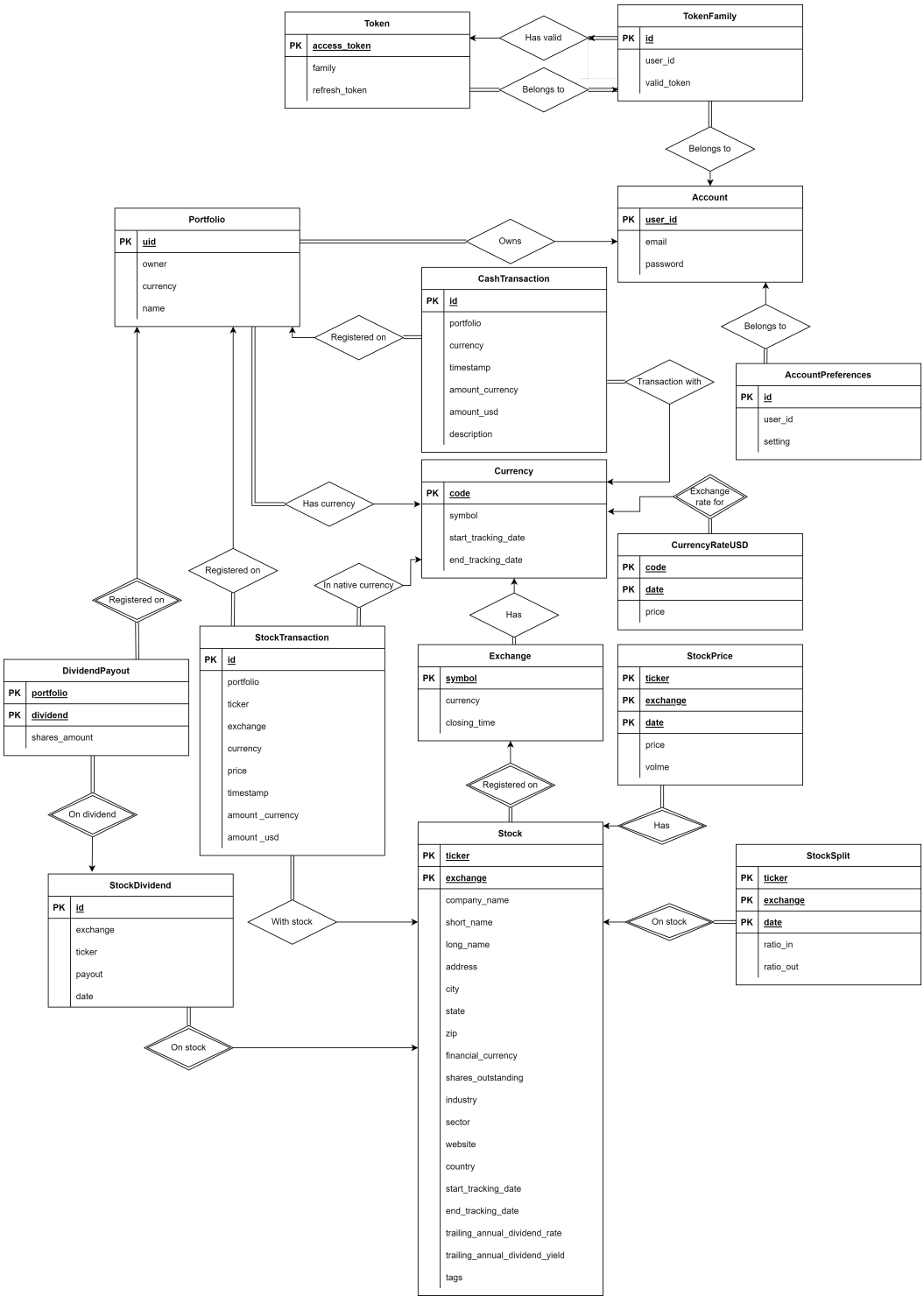


Figure 22: Entity-Relationship diagram for the database

Figure 22 shows entity-relationship diagram of the database. The ER-diagram follows the rules taught in the course "02170 Database Systems". The figure shows all entities and their attributes

from the conceptual design. All the entity sets are strong, except CurrencyRatesUSD, Stocks, StockPrices, DividendPayouts, StockSplits and StockDividends. A strong entity set is defined by an entity set, that can create its primary key by its own attributes. If the primary key is however created by the use of a foreign key, the entity set becomes weak. The problems with a weak entity set is, that it is completely dependent on the entities that it gets its primary key attributes from. Another problem is an increased redundancy in data, as the primary key data will then be displayed twice - one in the weak entity set and one in the set where it comes from. However, the use of weak entity set can give a better maintaining of data, as it is possible to use the on delete commands, to cascade any data, if the instance is deleted in the entity set it came from. This way, it will be easier to cleanup any unwanted data throughout the database.

To quickly summarize, the "DividendPayouts" table refers to a transaction contributing to the balance of a portfolio, where the "StockDividends" refers to the dividend that has been paid out by the stock, which is not necessarily a "DividendPayouts", if no users owns the stock.

Throughout the whole database, all the entity relations, uses one-to-many or many-to-one cardinality. The reason behind choosing this cardinality, is for the simplicity it introduces in the database, as it showcases the relationship of one instance of an entity, being associated with multiple instances of another entity. This gives the design of a database a more straightforward way of understanding it, as instances are not intertwining through many-to-many cardinality. Alongside this, every relationship is a binary relationship, as no relation refers back and fourth to the same entity, which would have made it unary. This is done to uphold the normal-forms, as any attribute that could make another attribute depended on it, is extracted to another table, to avoid redundancy. The normal-forms will be explained in depth in section 4.11.3 Normalization. We can also see on the figure 22, that every relation in the database uses partial participation, except the relation "Family", from Tokens to TokenFamily, that uses a total participation. The reason as to why partial participation is used for almost all relations, is due to how the data interacts with each other, and to remove redundancy, as entities that are in relation, only needs to be related if necessary. This is why Tokens and TokenFamily is the only total participation, as a token family and a token needs to be related, for a token to work in the authentication process.

4.11.2 Logical Design

Moving from the conceptual design to the logical design phase, we will now move from the high-level, abstraction, to the more detailed view of the database, and its functionality. The main objective of this phase, will be to convert the ER-diagram into relation schemes, that can be used as a direct blueprint for the implementation of the database. To do this, we will create a logical schema diagram.

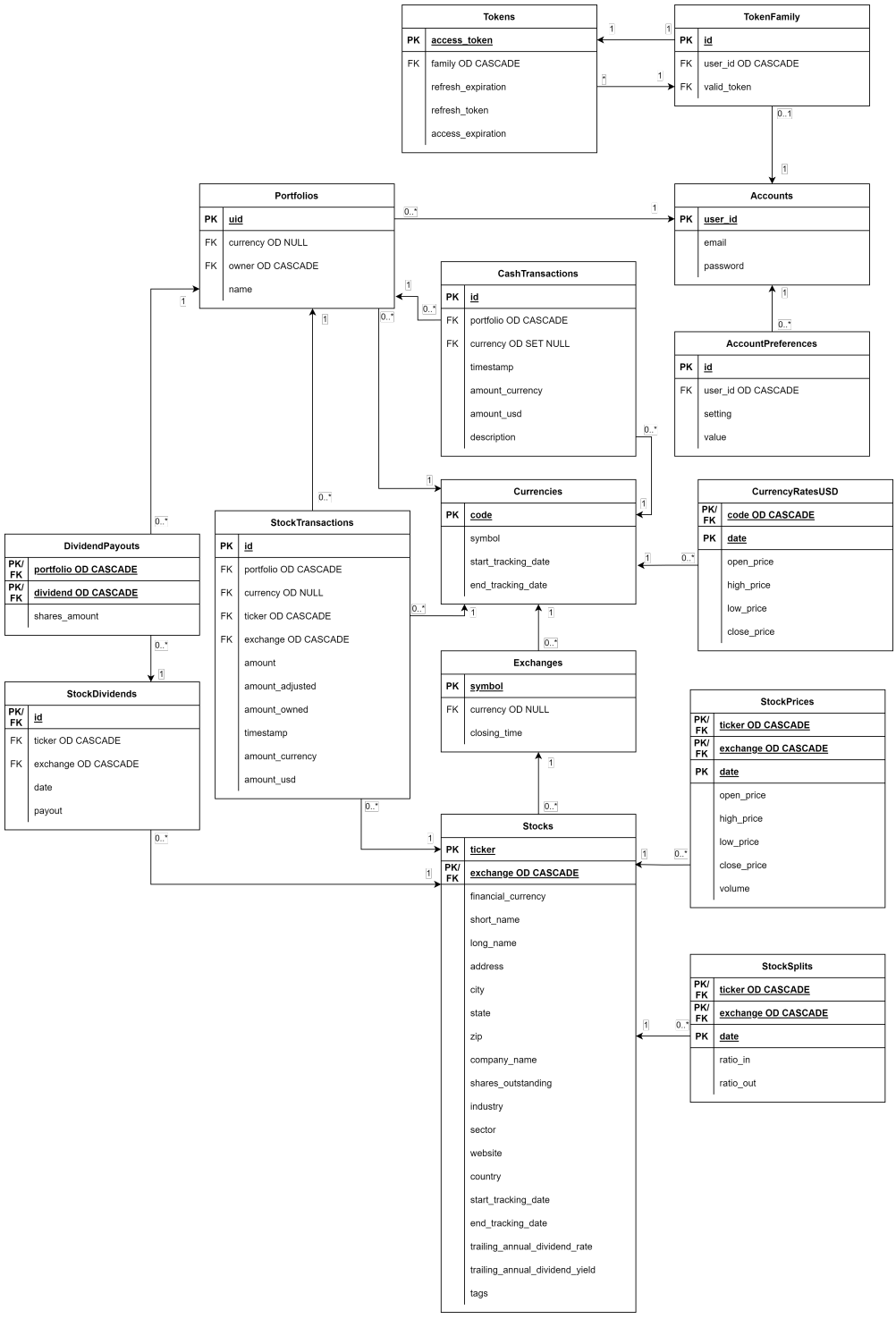


Figure 23: Logical schema diagram of the database

The figure 23 showcases the final relation schema for the database. This means that it shows what the foreign keys are and where they come from, alongside what will happen if the foreign

key is deleted. This is depicted through "OD", meaning "ON DELETE", and then if it is going to be cascaded, or set to null. For most of the database the deletion of a foreign key means it is going to be cascaded. This is due to the fact, that the foreign keys are often used to some sort of identification. This is for example every foreign key to either ticker or exchange, which is used in StockTransactions, Stocks, StockDividends, StockSplits and StockPrices. As the specific ticker symbol can be present on multiple exchanges, if the exchange were to be deleted, there would be multiple tickers with the exact same symbol, but without any identification on what exchange they are registered on, meaning that two identical ticker symbols, could be two completely different stocks, registered on different exchanges. That is why, if an exchange is deleted, it is necessary to cascade everywhere it is used, as the stock would not be identifiable anymore.

Moving to the foreign keys of the tokens tables, we have the foreign key "access_token" that is used in the TokenFamily table as "valid_token". This is set to do nothing. This is simply due to the circular dependency, between Tokens and TokenFamily. This makes a restriction in T-SQL, which will not allow to have "on delete" commands on both of the foreign keys, but only one of them. This is however fine, as having the cascade on the family of the tokens one, plays into the invalidation of the token family, which involves deleting the family if it becomes invalidated. Deleting a family, will therefore delete all the tokens of that family. It should not be possible to delete a token, but if it somehow happens to delete the valid token from the tokens table, then authenticating the token family, will then not have a valid_token. This invalidates the family, meaning it is deleted. No matter what then, unnecessary data will be deleted from the database.

4.11.3 Normalization

After setting up the schemes for the database, we can now look at the normalization for the individual tables. The goal for the database, is to secure that every table follows the first, second and third normal-form. The only table that varies from this is Stocks, which will be explained why later in this section. From here and throughout the rest of this section, the normal-forms will be referred to as 1NF, 2NF, 3NF, for simplicity.

Normalization is used to minimize data redundancy throughout the tables, and to try and avoid problems when doing modifications. The way to do normalization, is to decompose tables into smaller individual tables, by still preserving data using foreign keys. The goal of this, is to isolate data, so that anything added, updated or deleted can be done in just one table, where the rest of the database will change accordingly, based on the foreign keys. This is to ensure data consistency throughout the database.

The three normal-forms are defined as the following, by using the definitions of from the course 02170 Database Systems at DTU [7]:

- 1NF - All attributes depend on the key
  - Each attribute value must be a single value, meaning it must be atomic, and not multi-valued or composite.
  - Example: Having a list value in an attribute, will violate 1NF. It must be a single varchar, integer, etc.
- 2NF - All attributes depend on the whole key
  - It must be on 1NF.
  - Each non-primary key attribute must depend on every attribute of a composite key.
  - Is the primary key not a composite key, then the table is automatically following 2NF.
  - Example: Having a composite primary key consistent of (house_budget, house_number) and an attribute house_owner, the attribute can indirectly depend on house_number for the owner of the house, meaning that it violates 2NF.
- 3NF - All attributes depend on nothing but the key
  - It must be on 2NF.
  - Each non-primary key attribute must depend directly and entirely on the whole primary key.
  - Any non-primary key attribute must not depend indirectly on any other attribute.
  - Example: Having the primary key (product_id) and the attributes product_name and product_category, product_category can indirectly depend on the product_name instead of the primary key.

With the definitions explained, we can now go through each of the tables and explain why they are on 2NF and 3NF. For simplicity, we can state from the start that every table follows 1NF, as they all only contains atomic values.

Accounts
We assume the following, minimal functional dependencies:

$$user_id \rightarrow email$$

$$email \rightarrow password$$

We can see that Accounts is in 2NF, as there is no composite key, making it follow 2NF. For 3NF however, there exists a transitive dependency between email and password. This is a problem, as the same email would therefore be able to be registered multiple times, with a different password. A way to get around this, is through the implementation, to make email unique, therefore making it a candidate key instead, and bringing the table to 3NF. It still makes more sense to use the user id as the primary key, as the user has the ability to change the email. The email would still be unique, but to ensure reliability and stability in the data, it makes more sense to choose the user id.

Tokens
We assume the following, minimal functional dependencies:

$$access_token \rightarrow refresh_token, refresh_expiration, access_expiration, family$$
$$refresh_token \rightarrow refresh_expiration, access_expiration, family$$

We can see that Tokens is in 2NF, as there is no composite key, making it follow 2NF. For 3NF however, there exists a transitive dependency between the refresh token and the rest of the non-primary attributes. This is could introduce a problem, as there could potentially exists multiple instances of the refresh token, but with different expiration and families. However, as we know the refresh token must be unique, to ensure the correct authentication of the user. The authentication uses refresh rotation, meaning a new instance of both unique refresh and access token is created every single time. As we know the refresh token is always unique, this becomes a candidate key instead, bringing the table to 3NF.

TokenFamily
We assume the following, minimal functional dependency:

$$id \rightarrow user_id, valid_token$$
$$valid_token \rightarrow user_id$$

We can see that TokenFamily is in 2NF, as there is no composite key, making it follow 2NF. For 3NF, we can see that there technically exists a transitive dependency between the valid token and the user id. This exists, as we know that the valid token is unique, and could therefore be used to identify the user id. However, as explained in section 4.7.1 Authentication, the authentication system uses refresh rotation, meaning that the valid token can be updated multiple times. It will still be unique after the update, making it a candidate key, but as it can be updated multiple times, it is more reliable to use the id as the primary key. This design choice makes for better stability in the data over time.

Portfolios
We assume the following, minimal functional dependency:

$$id \rightarrow name, owner, currency$$

We can see that Portfolios is in 2NF, as there is no composite key, making it follow 2NF. For 3NF, we have no transitive dependencies in the table, as everything is dependent on the uid.

Currencies
We assume the following, minimal functional dependency:

$$code \rightarrow symbol, start_tracking_date, end_tracking_date$$

We can see that Currencies is in 2NF, as there is no composite key, making it follow 2NF. For 3NF, we have no transitive dependencies in the table, as everything is dependent on the code. There can exists multiple currencies with the same symbol, US dollars and Australian dollars both have symbol $, and multiple currencies can have the same start and end tracking date.

Exchanges
We assume the following, minimal functional dependency:

$$symbol \rightarrow currency, closing_time$$

We can see that Currencies is in 2NF, as there is no composite key, making it follow 2NF. For 3NF, we have no transitive dependencies in the table, as everything is dependent on the symbol. There can exists multiple exchanges with the same currency, for example NASDAQ and NYSE both having "USD", and multiple with the same closing time.

Stocks
We assume the following, minimal functional dependency:

$$ticker, exchange \rightarrow company_name, city, zip, website, short_name, long_name, ...$$

$$company_name \rightarrow website, short_name, long_name$$

$$zip \rightarrow city$$

For simplicity of the dependency, we are only showcasing the ones that have transitive dependencies, as there is too many attributes in the table to be displayed.

Not considering the transitive dependencies, but just the composite key of ticker and exchange, we can say that every non-primary attribute is dependent on both ticker and exchange, as its primary key. Ticker and exchange is required to identify a stock, as there can exist a stock with the exact same ticker on a different exchange, and there exists multiple tickers on an exchange. Both are therefore required to identify the stock, making it follow 2NF.

For 3NF, we have two transitive dependencies. One being the website, short_name and long_name, that can be identified from the company_name. The other is the city being identifiable from the zip code. By knowing the company name, you can get the website, the short name and long name of the company, as these will be unique for the instance. However, company name is not unique itself, as it can exist on multiple exchanges, if the same company is listed on more than one exchange. Same is with the zip code and city, where the city is unique to the zip code. Both these transitive dependencies introduces redundant data, as it is displayed multiple times, without it actually needing to be. A way to fix this and bring the table to 3NF, would be to extract the zip code and city to its own table. This way, you could only save the zip code to the table, and then make a lookup on the city in a different table. Similar with the company name, information that is exclusive to this exact company, like the short name, long name and website, could be extracted to its table. This way, only company name could be stored in the stocks table, and a lookup could then happen to the company information.

A choice was made, to keep the table with redundant data, as extracting the specific company data and city information to their own tables, would introduce unnecessary complexity. By not extracting them, it can improve the performance of the query. On top of this, it is not data that are going to be updated. The city and zip code is never updated. The company names and websites, are very rarely updated - and if it is updated, it is not certain that it will update on every exchange at the same time.

Considering the trade-offs between achieving 3NF and the increased redundancy that would come from storing all the information within the Stocks table, it was determined that the benefits of achieving 3NF were not substantial enough to justify the additional redundancy of data. The table is therefore only following 2NF.

CurrencyRatesUSD
We assume the following, minimal functional dependency:

$$code, date \rightarrow open_price, high_price, low_price, close_price$$

We can see that CurrencyRatesUSD follows 2NF, as all the non-primary attributes is dependent on the composite key of code and date. For the same argument as before, it also follows 3NF.

StockTransactions
We assume the following, minimal functional dependency:

$$id \rightarrow portfolio, ticker, exchange, amount, amount_adjusted,$$
$$amount_owned, timestamp, amount_currency, amount_usd, currency$$

We can see that StockTransactions is in 2NF, as there is no composite key, making it follow 2NF. For 3NF, we have no transitive dependencies in the table, as everything is dependent on the id. There can exists multiple stock transactions to the same portfolio.

StockDividends
We assume the following, minimal functional dependency:

$$id \rightarrow ticker, exchange, date, payout$$

$$ticker, exchange, date \rightarrow payout$$

We can see that StockDividends follows 2NF, as there is no composite key. However, there exists a transitive dependency in the table, making it not follow 3NF. This transitive dependency is, that all the non-primary attributes depend on the id, but the payout can depend on a composite key of ticker, exchange and date. It can depend on this, as there can only exist a single dividend payout, on a specific date for a stock. This makes it unique, breaking 3NF. This table was originally build with a composite key of ticker, exchange and date, to keep it on 3NF. Howeber, this was changed when the table of "DividendPayouts" was introduced. DividendPayouts, as mentioned before, is used to keep track of which dividends should be transactions for a user. To reduce the redundancy of data in the database, the StockDividends table got its primary key changed to an id instead. This made it much simpler for the DividendPayout to have a reference to a dividend, as instead of saving the ticker, exchange and date on both StockDividends and DividendPayouts, DividendPayouts could simply store a reference to the StockDividends id. To keep this simplicity and reduce the amount of redundancy in the DividendPayouts table, it was chosen to keep the StockDividends table on 2NF.

StockPrices
We assume the following, minimal functional dependency:

$$ticker, exchange, date \rightarrow open_price, high_price, low_price, close_price, volume$$

We can see that StockPrices follows 2NF, as all the non-primary attributes is dependent on the composite key of ticker, exchange and date. We know this, as there can exists multiple instances of daily prices and volumes, across all stocks. We can therefore not just look at he ticker and exchange or the date. This makes it follow 2NF. For 3NF, we have no transitive dependencies, as the prices will never be unique, as the different prices can exists on multiple different stocks.

DividendPayouts
We assume the following, minimal functional dependency:

$$portfolio, dividend \rightarrow shares_amount$$

We can see that DividendPayouts follows 2NF, as shares_amount will be dependent on both portfolio and the reference to a dividend. We know this, as there can exists multiple dividend payouts across portfolios, if the stock is owned across multiple portfolios. For the same reason, shares_amount is dependent on the whole key and nothing but the key, making it both 2NF and 3NF.

AccountPreferences
We assume the following, minimal functional dependency:

$$id \rightarrow user_id, setting, value$$

We can see that AccountsPreferences is on 2NF, as there is no composite key. We also see that there is no transitive dependencies. This is due to the fact, that a user can have multiple different settings, each of them having values that is not unique. It is therefore dependent on just the key, making it 3NF.

StockSplits
We assume the following, minimal functional dependency:

$$ticker, exchange, date \rightarrow ratio_in, ratio_out$$

We can say that StockSplits is on 2NF, as ratio_in and ratio_out has to depend on the complete composite key. This is due to the fact, that ticker and exchange is used to identify the stock and as there can technically, although not very likely, happen multiple splits a day, they also has to depend on the date. This means that there can exists splits on the same date for the same stock, making it dependent on the whole key and nothing but the key, living up to 2NF and 3NF.

CashTransactions
We assume the following, minimal functional dependency:

$$id \rightarrow portfolio, currency, timestamp, amount_currency, amount_usd, description$$

This lives up to 2NF and 3NF for the same reason as StockTransactions. Here we just have a currency, but it acts the same way as the ticker and exchange. It is therefore on both 2NF and 3NF.

4.11.4 Database Objects

In this section, we are going to be talking about the different functions, triggers, procedures and views that exists in the database and their functionalities. The objects on the database are generally used to uphold the integrity and consistency of the data. By having such functionalities on the database, we can limit unnecessary connections from the backend to the database, and only get the data that the backend actually needs.

TRIGGER - DividendPayoutInsert
This trigger takes care of inserting the dividend payout transaction for a portfolio, when new dividends for a stock is added to the StockDividends table. When new dividends are added, it gets the portfolios of every user that owns this stock. It will then get the latest stock transaction of the portfolio, where the amount owned is more than 0. If it is 0, then it means that the user has sold all the stocks. After this is found, it will then insert dividend payouts for all the dividends that are being added to the StockDividend table, where the user owns the stock. This trigger is to ensure that dividend payouts are added automatically.

TRIGGER - UpdateAmountAndDividendPayoutDeletion
This trigger is used to update stock transactions and dividend payouts, whenever a stock transaction is deleted. When a stock transaction is deleted, all the transactions that comes after it, has to get the amount owned updated, to subtract the amount of the deleted transaction. Alongside this, it will then also update the shares_amount in the DividendPayout table, again to keep track of the amount owned of stocks.

TRIGGER - DeleteDividendPayout
This trigger is a cleanup trigger. When the shares_amount is updated from the previous trigger, UpdateAmountAndDividendPayoutDeletion, this trigger executes. It is triggered by update on the DividendPayouts table, where it simply checks if the shares_amount after updating is equal to 0. If it is, it deletes it, as it is not a dividend payout transaction we are interested in anymore then. This trigger is for cleaning up unnecessary data in the database, to ensure the consistency and integrity of the data, as when getting dividend payouts, we expect it to have value.

TRIGGER - UpdateAndInsertOnStockTransaction
Similar to the trigger "DividendPayoutInsert", this triggers on insert on the StockTransactions table instead of the StockDividends. The DividendPayoutInsert trigger was to ensure, that dividend payouts was added, whenever new stock dividends got added. This trigger, UpdateAndInsertOnStockTransaction, handles the insertion whenever a user makes a stock transaction that has dividends. When a user inserts a historic stock transaction, this trigger then gets all the stock dividends that has happened after the timestamp of the stock transaction, and adds them to the dividend payout table as transactions.

TRIGGER - UpdateStockSplits
This trigger is triggered on insert on the StockSplits table. This trigger is for updating the stock transactions, that has happened before a split. This means, that it handles when a user makes a stock transaction before a split, updating it to the correct amount it would have been adjusted to the split.

TRIGGER - MaintainTrackingDatesStockPrices
This trigger is working as a safety net. All it does is checking if start_tracking_date and end_tracking_date is updated correctly, after the insertion of new stock prices. It does this by taking what the tracking dates was updated to, and then checking if the dates of the inserted stock prices, are greater or lower than the tracking dates. If that is the case, then it updates the tracking dates to the correct ones.

TRIGGER - MaintainTrackingDatesCurrencyRatesUSD
This trigger works the complete same as MaintainTrackingDatesStockPrices, but doing it on CurrencyRatesUSD instead.

FUNCTION - CheckIfAccessIsValid
This function checks whether or not an access token is valid. It does this by using the parameters access token and a unix timestamp. It first checks if the family exists. If it does not, we know it is invalid. Then it checks if the token is expired, by looking up the expiration in the Tokens table and checking if it is greater than the unix timestamp parameter. It then checks if the "valid_token" in the TokenFamily table is the same as the access token in the parameter. All of these together, returns a table with the id of the family and a varchar, that is either "Valid", "Expired", "Invalid" or simply empty. The reason this is done on the database as a function, is to minimize the amount of data about tokens that are sent to the backend, to reduce security risks. By doing this, the backend only receives if it is valid or not and a family id.

FUNCTION - CheckIfRefreshIsValid
This function is the same as CheckIfAccessIsValid, it just takes a refresh token in its parameter instead.

FUNCTION - GetStockPrices & GetCurrencyRates
These two functions works the exact same, it just depends on if it is checking the CurrencyExchangeRatesUSD table or the StockPrices. They both take the parameters interval, start_date and end_date. They then differ, where CurrencyExchangeRatesUSD takes a code and StockPrices takes a ticker and an exchange. The interval is how regularly we want data, if we want data "daily", "weekly", "monthly", "quarterly" or "yearly". The start date and end date is then two varchars, depicting the date. The functions will then create a temporary table, that mimics the CurrencyExchangeRatesUSD or StockPrices tables, but using a CASE statement, to determine the date of it. If it should just be the date, if the interval is "daily", or if weeks, months, years, should be added to it, dependent on the interval. It then simply finds all the entries in the respective tables, that lies between the start date and end date, and sends the table to the backend.

PROCEDURE - BulkJsonStockPrices & ,-StockDividends & ,-CurrencyRates
These three procedures is used to speed up the insertion process of new exchange rates, stock prices or stock dividends. Instead of making individual insertion statements on the backend to the database, these three procedures takes a JSON array string as a parameter, and then either code or ticker and exchange, dependent on which procedure it is. It then creates a temporary table that is identical to CurrencyExchangeRatesUSD, StockPrices or StockDividends, where it selects all the values from the JSON array and inserts them into the temporary table. By doing this, it is then possible to do the MERGE statement. The MERGE statement tries to merge all values of one table into another. If an entry does not exists in the original table, the entry is then inserted. If it already exists, then it is updated instead. By doing this, we can uphold

the integrity of the data, as it will then be possible to update any existing values in the original tables, should any values not have been added correctly in an earlier insert. These procedures allows us to bulk insert a massive amount of data, as we can utilize the merge statement, instead of individual insert statements, significantly increases the performance of the insertion of massive amount of price, dividend and exchange rate data.

VIEW - AllTransactions
This view is utilized in the section 4.3.5 Transactions. This view combines all the different types of transactions, stock transactions, cash transaction and dividend payout, into a single view. This gives a much better overview of all the transactions of a portfolio, when having to get the value history to display for the frontend. Instead of having to do complex queries or many singular ones, the transactions are combined into one view, with all the necessary information. It does this by union the three tables, with all the values that are relevant for the calculations of the value history and getting all the transactions for the frontend. It then joins them on the portfolios, and adds a column "transaction_type", that inserts the name of the table it came from.

4.11.5 Populating the Database

Continuously populating the database was explained in section 4.3 Software Architecture - Backend. However, to start the database, it was populated with predefined values for tables Currencies and Exchanges. The reason behind this is, that they can not be dynamically added, as they require manual insertion. This is due to the fact of an exchange being added with a currency and a closing time. Two things that is not possible to get through the Yahoo Finance API. For the currency, they are added with a symbol, another thing that is not possible to get. To find out which exchanges and thereby currencies was needed, we created a web-scraper that scraped every single stock ticker and exchange from Yahoo Finance. How this was done, is explained in depth in the implementation section 5.1 Populating Database. Once all the tickers and exchanges was scraped and added to a csv file, the distinct exchanges was extracted. This way, we could do a lookup on Yahoo Finance and find what exchanges was supported and add them to an insert statement on the database. When looking up the exchanges, it was then also checked what currencies they used on these exchanges. These currencies, alongside their symbols, was also added to an insert statement. This should give significant coverage, for all the exchanges and currencies that users are interested in.

For the populating of stock data, we get profiles and prices from Yahoo Finance. However, for a profile to get added to the database, it is required for a user to search for them first. To get around this and get significant coverage of all stocks, so a user can create a transaction from the start without first searching the stock, the scraper also includes a script, that gets stock profiles from every single of the stocks, and adds them to individual JSON files. This way, we could then create a bulk insertion procedure on the database, that could take a combination of all the JSON files and add them all to the database. This would make it, so that the user is not required to search a stock first, and gives the added benefit of a much quicker search, as it only has to look in the database and not Yahoo Finance first.

With this, the database got a significant ground work, that made it more than suitable for new users, by having a massive stock coverage.

5 Implementation

5.1 Populating Database

For populating the database with all the different stock profiles of Yahoo Finance, a combination of Python and C# was used. The reason for the combination, was the requirement for scraping the Yahoo Finance website for all the stock tickers and exchanges. The library "Selenium" for Python was a top choice for this, as it acts as a normal user when scraping a website. This is different from libraries that takes the static site, such as "BeautifulSoup". This takes HTML of a specific URL, but if elements are not loaded on the initial startup of the site, they will not be present in the pulled HTML. This was the case for the Yahoo Finance website, which is why Selenium was used instead.

By using the Yahoo Finance screener, which is basically an advance search for all stock with different requirements such as region, marketcap, exchange, etc. As this screener can only get 10000 stocks at a time, it is necessary to first check through every single region and scrape the stocks from each of them individually, to not exceed 10000 stocks. The general way this is done in Selenium, is by navigating through the HTML elements, with classes and id's. First would be to scrape every single region, and add each of them to a CSV file. This is done by navigating through the region selector in the screener, selecting a region, clicking search, getting the URL, deselecting the region and then selecting the next region in the list. This produced a complete list of the first page of every screener, with just one region selected. When this was scraped, we could then move on and make a scraper for all stocks of each region. Below we have the first snippet of the scrape function.

```
def scrape_yf():
    url_list = get_urls_from_csv()
    driver = webdriver.Chrome()
    driver.get(url_list[0])
    yf_cookie_reject = driver.find_element(By.XPATH, '//button[@class="btn
        secondary reject-all"]')
    driver.implicitly_wait(3)
    yf_cookie_reject.click()
    driver.implicitly_wait(5)
```

It first gets all the URLs that was scraped before. It then sets up Chrome, so that it can interact through it as a normal user. The very first thing, before we can actually do anything scraping, is by clicking the reject all button on the cookies pop-up of the Yahoo Finance website. When clicked, by simply telling the driver to act a click on a the specified HTML element, we wait 5 seconds for the website to load and now the scraping can begin.

```
with open('tickers.csv', 'a', newline='') as csv_file:
parameter = '?count=100&offset={offset}'
for url in url_list:
    correct_url = url + parameter
    for offset in range(0, 9900, 100):
        try:
            print(correct_url + ", offset: " + str(offset))
            driver.get(correct_url.format(offset=offset))
            scraped_tickers = driver.find_elements(By.XPATH,
                '//td[@aria-label="Symbol"]')
            if (len(scraped_tickers) == 0):
                print("Break with url " + correct_url + ", offset: " +
                    str(offset))
                break
            for scraped_ticker in scraped_tickers:
                if ('.' in scraped_ticker.text):
                    ticker = scraped_ticker.text.split('.')[0]
                    exchange = scraped_ticker.text.split('.')[1]
                    writer(csv_file).writerow([ticker, exchange, ''])
                else:
                    ticker = scraped_ticker.text
                    writer(csv_file).writerow([ticker, '', ''])
        except Exception as e:
            print(e)
            print("Exception Break with url " + correct_url + ", offset: " +
                str(offset))
            break
```

By looking at the above code snippet, this is the rest of the scraping function. To start the scraping, we loop through all the URLs in the CSV file. Each URL is then equipped with a parameters, which is the count and offset. A page can only showcase 100 stocks at a time, and only 10000 stocks in total on all pages. We therefore loop through the offset from 0 til 9900, incrementing by 100 at a time. With the URL and its parameters, we then give the URL to the Chrome driver. This allows the open window, to navigate to a specific URL. It then scrapes in the ticker and exchanges of every single stock on the page, by the specific HTML element. If the length of the scraped tickers is 0 however, we know the page is empty, meaning that there is no more stocks. We therefore break the for loop, and moves on to the next URL. The extra check for if it contains a ".", is for American stocks. Stocks registered on "NYSE" or "NASDAQ", will on Yahoo Finance not have an exchange extension. Every other stock will have an extension, for examle Novo Nordisk in Denmark registered on CPH, which would have the ticker and exchange extension "NOVO-B.CO". When scraped, the ticker and exchange is then added to another CSV, which in the end contained a full list of every ticker and exchange.

With that done, we could now pull every single profile of every single exchange in the CSV file. This is done by calling the two Yahoo Finance endpoints to get the complete profiles, which was then added to individual JSON files. All information was pulled, even though we do not store all the information in the database, due to fact if we wanted to change what we stored later on. We then had the information already and was not required to get it again. The reason behind saving it in singular JSON files, was for easier maintainability and debugging when uploading to the database. We could now easier pinpoint an exact JSON file that gave an error, rather than looking through one massive JSON file.

As for actually populating the database, we then created a separate C# project. This C# project was in charge of loading in the CSV file of all the ticker and exchanges and uploading it to our database. This was completely separate, not using any of our endpoints, as this is purely for development and should therefore not interfere with the general application. This C# project simply looped through all the JSON files, extracted the information we wanted to save in the database and called an insert statement on the "StockProfiles" table.

5.2 Middleware - Authentication

This section will cover the direct implementation of the middleware, and the authentication that it uses. As a starter, the middleware uses the build in functionality in ASP.NET Core, to add a custom middleware to the web application.

```
WebApplication app = builder.Build();
app.UseMiddleware<Authentication.Middleware>();
```

It implements the middleware Authentication.Middleware, which is the class Middleware in the Authentication namespace. This class takes the interface IMiddleware, which is native to ASP.NET Core and implements the needed methods for a valid middleware. The function here being InvokeAsync.

```
public async Task InvokeAsync(HttpContext context, RequestDelegate next)
```

InvokeAsync takes a HttpContext and a RequestDelegate as its parameters. A HttpContext represents an individual HTTP request. It holds information about the path of the URL, the parameters of the request, any headers, etc. The HttpContext object is here used through the middleware, to access information about the request, so the middleware can find out what to do with it. A standard use of the HttpContext object to authenticate the access token can be seen here.

```
   String? accessToken = context.Request.Headers["Authorization"];
if (accessToken != null)
{
   if (!Authentication.Authenticate.AccessToken(accessToken))
   {
      context.Response.StatusCode = 401;
      await context.Response.WriteAsync("Access token is invalid.");
      return;
   }
}
```

We here see, that the middleware reads the access token of the Authorization header. If this token is not null, in other words it is provided, then it tries to authenticate the token. If the token authentication fails, then the HttpContext is supplied with a status of 401, unauthorized, and a response of the token being invalid. The task of the method is then returned back to the user, providing them with the details of it being invalid. If the token is authenticated however, then it awaits the RequestDelegate.

```
await next(context);
```

The RequestDelegate is a callback function. It is used to process any incoming HTTP requests, where it takes a HttpContext object as its parameter and returns a Task object, showcasing the asynchronous handling of the request. When it is awaited, it moves on with the middleware pipeline. As there is no more to the middleware, the endpoint handling will then start instead, indicating that the authentication was successful.

There is a few exceptions to the authentication in the middleware however. These exceptions are for posting tokens and posting users. The POST tokens being for signing in and the POST users being for signing up. As these can not require a token, because the user is not signed in and is therefore not in possession of a tokenset, these requests are let through the the endpoint handling.

```
if ((context.Request.Path == "/v1/tokens" && context.Request.Method == "POST")
    || ((context.Request.Path == "/v1/users" && context.Request.Method ==
    "POST")))
{
   await next(context);
   return;
}
```

It here first checks if the path of the HttpContext is the specified paths and if the methods of them is equal to POST. This way, we know that it is a user that tries to sign up/in and can be let through to produce their token set. Another special case is for when the access token has expired and the user wants to refresh their token. To utilize the simplicity of just using a singular header, Authorization, it checks in this special case of a PUT method on tokens, for a refresh token.

```
if (context.Request.Path == "/v1/tokens" && context.Request.Method == "PUT")
{
   String? refreshToken = context.Request.Headers["Authorization"];
   if (refreshToken != null)
   {
      RefreshAuthenticationResponse response =
          Authentication.Authenticate.RefreshToken(refreshToken);
      if (response.error == "Invalid")
      {
         context.Response.StatusCode = 401;
         await context.Response.WriteAsync("Refresh token is invalid");
         return;
      }
      else if (response.error == "Expired")
      {
         context.Response.StatusCode = 403;
         await context.Response.WriteAsync("Refresh token is expired");
         return;
      }
      else
      {
         context.Items["RefreshToken"] = refreshToken;
         context.Items["FamilyID"] = response.familyID;
         await next(context);
         return;
      }
   }
   else
```

```
        {
            context.Response.StatusCode = 401;
            await context.Response.WriteAsync("No token was provided");
            return;
        }
    }
```

As we see here, the if statement checks the path of the HttpContext is tokens and the method is PUT. If it is so, then we expect the token in the Authorization header to be a refresh token, instead of access. We then authenticates the refresh token instead. If it authenticates, the middleware allows it through to the endpoint handling, where it also returns the family id of the token. This is used to refresh the tokens. If not, a 401 unauthorized or 403 forbidden is returned. This is dependent on if the refresh token is expired or invalid.

For the authentication for the access token, it calls the function Authenticate.AccessToken(accessToken). The way it works is very similar to the Authenticate.RefreshToken(refrshToken), as this just checks it for an access token instead. This can be seen here.

```
      String? accessToken = context.Request.Headers["Authorization"];
    if (accessToken != null)
    {
        String isValid = Authentication.Authenticate.AccessToken(accessToken);
        if (isValid == "Invalid")
        {
            context.Response.StatusCode = 401;
            await context.Response.WriteAsync("Access token is invalid");
            return;
        }
        else if (isValid == "Expired")
        {
            context.Response.StatusCode = 403;
            await context.Response.WriteAsync("Access token has expired");
            return;
        }
        else
        {
            context.Items["AccessToken"] = accessToken;
            await next(context);
            return;
        }
    }
    else
    {
        context.Response.StatusCode = 401;
        await context.Response.WriteAsync("No token was provided");
        return;
    }
```

This code snippet is very similar to the refresh one, but this is the main authentication method. This takes any path, that is not covered by the before mentioned special paths. It is here, where it authenticates the access token, and checks if it is invalid or expired. The reason behind sending two different HTTP statuses to the frontend, is the fact that the frontend has to know if the user should login again, or if it can simply refresh the tokens. if the token is simply exipred, it refreshes it, if it is invalid, the user has to login again.

The main functionality of the authenticate function happens on the database site. This was done to secure any data of the tokens, as they will not be available to the backend. The whole check of the authentication happens on the database, that simply returns if it is valid, expired or invalid.

```
    public static String AccessToken(string accessToken)
{
    String query = "SELECT * FROM CheckIfAccessIsValid(@access_token, @UnixNow)";
    Dictionary<String, object> parameters = new Dictionary<string, object>();
    parameters.Add("@access_token", accessToken);
    parameters.Add("@UnixNow", Tools.TimeConverter.DateTimeToUnix(DateTime.Now));
    Dictionary<String, object>? data = Data.Database.Reader.ReadOne(query,
        parameters);

    if (data != null)
    {
        String isValid = data["is_valid"].ToString()!;
        int familyID = int.Parse(data["family_id"].ToString()!);
        if (isValid == "Valid")
        {
            return "Valid";
        }
        else if (isValid == "Expired")
        {
            return "Expired";
        }
        else if (isValid == "Invalid")
        {
            InvalidateFamily(familyID);
            return "Invalid";
        }
        else
        {
            return "Invalid";
        }
    }
    else
    {
        // If gets here, family does not exists, so token not valid, but can not
            invalidate family
        return "Invalid";
    }
}
```

It calls the function CheckIfAccessIsValid, which takes the parameters access token, and a timestamp of DateTime.Now() converted to unix time. The function on the database then returns a varchar, to depict the status of the token. It returns the status of the token for all of them, except for the invalid one, where it also invalidates the family. If an access token is invalid, then it means that a non-valid access token for the family is used, meaning we have to invalidate the family. This is done by simply deleting the family.

```
        String invalidateFamilyQuery = "DELETE FROM TokenFamily WHERE id =
            @family_id";
```

For the authentication on the database, lets take a look at how the function works.

```
  CREATE OR ALTER FUNCTION CheckIfAccessIsValid(@AccessToken VARCHAR(255),
      @UnixNow BIGINT)
RETURNS TABLE AS RETURN
(
   SELECT (
      CASE
         WHEN Tokens.access_expiration > @UnixNow AND TokenFamily.id IS NOT NULL
             AND TokenFamily.valid_token = @AccessToken THEN 'Valid'
         WHEN Tokens.access_expiration < @UnixNow THEN 'Expired'
         WHEN NOT(TokenFamily.valid_token = @AccessToken) THEN 'Invalid'
         WHEN TokenFamily.id IS NULL THEN ''
         ELSE ''
      END
   ) AS is_valid,
   Tokens.family AS family_id
   FROM Tokens LEFT JOIN TokenFamily ON TokenFamily.id = Tokens.family WHERE
       Tokens.access_token = @AccessToken
);
```

This function takes in the access token and a unix timestamp as the parameters. It then returns a table of the varchar "is_valid" and "family_id". The is_valid is determined by the case statement, that runs through different cases. The first for the token to be valid, meaning that the family must exists, the valid_token for the family must be equal to the given access token and the expiration of the access token, must be bigger than the given unix timestamp. If this is not the case, it checks whether it is expired, or invalid. If it returns empty, it means that the family did not exists, so the token is invalid, but we can not invalidate the family, as it does not exists. This is depicted in the else statements from the code snippet of the method AccessToken(string accessToken). It then also gets the family id of the family, by joining the Tokens table on the TokenFamily. This is done to ensure the return of the correct token family, if it is to be invalidated.

When the authentication is done, the middleware then takes of either sending the RequestDelegate with the HttpContext to the correct endpoint, or returning a 401 or 403 HTTP status back to the frontend, whether the access token was expired or invalid.

5.3 Middleware - Error Handling

The eror handling middleware, is similar in the way it is build to the authentication middleware. They both utilizes the "IMiddleware" interface from the ASP.NET Core framework. They are also both added as part of the middleware for the setup for the webapplication.

```
  WebApplication app = builder.Build();
  app.UseMiddleware<Authentication.Middleware>();
app.UseMiddleware<ErrorHandlingMiddlware>();
```

This middleware is therefore, in its base, the same as the authentication middleware, however this one is build different in its functionality.

```
  public async Task InvokeAsync(HttpContext context, RequestDelegate next)
{
   try
   {
      await next(context);
   }
   catch (StatusCodeException e)
   {
      context.Response.StatusCode = e.StatusCode;
      await context.Response.WriteAsync(e.Message);
   }
   catch (Exception e)
   {
      System.Console.WriteLine(e);
      context.Response.StatusCode = 500;
      await context.Response.WriteAsync(e.Message);
   }
}
```

As seen on the above snippet, it again takes a HttpContext and a RequestDeletegate as the input parameters, just as the authentication. The difference here is though, that the RequestDelegate with the HttpContext is awaited in a try-catch statement. What this does is, that it will finish up the process of the HttpContext, and if any exceptions was thrown, it will be caught here. It will first be caught by the StatusCodeException, which is the custom, controlled exception, used to communicating errors to the frontend. It then also catches any other exception, should something happen that is not controlled by the StatusCodeException. Both catch statements awaits the response of the HttpContext, sending a HTTP StatusCode and a message back to the frontend. Thsi way, every exception will be caught and the frontend will always get information about anything happening on the frontend. Now, lets take a look at the implementation of the StatusCodeException.

```
public class StatusCodeException : Exception
{
 public int StatusCode { get; set; }
 public StatusCodeException(int statusCode) : base()
 {
    this.StatusCode = statusCode;
 }
 public StatusCodeException(int statusCode, string message) : base(message)
 {
    this.StatusCode = statusCode;
 }
}
```

Above here we see the implementation. The class StatusCodeException extends the Exception base class, which implements the constructors needed for an exception. This is the base, where one takes just a status code, and the other takes both a status code and a message. The ": base()" invokes the constructor of the base class, Exception, which constructs it as a system identifiable Exception.

6 Tests

6.1 Unit Tests

The goal and purpose of unit testing, is to verify that units work as intended. It is a software development practice, where individual units of the application are tested, by asserting different statements about the response of a function. Unit testing can identify bugs and defects in the application and it helps to maintain code quality and hence application stability. Unit testing makes it significantly easier to implement new features as testing after implementation can show whether existing features still have the desired functionality.

The tests were created by using the MSTest framework. This was chosen mostly due to it being native to the .NET framework. For running the test the MSTest framework adds the functionality straight to Visual Studio Code, by adding buttons to run each test, or using the testing panel that gives an overview of every single test file, with the individual tests inside.

Another unit testing framework could have been NUnit. It is also frequently used and sets up easily for all .NET languages. However, a test setup that would integrate without problems and directly with the ASP.NET Core framework was preferable.

The test was created for three different layers. The top layer, which is the general setup methods for each of the endpoints and the application itself. This layer is to test out that that the endpoints and the application is being setup correctly, alongside test if the middleware functions as intended. The second layer is the direct endpoint methods. These methods are for the second layer of logic, before going into the business logic. These are often used to check if any values are missing or invalid from the query parameters or the body of the request. These tests basically simulates an endpoint being called with the necessary values.

The last level is then the business logic itself. Here the general functionality of the application is tested, to make sure all the methods works as they are intended to.

The tests were created to work independently by utilizing the "Initialize" and "Cleanup" methods of the MSTest framework. There are three layers to them: "Test", "Class" and "Assembly". The "Assembly" layer runs once before any test, and once after all the test for the cleanup. The "Class" layer runs first in the test class and when all test in the class is finished the cleanup

runs. Then finally, the "Test" layer runs before and after every individual test in the test class, where the "Test" layer of initialization and cleanup is implemented. An example of the "Test" layer would for example be to create a user in the database before every test. The test was then executed. Regardless of the result, the cleanup of the "Test" layer would delete the user again. This could also be used for setting up portfolios, transactions, etc., and then cleaning it all up afterwards. This made testing flexible and robust as no test was dependent on anything to be successfully created by a previous test. Utilizing the initialize and cleanup could look like this:

```
private UserTestObject? userTestObject;
private StockApp.Portfolio? portfolio;

[TestInitialize]
public void Initialize()
{
    userTestObject = UserHelper.Create();
    portfolio = PortfolioHelper.Create(userTestObject);
}

[TestCleanup]
public void Cleanup()
{
    UserHelper.Delete(userTestObject!);
}
```

This initialize and cleanup is used for testing the business logic of "CashTransactions". Before creating a cash transaction, a user with a linked portfolio are needed to add the transaction to. Initialize is then called before every test in the class, creating a userTestObject, which holds a user, an access token, a refresh token and a family id, and a portfolio. The cash transactions test could then utilize these objects, run through the test and delete the user object afterwards.

Test examples from the top layer, application setup tests.

```
[TestMethod]
public void SetupApplicationTest()
{
   try
   {
      API.v1.Application.Setup(false);
   }
   catch (System.Exception e)
   {
      Assert.Fail("Application could not setup correctly: " + e.Message);
   }
}

[TestMethod]
public void SetupEndpoint_PostUsersTest()
{
   try
   {
      WebApplication app = WebApplication.CreateBuilder().Build();
      API.v1.PostUsers.Setup(app);
   }
   catch (System.Exception e)
   {
      Assert.Fail("Endpoint could not setup correctly: " + e.Message);
   }
}
```

These two examples, tests the general setup of the application and then the general setup of a singular endpoint, here the PostUsers endpoint. Both of them are equipped with try-catch statements, that catches anything. If anything is caught, an assertion is made that the test fails, prints a meaningful message alongside the error message itself.

From the top layer, another example is a test of the authentication middleware.

```
    [TestMethod]
public async Task MiddlewareTest_ValidAccessTokenAndCommonPathSuccessTest()
{
    DefaultHttpContext context = new DefaultHttpContext();
    context.Request.Path = "/v1/portfolios";
    context.Request.Method = "GET";
    context.Request.Headers["Authorization"] = userTestObject.accessToken;
    RequestDelegate next = (HttpContext context) =>
    {
        return Task.CompletedTask;
    };
    Authentication.Middleware middleware = new Authentication.Middleware();
    await middleware.InvokeAsync(context, next);
    Assert.AreEqual(StatusCodes.Status200OK, context.Response.StatusCode);
}

    [TestMethod]
public async Task MiddlewareTest_ExpiredAccessTokenAndCommonPathTest()
{
    TokenHelper.MakeTokensExpired(userTestObject.accessToken!,
        userTestObject.refreshToken!);
    DefaultHttpContext context = new DefaultHttpContext();
    context.Request.Path = "/v1/portfolios";
    context.Request.Method = "GET";
    context.Request.Headers["Authorization"] = userTestObject.accessToken;
    RequestDelegate next = (HttpContext context) =>
    {
        return Task.CompletedTask;
    };
    Authentication.Middleware middleware = new Authentication.Middleware();
    await middleware.InvokeAsync(context, next);
    Assert.AreEqual(StatusCodes.Status403Forbidden, context.Response.StatusCode);
}
```

These two tests are part of the authentication middleware test. This test class utilizes the user initialize and cleanup at every test. For the first test there were a valid access token and a common path. A common path is any path that has not to be handled in a specific way, for example the sign up where tokens are not yet created. HttpContext was created with all the necessary values; path, method, and authorization header. A request delegate, that just returns a completed task, was also created. The middleware was tested by running it with the HttpContext and RequestDeletegate. Using the assertion afterwards, the expected response of the HttpContext was found to be 200.

Next was testing for an expired access token. The test was set up the same way. The access token was expired by utilizing a helper class. It updates the token's expiration date directly in the database. Expected error was 403 instead for an expired token. The middleware for the error handling was tested in a very similar way.

Test examples for the second layer, the general endpoint.

```
    [TestMethod]
public async Task PostStockTransactionsTest_PostSuccessfullyTest()
{
    PostStockTransactionsBody postStockTransactionsBody = new
        PostStockTransactionsBody(
       successfulStockTransaction.portfolioId!,
            ...
    );
    PostStockTransactionsResponse response = await
        PostStockTransactions.Endpoint(postStockTransactionsBody,
        userTestObject.accessToken!);
    Assert.IsTrue(response.response == "success", "Response should be success
        but was " + response.response);
    Assert.IsTrue(response.id != 0, "Response id should not be 0");
}

    [TestMethod]
public async Task PostStockTransactionsTest_MissingElementsTestTest()
{
    // Missing portfolio id
    StockApp.StockTransaction stockTransactionData = new
        StockApp.StockTransaction();
    stockTransactionData.portfolioId = portfolio.id;
    stockTransactionData.amount = 1;
    stockTransactionData.exchange = "NASDAQ";
    stockTransactionData.timestamp =
        Tools.TimeConverter.DateTimeToUnix(DateTime.Now);
    stockTransactionData.priceNative = new StockApp.Money(100, "USD");
    PostStockTransactionsBody postStockTransactionsBody = new
        PostStockTransactionsBody(
       stockTransactionData.portfolioId!,
            ...
    );
    StatusCodeException exception = await
        Assert.ThrowsExceptionAsync<StatusCodeException>(async () => await
        PostStockTransactions.Endpoint(postStockTransactionsBody,
        userTestObject.accessToken!));
    Assert.IsTrue(exception.StatusCode == 400, "Status code for missing
        portfolio id should be 400 but was " + exception.StatusCode);

        // Missing exchange
        ...
        // Missing ticker
        ...
        // Missing timestamp
        ...
        // Missing amount
        ...
        // Missing price currency
        ...
    }
```

Above are two tests, that tests the post stock transaction endpoint. This test class utilizes the initialize and cleanup of a user at every test. Initialize also created a portfolio; ready to add the transaction to. It also pre-defines a ”successfulStockTransaction”, which holds all the elements

of a successful transaction.

The first test is simply to test a successful posting of a stock transaction. It sets up a body, by defining each of the elements and using the pre-defined successful transaction. It then calls the endpoint method with the body and access token of the created userTestObject. The response is then asserted to be success, and the id we are getting back, that is corresponding to the created transaction, should then not be 0. The second test, tests missing elements. This is a common test for all the different endpoints, where, if the body has multiple different elements, a missing element test has been set up to test missing every value. As seen here, we have where it is missing the portfolio id, where multiple tests underneath it then tests for other missing elements. We here, when calling the endpoint, asserts that a StatusCodeException should be thrown. This exception should then have a status of 400 for bad request, as there is missing values.

Test example for the last layer of the tests, the business logic layer.

```
[TestMethod]
public void PortfolioTest_ChangeName_SuccessfulTest()
{
   portfolio.AddToDb();
   portfolio.ChangeName("New name");
   Portfolio gottenPortfolio = PortfolioHelper.Get(portfolio.id!);
   Assert.IsTrue(gottenPortfolio.name == "New name", "Portfolio name should be
       \"New name\" but was \"" + gottenPortfolio.name + "\"");
}

[TestMethod]
public void PortfolioTest_ChangeName_NullPortfolioIDTest()
{
      portfolio.AddToDb();
   portfolio.id = null;
   StatusCodeException exception =
       Assert.ThrowsException<StatusCodeException>(() =>
       portfolio.ChangeName("New name"));
   Assert.IsTrue(exception.StatusCode == 400, "Status code should be 400 but
       was " + exception.StatusCode);
}
```

Above code snippet shows two tests, in which the name of the portfolio was tried to be changed. The file initializes a user and sets up a successful portfolio. It was not adding it to the database, as other tests in the file, were doing tests that did not require the portfolio to be added to the database yet. The first is a successful changing of name test. It only utilizes functions from the business logic, such as adding the portfolio to the database, and then calling the change name function on the portfolio directly. By using a helper function afterwards, the portfolio was retrieved from the database, where it was asserted that the name gotten of the portfolio, was the same as the new name given. The second test is a null value test. Similar to the missing elements one, the business logic tests different values for if they are null. Here the portfolio was added to the database, then id of the portfolio object was set to null. Then it was asserted, that the change name method of the portfolio object, is going to throw a StatusCodeException. This should be a 400, as there was missing values for changing the name.

While all tests would always cover missing elements and null values, they also covered calling endpoints for every single currency, if the endpoint takes a currency parameter. An example of such a test, is the one below, that loops over a list of the supported currencies of the application.

```
    [TestMethod]
public async Task UserTest_GetTransactions_GetAllCurrencies()
{
    StockApp.Portfolio portfolio = PortfolioHelper.Create(userTestObject);
    StockApp.StockTransaction stockTransaction = new StockApp.StockTransaction();
    stockTransaction.portfolioId = portfolio!.id;
    stockTransaction.ticker = "VICI";
    stockTransaction.exchange = "NYSE";
    stockTransaction.amount = 10;
    stockTransaction.timestamp =
        Tools.TimeConverter.DateTimeToUnix(DateTime.Now) - 8000000;
    stockTransaction.priceNative = new StockApp.Money(100, "CAD");
    PostStockTransactionsBody postStockTransactionsBody = new
        PostStockTransactionsBody(
       ...
    );

    PostCashTransactionsBody postCashTransactionsBody = new
        PostCashTransactionsBody(
       ...
    );
    PostCashTransactionsResponse postCashTransactionsResponse = await
        PostCashTransactions.Endpoint(postCashTransactionsBody,
        userTestObject.accessToken!);
    Assert.IsTrue(postCashTransactionsResponse.response == "success", "response
        should be success but was " + postCashTransactionsResponse.response);

    foreach (String currency in Dictionaries.currencies)
    {
        GetTransactionsResponse transactions = await
            GetTransactions.Endpoint(userTestObject.accessToken!, currency);
        Assert.IsTrue(transactions.response == "success", "response should be
            success but was " + transactions.response);
        Assert.IsTrue(transactions.transactions.All(transaction =>
            transaction.balance!.currency == currency), "currency should be " +
            currency + " but was " +
            transactions.transactions[0].balance!.currency);
        Assert.IsTrue(transactions.transactions.All(transaction =>
            transaction.combinedBalance!.currency == currency), "currency should
            be " + currency + " but was " +
            transactions.transactions[0].combinedBalance!.currency);
        Assert.IsTrue(transactions.transactions.All(transaction =>
            transaction.value!.currency == currency), "currency should be " +
            currency + " but was " +
            transactions.transactions[0].value!.currency);
    }
}
```

The test above is on the user object in business logic. On the method of getting all the transactions for a user, retrieving all of transactions in all registered currencies in the database. This file initializes by creating a user and a portfolio. The test itself then creates a stock and a cash transaction. With this successfully created, the test jumps down to the for-each loop. This loops through all of the currencies in a list from another file, representing all the supported currencies in the database. It then loops through, get the transactions for each of them in the specified currency, and then checks through the complete list, that should contain information

about which currency it is, and assert that it is equal to the currency in the loop. All tests that has currency as a parameter, has this kind of test to make sure that they can all handle the different kind of currencies that we support in the database.

Another type of tests, is the test of getting a stock from every, single exchange in the database. This is mostly to check if every exchange is working correctly, and that all exchanges has their data formatted correctly, as for the endpoint of getting stock profiles to successfully get it.

This in total gets an as complete tests foundation as possible, that covers all the different kind of errors, exceptions and successful tests that the application can encounter, on the different layers. By setting this general foundation for tests, as to what they need to cover, test-driven development was also used. By having a foundation of the tests, we could then create the tests for new implementations, before implementing them. This allows for a significantly easier overview of the implementation of new features, as before even implementing them, we have already created the expected behaviour of it, by creating tests to test the successful outputs and the various errors. The test seen above throughout this section, was all used as part of the test driven development, as they all defined the behaviour and flow of a new implementation. It could then be implemented and easily tested, to see if the implementation actually followed the expected behaviour or not.

6.1.1 Test Results and Code Coverage

For showcasing the code coverage of the unit tests on the backend, the coverlet collector was used. It integrates easily with the .NET and MSTest framework. Coverlet collector generates an info file in the lcov format after running all the tests, that showcases each of the steps that it took while executing. This gives the total percentage of coverage of lines, branches, and methods. To get a better overview the include tag of coverlet are used, to just test the coverage for certain namespaces, to see the coverage of each of them. Such a command could look like this.

```
dotnet test /p:CollectCoverage=true /p:CoverletOutputFormat=lcov
    /p:CoverletOutput=./lcov.info /p:Include="[*]*API.v1*"
```

This command runs all tests while telling coverlet to collect coverage in an lcov format, and output it to a file called lcov.info. For this specific command, we then added the include filter, to only test for the ”API.v1” namespace, which is the namespace that contains the setup and routing for all the endpoints. Lets now take a look at the individual namespace and their coverage.

Exception

- Lines - 96%
- Methods - 100%
- Branches - 100%

The Exception namespace, referring to the error handling, is very close to be at 100 total percent. The only thing missing is when an unhandled exception is thrown, one that is not StatusCodeException, as this can not be forced, when all errors are handled correctly.

Tools

- Lines - 83.7%
- Methods - 100%
- Branches - 76.7%

The Tools namespace, referring to extra functions that takes care of time converting, currency checking and converting etc., is missing from both lines and branches. The reason behind this, is some very specific checks for the currency conversion, that is specific for when we get data from Yahoo Finance. This can not be forced, as we can not actively choose to get data from Yahoo Finance. However, this still tests close to everything for the namespace.

StockApp

- Lines - 93.4%
- Methods - 100%
- Branches - 92%

The StockApp namespace, refers to most of the business logic of the application. With 100% and very close to 100% for both lines and branches, the business logic is significantly covered. Was is missing from getting it to 100%, is mostly catches for if an unknown database exception happens. This is for example, if trying to insert an entry in the database, but because of connection issues or something else, an exception will happen. This is not exceptions we can force and should generally not happen, which means we can therefore not test them, making the total coverage of the StockApp namespace significant.

Data

- Lines - 78.9%
- Methods - 92.8%
- Branches - 31.1%

The Data namesapce, refers to the fetchers of the application. This is the lowest covered namespaces of all of them. This due to the Yahoo Finance fetchers, and generally how to deal with data from Yahoo Finance. Getting stocks from Yahoo Finance has showcased, that the data might sometime be different in its format, depending on the stock. Sometimes column names will be sent with it, sometimes not. We also have different checks for connection issues, null values in the price data, or how to handle the stock profile information, as it various a lot on which information is present and what is not. Because of the inconsistent we get from Yahoo Finance, it is simply not possible to test it correctly. By therefore breaking the Data namespace into the Yahoo Finance fetcher and the database fetcher, we see the following coverages:

- Yahoo Finance Fetcher:
  - Lines - 72.8%
  - Methods - 85.7%
  - Branches - 15%

- Database Fetcher: .
  - Lines - 91.1%
  - Methods - 100%
  - Branches - 78%

These results shows the database fetcher is significantly better covered than the Yahoo Finance fetcher. This is how it should be, as because of the dependency inversion, the Yahoo Finance fetcher should be easy to switch out with any other API. However, the database fetcher will always be the same, meaning that should be tested thoroughly. Looking at the split up fetchers, we can then say that even though the Data namespace does not seem to be tested correctly overall, it is indeed covered significantly.

Authentication

- Lines - 91.3%
- Methods - 100%
- Branches - 95.2%

The Authentication namespace, refers to everything that got to do with authenticating the user and the whole token system. IT is close to having 100% coverage, with the exception of lines and branches. These two are, again, because of database exceptions, such as inserting, updating and deleting, which can not be forced. The coverage is therefore significant of this namespace.

API.v1

- Lines - 89.5%
- Methods - 100%
- Branches - 98.6%

The API.v1 namespace, refers to everything that has to do with setting up and handling endpoints. This is also the setup of the application itself. The missing lines, comes from every setup method of the endpoints and the setup of the application itself. Every setup method of the endpoint, is equipped with getting the access token as an item from the HttpContext from the middleware. In the test of setting the endpoints up, a HttpContext was not created, meaning this is all the missing lines. As the access token is checked to be present in the middleware, there is no reason to test for the endpoints to get the access token from the HttpContext, meaning that the coverage is significant.

Total Application

- Lines - 87%
- Methods - 97.7%
- Branches - 59.3%

The total application comes to a rather significant coverage. Because of the very low line coverage of the data namespace, it significantly decreases the average branch coverage from the others. However, not counting in the Yahoo Finance fetcher and methods associated directly with Yahoo Finance, the coverage of the application as a whole, is significant to test every aspect of its features.

6.2 Manual Tests

Manual tests was done both for the frontend and the backend. Manual tests utilizes the ability of moving through the application as a normal user would. It can create scenarios that has not been covered by unit testing, as unit testing it more used to verify if components work as intended. Manual testing has the ability to do whatever the user feels like, explore the boundaries of the application, what can and can not be done, etc. Manual testing is also very useful through the development and debugging process.

For the backend, manual testing was done through Postman. Postman allowed us to set up every individual endpoint and call it with different values, being that in the header, the query parameters or the body. This way, it was possible to experiment with multiple different values and test what would happen. Because of the use of Postman, requesting and testing the endpoints were much more simple, as Postman could display everything that was returned from the server, being that the response itself, the HTTP status code, while also showcasing the amount of time it took for the request to get a response. By getting the time of it too, we could regularly monitor any endpoints that would take a long time to response, and investigate the reason behind it. By using Postman, and calling an endpoint correctly with the path and method, we get a complete authentic experience of the end-to-end user structure, as this is how the frontend will be calling the backend. This is also tested in the unit tests by calling the endpoint methods, but using Postman we could then call the endpoint path directly, as the frontend would.

The frontend was only tested using manual tests, not unit testing. The reason behind this, is the structure of the frontend which have had its business logic kept to a minimum. The frontend should be able to, as close as possible, to take data it got from the backend and show it to the user directly, be it in a graph or a message for the user when something has gone wrong. Because of this, unit tests was not done for the frontend as the business logic was so minimal. The frontend was tested thoroughly with manual testing, as the frontend depends on the design. This is where we have to do continuous manual testing to see if the design works correctly for different screens. This way we can also test to see if the data gotten from the backend looks correctly when implementing it with the design of the frontend.

7 Discussion

For the project's conclusion, it is significant to address the entirety of the process, commencing from the idea generation of the application and ending with the application development. From the outset, the application was conceived based on the valuable input obtained through a questionnaire. This questionnaire, distributed among potential users in different stock group enthusiast groups on Facebook, provided a direction for the features of the app, merging their requests with our own personal desires for features.

Even though the sample size was relatively small, with only 15 replies, the feedback greatly influenced the design of the application, confirming the existence of a potential user base. Through this merge of user responses and our ideas, the groundwork was established for a solution. This solution was crafted to target the shortcomings of our competitors, by incorporating missing features and implementing different "quality of life" angles.

Agile development was employed, utilizing user stories and a user story map to outline the features of the application. The map highlighted the overall functionality of the application and distributed the features across five different releases based on their priority. This approach ensured the use of agile development principles and tried to balance the workload across releases.

The architecture design for the frontend, backend, and database provided a solid foundation for the application and future implementations and improvements. The middleware was a vital and centralized piece for error handling and authentication, with the value history object summarizing the total assets of the user. Nevertheless, a more robust foundation could have been achieved by incorporating pre-existing technologies for authentication and password handling. Considering the security of the project was not a primary focus, these were considered too complex and time-consuming to implement.

Dependency inversion for the backend was crucial due to the potential necessity of switching to a different third-party API than Yahoo Finance. This made it vital to isolate Yahoo Finance functions from the rest of the application, minimizing any change impacts on the API. Similarly, the "ON DELETE" statements in the database design facilitated data cleanup, although it could be argued that more tables could have better implementations to follow the third normal form.

The frontend development was done entirely in Flutter, which allowed for the same code and application to be used across different platforms. However, this choice limited some functionalities of the web application compared to using Angular. If more time for development becomes available in the future, switching to Angular for the web application could be considered for better specialization.

Reviewing the user story map, the first three releases were fully implemented, providing a solid foundation for the application. Release four and five were partially or not implemented at all, due to limitations from the API and time constraints. We realized that the initial time plan was too optimistic, but prioritizing the creation of a solid foundation proved beneficial for the overall quality and simplicity of future development.

Unit testing played a crucial role in the development process, ensuring the stability of existing features while implementing new ones. With almost 100% code coverage of the most important parts of the backend, including API endpoint logic and business logic, the tests greatly contributed to the development of the application.

8 Conclusion

In conclusion, despite a few shortcomings of implementations of features, the project has resulted in a complete and functional multi platform stock tracking application. It effectively integrates all the pieces of the backend, frontend and database, with only a small amount of features missing implementation, from the user story map. Future development should from here, focus on implementing the remaining of the features, to enhance the overall user experience of the application.

Despite a challenging project and the struggles we faced, we were able to lay out a robust groundwork and gain a massive amount of knowledge and understandings, about new technologies in C#, Flutter and T-SQL as well as good practices in building out larger projects by having dependency inversion and scalability in mind while also defining what each part of the solution is responsible for. This constant learning curve greatly benefited us throughout the project, and will benefit us greatly in the future.

Moving forward, for the application to be further developed and later on published we still have some work to do. There is a need to implement the final features to enhance user experience. We do also have to switch away from the Yahoo Finance API to another API due to restrictions on commercial use. These goals are very attainable because of how few missing features remains and the simplicity introduced by the dependency inversion to switch APIs. With further development, there also comes into consideration of the use of online services, such as Azure, not being free anymore. It helps in the current version, but publishing the application and growing the user base, will prove the need of upscaling. A new strategy will have to be discussed, as to how the application can integrate better security and upscaling, while also having a user base that is large enough and willing to pay for certain features.

The application was designed with a modular framework. This enabled in the in-sourcing of features and will allow for compartmentalizing future implementations and improvements. Unit testing proved each functionalities of the modules, and confirmed the design of the overall architecture.

Considering the time invested and put into the project as a whole from start to finish including the application development and report writing we believe the time was well spent. The solid foundation laid during development and the alignment of user feedback from the questionnaire with direction of the application provides a strong motivation for future development. Overall we are highly satisfied with the application, the report and the general process of the development flow.

9 Glossary

Broker A stock broker is a licensed company who facilitates the buying and selling of stocks on behalf of investors . 8

cryptocurrency A digital currency that uses cryptography for security and operates on decentralized networks. Examples include Bitcoin, Ethereum, and Litecoin.. 5

dividend A distribution of a company's earnings to its shareholders. It is typically paid out in cash.. 4

ETF An ETF (Exchange-Traded Fund) is a type of investment fund that is traded on stock exchanges, which combines a bunch of stocks into a single fund, providing greater diversity for the investor.. 4

forex Forex, or foreign exchange, is the global decentralized market where currencies are traded. The act of forex, is the act of trading currencies . 6

Index Fund An index fund is a type of ETF that aims to replicate the performance of a specific market index, such as the S&P 500. 4

reverse split A corporate action where a company merges existing shares into fewer shares, resulting in an increase in share price and a decrease in the number of outstanding shares . 9

stock A stock refers to a financial asset represented by a unique symbol called a ticker, which is assigned to publicly traded shares of a company and is associated with a specific stock exchange where it can be bought and sold.. 4

stock split A corporate action where a company splits its existing shares into multiple shares, resulting in a decrease in share price and an increase in the number of outstanding shares of the company . 9

stock tracking application A software application designed to monitor and manage the performance, prices, and development of various stocks on the stock market . 4

10 References

[1] Savindhya Amavi Surasena Bootcamp. What is cognitive load in ux? https://bootcamp.uxdesign.cc/what-is-cognitive-load-in-ux-ff068fb8c44d. An article on the basics of cognitive load theory and how to implement it.

[2] European Commission. What personal data is considered sensitive? https://commission.europa.eu/law/law-topic/data-protection/reform/rules-business-and-organisations/legal-grounds-processing-data/sensitive-data/what-personal-data-considered-sensitive_en. Article by the European Commission on what data is considered sensitive.

[3] Finnhub. Finnhub stock apis. https://finnhub.io/. Homepage of the Finnhub API website.

[4] Google. Finance data listing and disclaimers. https://www.google.com/googlefinance/disclaimer/. Overview of the exchanges supported by Google Finance.

[5] Google. Material design 3. https://m3.material.io/. Official website of Material Design 3.

[6] Google. Online security survey - google / harris poll. https://services.google.com/fh/files/blogs/google_security_infographic.pdf. Overview of a security survey on passwords done by Google in 2019.

[7] Anne Haxthausen and Flemming Schmidt. 7. normalization. https://learn.inside.dtu.dk/d2l/le/content/60224/viewContent/244519/View. Slides about normalization from course 02170 Database Systems Spring 21 at DTU.

[8] Steve Krug. *Don't Make Me Think, Revisited - A Common Sense Approach to Web Usability*. New Riders; 3rd edition, 2003.

[9] Sjoerd Langkemper. The case for client-side hashing: logging passwords by mistake. https://www.sjoerdlangkemper.nl/2020/02/12/the-case-for-client-side-hashing-logging-passwords-by-mistake/. Article about hashing password on the client.

[10] marketstack. marketstack - real-time & historical stock data api. https://marketstack.com/. Homepage of the marksetstack API website.

[11] Meta. Keeping passwords secure. https://about.fb.com/news/2019/03/keeping-passwords-secure/. Mete was logging plain text password.

[12] Microsoft. Resource limits for single databases using the dtu purchasing model - azure sql database. https://learn.microsoft.com/en-us/azure/azure-sql/database/resource-limits-dtu-single-databases?view=azuresql. Article about the resource limits of the Azure SQL Database.

[13] OAuth. Oauth 2.0. https://oauth.net/2/. Official website for the OAuth 2.0 protocol.

[14] Okta. Refresh access tokens. https://developer.okta.com/docs/guides/refresh-tokens/main/. Description of refresh token, refresh token rotation and refresh token reuse detection.

[15] Polygon. Market data apis. https://polygon.io/. Homepage of the Polygon API website.

[16] Postman. Postman api platform. https://www.postman.com/. Official website for the Postman API Platform.

[17] GDPR & the European Union. Writing a gdpr-compliant privacy notice (template included). https://gdpr.eu/privacy-notice/. Article about writing a GDPR privacy notice, with the inclusion of a PDF template.

[18] Lisa Vaas. Instagram accidentally reveals plaintext passwords in urls. https://nakedsecurity.sophos.com/2018/11/20/instagram-accidentally-reveals-plaintext-passwords-in-urls/. Instagram put users password in URLs.

[19] Alpha Vantage. Stock market api, reimagined. https://www.alphavantage.co/. Homepage of the Alpha Vantage API website.

[20] Yahoo! Exchanges and data providers on yahoo finance. https://help.yahoo.com/kb/exchanges-data-providers-yahoo-finance-sln2310.html. A complete list of all exchanges and data providers available on Yahoo Finance.

[21] Yahoo! Yahoo finance. https://finance.yahoo.com/. The official Yahoo Finance website. As the API has been deprecated since 2017, there is no official page for the API.

11 Appendix

11.1 Appendix 1 - Questionnaire



Fullstack Multiplatform Aktie Tracking Applikation

Tusind tak for at have klikket på vores spørgeskema! Vi er to studerende fra **DTU (Danmarks Tekniske Universitet)**, som skal til at lave vores bachelor i Softwareteknologi. Navnet på projektet lyder " **Fullstack Multiplatform Aktie Tracking Applikation**". Vi er altså i gang med at lave en applikation, hvorpå man kan spore sine aktier/crypto/andet, både på telefonen som app, men også på en hjemmeside.

I forbindelse med dette projekt, vil vi meget gerne høre om dine ideer og tanker til, hvad du synes andre platforme mangler og gør godt.
I sidste ende - **hvordan ser din perfekte platform ud?**

Denne undersøgelse er 100% anonym, og intet er obligatorisk at udfylde.

Hvor mange aktier/indeksfonde/cryptovaluta'er/andet har du i din samlet portefølje? Gerne opskriv dem individuelt
(Aktier: 5
Crypto: 10
osv...)

Long-answer text

Hvilke platforme anvender du til at spore disse? (Yahoo Finance, Nordnet, Saxo, Google Finance, andet)

Nævn meget gerne alle dem du bruger

Long-answer text

Hvilke funktioner synes du er geniale, som disse platforme har?

Long-answer text

Hvilke funktioner synes du, hvis nogen, der mangler i disse platforme?

Long-answer text

Hvilke funktioner ville din perfekte platform have, som andre ikke har? Dette kan også være funktioner som er splittet ud på flere platforme, og du blot vil have dem samlet et sted.

Long-answer text

Hvor stor en sandsynlighed ville der være for at du ville udskifte dine nuværende platforme, med en ny?

1 2 3 4 5 6 7 8 9 10

Jeg er ikke til at rykke i ○ ○ ○ ○ ○ ○ ○ ○ ○ ○ Jeg er så på, gi' mig noget nyt

Hvorfor ser din sandsynlighed sådan ud?

Long-answer text

Figure 24: The form of the questionnaire, with the questions in Danish

11.1.1 Answers

	A	B	C	D	E	F	G	H
1	Timestamp	Hvor mange aktier/indeksfonde/cryptovaluta'er/andet har du i din samlet portefølje? Gerne opskriv dem individuelt (Aktier: 5 Crypto: 10 osv...)	Hvilke platforme anvender du til at spore disse? (Yahoo Finance, Nordnet, Saxo, Google Finance, andet) Nævn meget gerne alle dem du bruger	Hvilke funktioner synes du er geniale, som disse platforme har?	Hvilke funktioner synes du, hvis nogen, der mangler i disse platforme?	Hvilke funktioner ville din perfekte platform have, som andre ikke har? Dette kan også være funktioner som er splittet ud på flere platforme, og du blot vil have dem samlet et sted.	Hvor stor en sandsynlighed ville der være for at du ville udskifte dine nuværende platforme, med en ny?	Hvorfor ser din sandsynlighed sådan ud?
2	10/01/2023 16:45:50	60	euroinvestor	de følger op på kursen dalig	køb/salg oversigt burde være nemmere at overksue	en udreging der viser hvad købskursen på restrende aktier ville være hvis jeg solgte en del til dagskurs	8	håber altid på en bedre
3	10/01/2023 16:47:54	Indeksfond: 1	Nordnet	Overvågningslister	Er forholdsvis ny i aktie-gamet, så Facebook gruppen i har sendt jeres link på har været et godt lære sted.	Syntes Nordnet har en rigtig overskuelig platform med det der hører med til.	2	Nordnet er en rigtig god platform, for ny i aktie-gamet.
4	10/01/2023 16:50:07	Aktier 12 Fonde 10	Nordnet, NORD.Investment	Månedsopsparing hos Nordnet			2	Jeg synes Nordnet har en ganske glimrende produkt for de fleste hobby aktionærer
5	10/01/2023 17:01:57	Aktier: 30+, fonde: 5+	Et mix af brokeren selv og min egen webapp jeg har lavet specifikt til formålet	Ingen	Mulighed for at registrere udbetalte/modtagne udbytter - det har min egen webapp dog.. ;)	Det er ret så væsentligt at der tages højde for skatteforhold, især i forhold til udbytter . Udenlandske trackere fatter jo intet af dansk skat.	5	Der skal nogle hårdtslående argumenter til. Og det skal være gratis. Ellers er der ingen ide i at skrotte min egen webapp.
6	10/01/2023 17:10:42	Aktier: 50+ ETFer: 5+ på forskellige konti Copytrade på crypto for lille beløb. 8-12 forskellige. Et par fonde	Google finance og Yahoo til det hurtige. Excel til porteølje, overblik og lidt værdiansætning. Forskellige kilder til analyser, blandt andet GMS, jyske bank og private investorer. Derudover benytter jeg eToro til at følge med i andre porteføljer for inspiration.	Jeg synes generelt det er "nemt" at finde de regnskaber jeg gerne vil benytte, inspiration til nye metoder til værdiansætning mv.	Dog savner jeg et værktøj hvor man kan plotte alle ens aktier ind, som kan give en lidt dybere analyse af ens portefølje og risiko. Hvor meget ligger i de forskellige industrier, hvilke forretningsmodeller er der overvægt af, hvilke lande, samt hvilke styrker og svagheder dette har. Problemet kan også være at data ikke bliver hentet ned, så hvis man lavede en platform hvor det meste er digitalt, men hvor der er en mulighed for at tilføje data eller egne kategorier		5	Det er omstændigt at rykke og vende sig til noget nyt. Så det skal være en super løsning, som man ved virker. Man skifter sjælendt for bare at prøve noget kort
7	10/01/2023 21:41:13	Aktier 7	Nordnet	Giver et godt overblik	Jeg savner at det kan spore alle mine aktiver	Den perfekt platform ville indholde flere sociale aspekterne !	10	Fordi jeg savner overblik og en platform med flere aspekter
8	10/01/2023 21:51:58	Aktier 7	Nordnet, Yahoo, Simply WallStreet	Fairvalue, insider oplysninger	Beskrivelse af Intjeningsstyrke	AI drevet teknisk analyse	8	Der kan altid være noget der er bedre
9	10/01/2023 21:54:27	aktier 5 crypto 5	Yahoo Finance	Deres app fungerer super godt hvor man bare kan lægge sin købspris ind og evt hvis man køber flere udregner den blot gennemsnitspris automatisk og viser afkast i real time	kan ikke komme i tanker om nogen		6	Den fungerer allerede til mit forbrug
10	10/01/2023 22:26:09	51	Google Finance, eget excelark og trackyourdividend	Jeg er glad for at PF kan opgøres i valgfrie valutaer	kan ikke komme i tanke om noget	Valutafunktionen fra Google Finance samt [en udvidet] udbyttefremskrivningen fra Trackyourdividend samlet i samme opgørelse kunne være rar at have	10	Jeg leder efter noget der kan hjælpe med overblik. Udvikler lidt i eget regi i excel for bedre at holde styr.
11	11/01/2023 00:08:02	Aktier: 1 Fonde: 1 Crypto: 6	Nordnet, CoinMarketCap, CoinGecko	De viser hvor meget jeg har og historikken over hvad det har været værd	Jeg vil gerne kunne se alle mine investeringer et sted.	Jeg vil gerne kunne se alle mine investeringer et sted.	10	Har ikke opbygget loyalitet til nogle platforme, så hvis der er noget smartere så er jeg klar
12	11/01/2023 09:39:40	Aktier: 5 Investeringsforeninger/ indeksfonde: 7	Yahoo Finance vha python og yfinance pakken	Mulighed for at hente data ned og behandle det vha python	Historisk data for (danske) investeringsforeninger/indeksfonde	Forventet fremtidig udbytte Estimeret skat Watch liste	4	Jeg har brugt lang tid på at bygge de tracking funktioner, jeg er interesseret i, med det data der er tilgængeligt på Yahoo Finance, i Excel/python
13	11/01/2023 10:32:53	Aktier: 2, Invest.Foreninger: 3	DanskeBank, Børsen, Yahoo Finance, Euroinvestor	Euroinvester angiver gratis kurser i realtid. Børsen skriver relevante nyheder.	Den gennemsnitlige handelskurs de seneste 3 eller 6 måneder. 6 Det gennemsnitlige antal papirer handlet pr. dag i de seneste 3 eller 6 måneder.	Se ovenstående svar.	7	Jeg tvivler på, at i kan lave en platform, der opfylder alle mine ønsker incl. nyheder.
14	11/01/2023 11:45:55	20	Nordnet, Google Finance, tradingview.com	Nordnet: Alarm funktion, tradingview.com er simpelthen unik - med fx. avancerede alarmer osv.	At kunne sætte alarmer, hvor forskellige instrumenter deltager i kriterierne som udløser alarmen	Fx.: At kunne sætte alarmer, hvor forskellige instrumenter deltager i kriterierne som udløser alarmen, og at alarm kan gives via APP på telefon, SMS, eller mail.	9	Formål: At reducere mit daglige tidsforbrug, og øge mine muligheder for at "time markedet" bedre.
15	11/01/2023 13:43:50	Aktier: 12 Indeksfonde: 3 Crypto: 3 Forex: 1	Euroinvestor samt Nordnet	Euroinvestor bruges til at holde styr over nyheder der påvirker aktiemarkedet, både kort og langsigtet. Nordnet bruges til at danne overblik over enkelte aktier. Det gode ved Nordnet er, det giver et let overblik over kursudviklingen over forskellige tidshorisonter.	Det kunne godt have været fedt hvis der var mulighed for at se enkelte dage med fuldt overblik tilbage i tiden. Så fx d.15/5-2019, hvor man kunne se kursudviklingen for hele dagen. Det kan bruges til at se, hvordan aktien regerede på en bestemt nyhed, og derved give et estimat over hvordan aktien vil udvikle sig ved lignende nyheder. Noget mere teknisk analyse, fx bar chart	Normalt Line chart, bar chart, mulighed for at se enkelte dage med fuldt overblik tilbage i tiden. Nyheder for den enkelte aktie, når den vælges, således det kommer frem i bunden og giver et fint overblik over nyheder	10	Jeg har endnu ikke fundet en platform hvor jeg kan få alt information på en gang, så hvis dette var muligt, vil jeg skifte til den med det samme, derved undgå at skulle bruge flere forskellige platforme
16	13/01/2023 11:37:24	12 aktier, 3 fonde og en slags obligation.	Yahoo finance primært.	Samler de fleste nyheder omkring et selskab et sted	Der er ikke ALLE nyheder omkring selskabet kun en andel og artikler der indeholder mindre updates bliver ikke udgivet	Samle ALLE analyse og artikler og informationer om selskaber et sted.	9	Jeg leder efter noget hvor alt er samlet et sted.
17								
18		**New assets to think of:**	**Platforms to inspire from / take a look at:**	**New features:**	**New features:**	**New features:**		
19		Nothing new	portfolio-performance.info https://www.trackyourdividends.com/ https://simplywall.st/ https://www.tradingview.com/	Overview of how analysis work and how the formula for them is calculated Analysis: fairvalue Export portfolio data as CSV, JSON, whatever Display newest articles about an asset, general news	(Deeper analysis of portfolio) (Full overview of an assets worth on a specific day back in time)			

Figure 25: The answers from the questionnaire, put into a Google Sheets for better overview.

11.2 Appendix 2 - JSON schemas of the REST API

This appendix contains the JSON schema of every single endpoint of the application.

11.2.1 /users POST

```
{
    "endpoint": "/v1/users",
    "method": "POST",
    "description": "Create a new user",
    "request": {
        "headers": [
            {
                "name": "Content-Type",
                "type": "string",
                "description": "Request content type"
            }
        ],
        queryParameters: [ ],
        "body": {
            "email": {
                "type": "string",
                "required": true,
                "description": "Email of the user"
            },
            "password": {
                "type": "string",
                "required": true,
                "description": "Password of the user"
            }
        }
    },
    "response": {
        "body": {
            "response": {
                "type": "string",
                "description": "Response status"
            },
            "uid": {
                "type": "string",
                "description": "The id of the created user"
            }
        }
    }
}
```

11.2.2 /users DELETE

```
{
    "endpoint": "/v1/users",
    "method": "DELETE",
    "description": "Deletes a user",
    "request": {
        "headers": [
            {
                "name": "Authorization",
                "type": "string",
                "required": true,
                "description": "Access token"
            },
            {
                "name": "Content-Type",
                "type": "string",
                "required": true,
                "description": "Request content type"
            }
        ],
        queryParameters: [ ],
        "body": { }
    },
    "response": {
        "body": {
            "type": "object",
            "properties": {
                "response": {
                    "type": "string",
                    "description": "Response status"
                }
            }
        }
    }
}
```

11.2.3 /users/email PUT

```
{
    "endpoint": "/v1/users/email",
    "method": "PUT",
    "description": "Changes a users emails",
    "request": {
        "headers": [
            {
                "name": "Authorization",
                "type": "string",
                "required": true,
                "description": "Access token"
            },
            {
                "name": "Content-Type",
                "type": "string",
                "required": true,
                "description": "Request content type"
            }
        ],
        queryParameters: [ ],
        "body": {
            "newEmail": {
                "type": "string",
                "required": true,
                "description": "The new email to change to"
            },
            "password": {
                "type": "string";
                "required": true,
                "description": "New hashed password with email salt"
            }
        }
    },
    "response": {
        "body": {
            "type": "object",
            "properties": {
                "response": {
                    "type": "string",
                    "description": "Response status"
                }
            }
        }
    }
}
```

11.2.4 /users/password PUT

```
{
    "endpoint": "/v1/users/password",
    "method": "PUT",
    "description": "Changes a users password",
    "request": {
        "headers": [
            {
                "name": "Authorization",
                "type": "string",
                "required": true,
                "description": "Access token"
            },
            {
                "name": "Content-Type",
                "type": "string",
                "required": true,
                "description": "Request content type"
            }
        ],
        queryParameters: [ ],
        "body": {
            "oldPassword": {
                "type": "string",
                "required": true,
                "description": "The old password of the user"
            },
            "newPassword": {
                "type": "string",
                "required": true,
                "description": "The new password of the user"
            }
        }
    },
    "response": {
        "body": {
            "type": "object",
            "properties": {
                "response": {
                    "type": "string",
                    "description": "Response status"
                }
            }
        }
    }
}
```

11.2.5 /users/preferences GET


```
{
  "endpoint": "/v1/users/preferences",
  "method": "GET",
  "description": "Gets all preferences of a user",
  "request": {
    "headers": [
      {
        "name": "Authorization",
        "type": "string",
        "required": true,
        "description": "Access token"
      },
      {
        "name": "Content-Type",
        "type": "string",
        "required": true,
        "description": "Request content type"
      }
    ],
    queryParameters: [ ],
    "body": { }
  },
  "response": {
    "body": {
      "type": "object",
      "properties": {
        "response": {
          "type": "string",
          "description": "Response status"
        },
        "preferences": {
          "type": "array",
          "items": {
            "type": "object",
            "properties": {
              "key": {
                "type": "string",
                "description": "The preference of the user"
              },
              "value": {
                "type": "string",
                "description": "The value of the preference"
              }
            }
          },
          "description": "User preference list"
        }
      }
    }
  }
}
```

11.2.6 /users/preferences POST


```
{
    "endpoint": "/v1/users/preferences",
    "method": "POST",
    "description": "Creates a user preference",
    "request": {
        "headers": [
            {
                "name": "Authorization",
                "type": "string",
                "required": true,
                "description": "Access token"
            },
            {
                "name": "Content-Type",
                "type": "string",
                "required": true,
                "description": "Request content type"
            }
        ],
        queryParameters: [ ],
        "body": {
            "portfolioId": {
                "type": "string",
                "required": true,
                "description": "The id of the portfolio"
            },
            "setting": {
                "type": "string",
                "required": true,
                "description": "The setting to post"
            },
            "value": {
                "type": "string",
                "required": true,
                "description": "The value of the setting"
            }
        }
    },
    "response": {
        "body": {
            "type": "object",
            "properties": {
                "response": {
                    "type": "string",
                    "description": "Response status"
                },
                "id": {
                    "type": "integer",
                    "description": "The id of the created user preference"
                }
            }
        }
    }
}
```

11.2.7 /users/preferences DELETE

```
{
    "endpoint": "/v1/users/preferences",
    "method": "DELETE",
    "description": "Deletes a user preference",
    "request": {
        "headers": [
            {
                "name": "Authorization",
                "type": "string",
                "required": true,
                "description": "Access token"
            },
            {
                "name": "Content-Type",
                "type": "string",
                "required": true,
                "description": "Request content type"
            }
        ],
        queryParameters: [ ],
        "body": { }
    },
    "response": {
        "body": {
            "type": "object",
            "properties": {
                "response": {
                    "type": "string",
                    "description": "Response status"
                }
            }
        }
    }
}
```

11.2.8 /stock-profiles GET


```
{
    "endpoint": "/v1/stock-profiles",
    "method": "GET",
    "description": "Gets information about a stock and its company",
    "request": {
        "headers": [
            {
                "name": "Authorization",
                "type": "string",
                "required": true,
                "description": "Access token"
            },
            {
                "name": "Content-Type",
                "type": "string",
                "required": true,
                "description": "Request content type"
            }
        ],
        "queryParameters": [
            {
                "ticker": "string",
                "required": true,
                "description": "The ticker of the stock"
            },
            {
                "exchange": "string",
                "required": true,
                "description": "The exchange of the stock"
            }
        ],
        "body": { }
    },
    "response": {
        "body": {
            "type": "object",
            "properties": {
                "response": {
                    "type": "string",
                    "description": "Response status"
                },
                "stock": {
                    "type": "object",
                    "properties": {
                        "ticker": {
                          "type": "string",
                          "description": "Ticker of the stock"
                        },
                        "exchange": {
                          "type": "string",
                          "description": "Exchange of the stock"
                        },
                        "displayName": {
                          "type": "string",
```

```
      "description": "Display name of the company"
    },
    "shortName": {
      "type": "string",
      "description": "Short name of the company"
    },
    "longName": {
      "type": "string",
      "description": "Long name of the company"
    },
    "industry": {
      "type": "string",
      "description": "Industry the company belongs to"
    },
    "sector": {
      "type": "string",
      "description": "Sector the company belongs to"
    },
    "website": {
      "type": "string",
      "description": "Company website"
    },
    "country": {
      "type": "string",
      "description": "Country of the company"
    },
    "address": {
      "type": "string",
      "description": "Address of the company"
    },
    "city": {
      "type": "string",
      "description": "City of the company"
    },
    "state": {
      "type": "string",
      "description": "State of the company"
    },
    "zip": {
      "type": "string",
      "description": "Zip code of the company"
    },
    "financialCurrency": {
      "type": "string",
      "description": "The currency the company uses"
    },
    "sharesOutstanding": {
      "type": "number",
      "description": "Number of shares outstanding of the
          company"
    },
    "trailingAnnualDividendRate": {
      "type": "number",
      "description": "Trailing annual dividend rate if company
          pays dividend"
    },
```

```
                        "trailingAnnualDividendYield": {
                          "type": "number",
                          "description": "Trailing annual dividend yield if
                              company pays dividend"
                        }
                      },
                      "description": "Stock company information"
                    },
                }
            }
        }
    }
```

11.2.9 /stock-histories GET

```
{
  "endpoint": "/v1/stock-histories",
  "method": "GET",
  "description": "Gets the stock history of a stock",
  "request": {
    "headers": [
      {
        "name": "Authorization",
        "type": "string",
        "required": true,
        "description": "Access token"
      },
      {
        "name": "Content-Type",
        "type": "string",
        "required": true,
        "description": "Request content type"
      }
    ],
    "queryParameters": [
      {
        "name": "ticker",
        "type": "string",
        "required": true,
        "description": "Ticker of the stock"
      },
      {
        "name": "exchange",
        "type": "string",
        "required": true,
        "description": "Exchange of the stock"
      },
      {
        "name": "startDate",
        "type": "string",
        "required": true,
        "description": "The start date to get the history from"
      },
      {
        "name": "endDate",
```

```
        "type": "string",
        "required": true,
        "description": "The end date to get the history to"
      },
      {
        "name": "interval",
        "type": "string",
        "enum": ["daily", "weekly", "monthly", "quarterly", "yearly"],
        "required": true,
        "description": "The interval at which the prices should be present"
      },
      {
        "name": "currency",
        "type": "string",
        "required": true,
        "description": "The currency to get the history in"
      }
    ],
    "body": { }
  },
  "response": {
    "body": {
      "type": "object",
      "properties": {
        "response": {
          "type": "string",
          "description": "Response status"
        },
        "history": {
          "type": "object",
          "properties": {
            "ticker": {
              "type": "string",
              "description": "Ticker of the stock"
            },
            "exchange": {
              "type": "string",
              "description": "Exchange of the stock"
            },
            "startDate": {
              "type": "string",
              "format": "date",
              "description": "The start date of the stock history"
            },
            "endDate": {
              "type": "string",
              "format": "date",
              "description": "The end date of the stock history"
            },
            "interval": {
              "type": "string",
              "enum": ["daily", "weekly", "monthly", "quarterly", "yearly"],
              "description": "The interval at which the prices are presented"
            },
            "history": {
              "type": "array",
```

```
"items": {
  "type": "object",
  "properties": {
    "date": {
      "type": "string",
      "format": "date",
      "description": "Date of the price"
    },
    "openPrice": {
      "type": "object",
      "properties": {
        "amount": {
          "type": "number",
          "description": "Open price amount"
        },
        "currency": {
          "type": "string",
          "description": "Open price currency"
        }
      }
    },
    "highPrice": {
      "type": "object",
      "properties": {
        "amount": {
          "type": "number",
          "description": "High price amount"
        },
        "currency": {
          "type": "string",
          "description": "High price currency"
        }
      }
    },
    "lowPrice": {
      "type": "object",
      "properties": {
        "amount": {
          "type": "number",
          "description": "Low price amount"
        },
        "currency": {
          "type": "string",
          "description": "Low price currency"
        }
      }
    },
    "closePrice": {
      "type": "object",
      "properties": {
        "amount": {
          "type": "number",
          "description": "Close price amount"
        },
        "currency": {
          "type": "string",
```

```
                    "description": "Close price currency"
                  }
                }
              }
            }
          },
          "description": "Stock price history list"
        },
        "dividends": {
          "type": "array",
          "items": {
            "type": "object",
            "properties": {
              "date": {
                "type": "string",
                "format": "date",
                "description": "Date of the dividend payout"
              },
              "payout": {
                "type": "object",
                "properties": {
                  "amount": {
                    "type": "number",
                    "description": "Dividend payout amount"
                  },
                  "currency": {
                    "type": "string",
                    "description": "Dividend payout currency"
                  }
                }
              }
            }
          },
          "description": "Stock dividend history list"
        }
      }
    }
  }
}
```

11.2.10 /portfolios GET


```
{
  "endpoint": "/v1/portfolios",
  "method": "GET",
  "description": "Gets the portfolios of a user",
  "request": {
    "headers": [
      {
        "name": "Authorization",
        "type": "string",
        "required": true,
        "description": "Access token"
      },
      {
        "name": "Content-Type",
        "type": "string",
        "required": true,
        "description": "Request content type"
      }
    ],
    "queryParameters": [
      {
        "name": "id",
        "type": "string",
        "required": false,
        "description": "ID of the portfolio. If not provided, returns all
            portfolios"
      }
    ],
    queryParameters: [ ],
    "body": { }
  },
  "response": {
    "body": {
      "type": "object",
      "properties": {
        "response": {
          "type": "string",
          "description": "Response status"
        },
        "portfolios": {
          "type": "array",
          "items": {
            "type": "object",
            "properties": {
              "id": {
                "type": "string",
                "description": "ID of portfolio"
              },
              "name": {
                "type": "string",
                "description": "Name of portfolio"
              },
              "owner": {
                "type": "string",
```

```
                "description": "Owner of portfolio"
              },
              "currency": {
                "type": "string",
                "description": "Currency of portfolio"
              }
            }
          },
          "description": "List of portfolios owned by user"
        }
      }
    }
  }
}
```

11.2.11 /portfolios POST


```
{
  "endpoint": "/v1/portfolios",
  "method": "POST",
  "description": "Creates a new portfolio",
  "request": {
    "headers": [
      {
        "name": "Authorization",
        "type": "string",
        "required": true,
        "description": "Access token"
      },
      {
        "name": "Content-Type",
        "type": "string",
        "required": true,
        "description": "Request content type"
      }
    ],
    queryParameters: [ ],
    "body": {
      "type": "object",
      "properties": {
        "portfolio": {
          "type": "object",
          "properties": {
            "name": {
              "type": "string",
              "required": true,
              "description": "Name of portfolio"
            },
            "currency": {
              "type": "string",
              "required": true,
              "description": "Currency of portfolio"
            }
          }
        }
      }
    }
  },
  "response": {
    "body": {
      "type": "object",
      "properties": {
        "response": {
          "type": "string",
          "description": "Response status"
        },
        "id": {
          "type": "string",
          "description": "ID of created portfolio"
        }
      }
```

```
      }
    }
  }
```

11.2.12 /portfolios PUT

```
{
  "endpoint": "/v1/portfolios",
  "method": "PUT",
  "description": "Updates an existing portfolio the user owns",
  "request": {
    "headers": [
      {
        "name": "Authorization",
        "type": "string",
        "required": true,
        "description": "Access token"
      },
      {
        "name": "Content-Type",
        "type": "string",
        "required": true,
        "description": "Request content type"
      }
    ],
    queryParameters: [ ],
    "body": {
      "type": "object",
      "properties": {
        "newName": {
          "type": "string",
          "required": false,
          "description": "New portfolio name"
        },
        "newCurrency": {
          "type": "string",
          "required": false,
          "description": "New portfolio currency"
        },
        "id": {
          "type": "string",
          "required": true,
          "description": "ID of portfolio"
        }
      }
    }
  },
  "response": {
    "body": {
      "type": "object",
      "properties": {
        "response": {
          "type": "string",
          "description": "Response status"
        }
```

```
        }
      }
    }
  }
```

11.2.13 /portfolios DELETE

```
  {
    "endpoint": "/v1/portfolios",
    "method": "DELETE",
    "description": "Deletes an existing portfolio",
    "request": {
      "headers": [
        {
          "name": "Authorization",
          "type": "string",
          "required": true,
          "description": "Access token"
        },
        {
          "name": "Content-Type",
          "type": "string",
          "required": true,
          "description": "Request content type"
        }
      ],
      "queryParameters": [
        {
          "name": "id",
          "type": "string",
          "required": true,
          "description": "ID of portfolio"
        }
      ],
      "body": { }
    },
    "response": {
      "body": {
        "type": "object",
        "properties": {
          "response": {
            "type": "string",
            "description": "Response status"
          }
        }
      }
    }
  }
```

11.2.14 /search-results GET


```
{
  "endpoint": "/v1/search-results",
  "method": "GET",
  "description": "Gets search results from a query",
  "request": {
    "headers": [
      {
        "name": "Authorization",
        "type": "string",
        "required": true,
        "description": "Access token"
      },
      {
        "name": "Content-Type",
        "type": "string",
        "required": true,
        "description": "Request content type"
      }
    ],
    "queryParameters": [
      {
        "name": "query",
        "type": "string",
        "required": true,
        "description": "Search query"
      },
      {
        "name": "stocks",
        "type": "boolean",
        "required": true,
        "description": "Whether to search for stocks or not"
      }
    ],
    "body": { }
  },
  "response": {
    "body": {
      "type": "object",
      "properties": {
        "response": {
          "type": "string",
          "description": "Response status"
            },
            "stocks": {
                "type": "array",
                "items": {
                    "type": "object",
                    "properties": {
                        "ticker": {
                          "type": "string",
                          "description": "Ticker of the stock"
                        },
                        "exchange": {
                          "type": "string",
```

```
      "description": "Exchange of the stock"
    },
    "displayName": {
      "type": "string",
      "description": "Display name of the company"
    },
    "shortName": {
      "type": "string",
      "description": "Short name of the company"
    },
    "longName": {
      "type": "string",
      "description": "Long name of the company"
    },
    "industry": {
      "type": "string",
      "description": "Industry the company belongs to"
    },
    "sector": {
      "type": "string",
      "description": "Sector the company belongs to"
    },
    "website": {
      "type": "string",
      "description": "Company website"
    },
    "country": {
      "type": "string",
      "description": "Country of the company"
    },
    "address": {
      "type": "string",
      "description": "Address of the company"
    },
    "city": {
      "type": "string",
      "description": "City of the company"
    },
    "state": {
      "type": "string",
      "description": "State of the company"
    },
    "zip": {
      "type": "string",
      "description": "Zip code of the company"
    },
    "financialCurrency": {
      "type": "string",
      "description": "The currency the company uses"
    },
    "sharesOutstanding": {
      "type": "number",
      "description": "Number of shares outstanding of the
          company"
    },
    "trailingAnnualDividendRate": {
```

```
                        "type": "number",
                        "description": "Trailing annual dividend rate if company
                            pays dividend"
                    },
                    "trailingAnnualDividendYield": {
                        "type": "number",
                        "description": "Trailing annual dividend yield if
                            company pays dividend"
                    }
                }
            },
            "description": "Stock company information list"
        }
    }
  }
 }
}
```

11.2.15 /stock-transactions GET


```
{
  "endpoint": "/v1/stock-transactions",
  "method": "GET",
  "description": "Gets stock transactions of a user",
  "request": {
    "headers": [
      {
        "name": "Authorization",
        "type": "string",
        "required": true,
        "description": "Access token"
      },
      {
        "name": "Content-Type",
        "type": "string",
        "required": true,
        "description": "Request content type"
      }
    ],
    "queryParameters": [ ],
    "body": { }
  },
  "response": {
    "body": {
      "type": "object",
      "properties": {
        "response": {
          "type": "string",
          "description": "Response status"
        },
        "stockTransactions": {
          "type": "array",
          "items": {
            "type": "object",
            "properties": {
              "id": {
                "type": "integer",
                "description": "ID of the transactions"
              },
              "portfolioId": {
                "type": "string",
                "description": "ID of the portfolios"
              },
              "ticker": {
                "type": "string",
                "description": "Stock ticker of the transaction"
              },
              "exchange": {
                "type": "string",
                "description": "Stock exchange of the transaction"
              },
              "amount": {
                "type": "number",
```

```
                "description": "Amount of stocks, positive for bought, neagtive
                    for sold"
              },
              "amountAdjusted": {
                "type": "number",
                "description": "Adjusted transaction amount according to splits"
              },
              "amountOwned": {
                "type": "number",
                "description": "The amount owned total at this transactions"
              },
              "timestamp": {
                "type": "integer",
                "description": "Timestamp of the transactions"
              },
              "priceNative": {
                "type": "object",
                "properties": {
                  "amount": {
                    "type": "number",
                    "description": "Native price amount"
                  },
                  "currency": {
                    "type": "string",
                    "description": "Native price currency"
                  }
                }
              },
              "priceUSD": {
                "type": "object",
                "properties": {
                  "amount": {
                    "type": "number",
                    "description": "Price amount in USD"
                  },
                  "currency": {
                    "type": "string",
                    "description": "Always USD"
                  }
                }
              }
            }
          },
          "description": "Array of stock transactions"
        }
      }
    }
  }
}
```

11.2.16 /stock-transactions POST


```
{
    "endpoint": "/v1/stock-transactions",
    "method": "POST",
    "description": "Create a stock transaction for a user on a specifed
        portfolio",
    "request": {
        "headers": [
            {
                "name": "Authorization",
                "type": "string",
                "required": true,
                "description": "Access token"
            },
            {
                "name": "Content-Type",
                "type": "string",
                "required": true,
                "description": "Request content type"
            }
        ],
        queryParameters: [ ],
        "body": {
            "portfolioId": {
                "type": "string",
                "required": true,
                "description": "The id of the portfolio"
            },
            "ticker": {
                "type": "string",
                "required": true,
                "description": "The ticker of the stock"
            },
            "exchange": {
                "type": "string",
                "required": true,
                "description": "The exchange of the stock"
            },
            "amount": {
                "type": "number",
                "required": true,
                "description": "The amount of stocks bought or sold. Positive for
                    bought, negative for sold"
            },
            "timestamp": {
                "type": "number",
                "required": true,
                "description": "Unix timestamp of the transaction"
            },
            "priceNative": {
                "type": "object",
                "required": true,
                "properties": {
                    "amount": {
                        "type": "number",
```

```
                        "required": true,
                        "description": "The price of the transaction"
                    },
                    "currency": {
                        "type": "string",
                        "required": true,
                        "description": "The native currency of the transaction"
                    }
                }
            }
        }
    },
    "response": {
        "body": {
            "type": "object",
            "properties": {
                "response": {
                    "type": "string",
                    "description": "Response status"
                },
                "id": {
                    "type": "integer",
                    "description": "The id of the created stock transaction"
                }
            }
        }
    }
}
```

11.2.17 /stock-transactions PUT


```
{
  "endpoint": "/v1/stock-transactions",
  "method": "PUT",
  "description": "Updates a stock transaction",
  "request": {
    "headers": [
      {
        "name": "Authorization",
        "type": "string",
        "required": true,
        "description": "Access token"
      },
      {
        "name": "Content-Type",
        "type": "string",
        "required": true,
        "description": "Request content type"
      }
    ],
    "queryParameters": [ ],
    "body": {
      "type": "object",
      "properties": {
        "portfolio": {
          "type": "string",
          "required": true,
          "description": "ID of portfolio"
        },
        "id": {
          "type": "string",
          "required": true,
          "description": "ID of transaction"
        },
        "newAmount": {
          "type": "number",
          "required": false,
          "description": "New transaction amount"
        },
        "newTimestamp": {
          "type": "integer",
          "required": false,
          "description": "New transaction timestamp"
        },
        "newPrice": {
          "type": "number",
          "required": false,
          "description": "New transaction price"
        },
        "newCurrency": {
          "type": "string",
          "required": false,
          "description": "New transaction currency"
        }
      }
```

```
        }
      },
      "response": {
        "body": {
          "type": "object",
          "properties": {
            "response": {
              "type": "string",
              "description": "Response status"
            },
            "id": {
              "type": "integer",
              "description": "Updated transaction ID"
            }
          }
        }
      }
    }
```

11.2.18 /stock-transactions DELETE


```
{
  "endpoint": "/v1/stock-transactions",
  "method": "DELETE",
  "description": "Deletes a stock transaction",
  "request": {
    "headers": [
      {
        "name": "Authorization",
        "type": "string",
        "required": true,
        "description": "Access token"
      },
      {
        "name": "Content-Type",
        "type": "string",
        "required": true,
        "description": "Request content type"
      }
    ],
    "queryParameters": [
      {
        "name": "portfolio",
        "type": "string",
        "required": true,
        "description": "ID of the portfolio"
      },
      {
        "name": "id",
        "type": "integer",
        "required": true,
        "description": "ID of the transaction"
      }
    ],
    "body": { }
  },
  "response": {
    "body": {
      "type": "object",
      "properties": {
        "response": {
          "type": "string",
          "description": "Response status"
        }
      }
    }
  }
}
```

11.2.19 /cash-transactions POST


```
{
  "endpoint": "/v1/cash-transactions",
  "method": "POST",
  "description": "Creates a cash transaction",
  "request": {
    "headers": [
      {
        "name": "Authorization",
        "type": "string",
        "required": true,
        "description": "Access token"
      },
      {
        "name": "Content-Type",
        "type": "string",
        "required": true,
        "description": "Request content type"
      }
    ],
    "queryParameters": [ ],
    "body": {
      "type": "object",
      "properties": {
        "portfolio": {
          "type": "string",
          "required": true,
          "description": "ID of portfolio"
        },
        "currency": {
          "type": "string",
          "required": true,
          "description": "Currency of transaction"
        },
        "nativeAmount": {
          "type": "number",
          "required": true,
          "description": "Transaction amount in native currency"
        },
        "timestamp": {
          "type": "integer",
          "required": true,
          "description": "Timestamp of the transaction"
        },
        "description": {
          "type": "string",
          "required": true,
          "description": "Description of the transaction"
        }
      }
    }
  },
  "response": {
    "body": {
      "type": "object",
```

```
      "properties": {
        "response": {
          "type": "string",
          "description": "Response status"
        },
        "id": {
          "type": "integer",
          "description": "Created transaction ID"
        }
      }
    }
  }
}
```

11.2.20 /cash-transactions PUT

```
{
  "endpoint": "/v1/cash-transactions",
  "method": "PUT",
  "description": "Updates a cash transaction",
  "request": {
    "headers": [
      {
        "name": "Authorization",
        "type": "string",
        "required": true,
        "description": "Access token"
      },
      {
        "name": "Content-Type",
        "type": "string",
        "required": true,
        "description": "Request content type"
      }
    ],
    "queryParameters": [ ],
    "body": {
      "type": "object",
      "properties": {
        "id": {
          "type": "integer",
          "required": true,
          "description": "ID of transaction"
        },
        "portfolio": {
          "type": "string",
          "required": true,
          "description": "ID of portfolio"
        },
        "newNativeCurrency": {
          "type": "string",
          "required": false,
          "description": "New transaction currency"
        },
        "newNativeAmount": {
```

```
            "type": "number",
            "required": false,
            "description": "New transaction amount in native currency"
          },
          "newDescription": {
            "type": "string",
            "required": false,
            "description": "New transaction description"
          },
          "newTimestamp": {
            "type": "integer",
            "required": false,
            "description": "New transaction timestamp"
          }
        }
      }
    },
    "response": {
      "body": {
        "type": "object",
        "properties": {
          "response": {
            "type": "string",
            "description": "Response status"
          },
          "id": {
            "type": "integer",
            "description": "Transaction ID"
          }
        }
      }
    }
  }
```

11.2.21 /cash-transactions DELETE


```
{
  "endpoint": "/v1/cash-transactions",
  "method": "DELETE",
  "description": "Deletes a cash transaction",
  "request": {
    "headers": [
      {
        "name": "Authorization",
        "type": "string",
        "required": true,
        "description": "Access token"
      },
      {
        "name": "Content-Type",
        "type": "string",
        "required": true,
        "description": "Request content type"
      }
    ],
    "queryParameters": [
      {
        "name": "portfolio",
        "type": "string",
        "required": true,
        "description": "ID of portfolio"
      },
      {
        "name": "id",
        "type": "integer",
        "required": true,
        "description": "ID of transaction"
      }
    ],
    "body": { }
  },
  "response": {
    "body": {
      "type": "object",
      "properties": {
        "response": {
          "type": "string",
          "description": "Response status"
        }
      }
    }
  }
}
```

11.2.22 /transactions GET


```
{
  "endpoint": "/v1/transactions",
  "method": "GET",
  "description": "Get all transactions, across all portfolios of a user",
  "request": {
    "headers": [
      {
        "name": "Authorization",
        "type": "string",
        "required": true,
        "description": "Access token"
      },
      {
        "name": "Content-Type",
        "type": "string",
        "required": true,
        "description": "Request content type"
      }
    ],
    "queryParameters": [
      {
        "name": "currency",
        "type": "string",
        "required": true,
        "description": "Currency"
      }
    ],
    "body": { }
  },
  "response": {
    "body": {
      "type": "object",
      "properties": {
        "response": {
          "type": "string",
          "description": "Response status"
        },
        "transactions": {
          "type": "array",
          "items": {
            "type": "object",
            "properties": {
              "type": {
                "type": "string",
                "description": "Transaction type [cash, stock, dividend payout]"
              },
              "id": {
                "type": "integer",
                "description": "ID of transaction"
              },
              "portfolio": {
                "type": "string",
                "description": "ID of portfolio"
              },
```

```
                "timestamp": {
                  "type": "integer",
                  "description": "Timestamp of transaction"
                },
                "description": {
                  "type": "string",
                  "description": "Description of transaction"
                },
                "value": {
                  "type": "object",
                  "properties": {
                    "amount": {
                      "type": "number",
                      "description": "Price of this transaction"
                    },
                    "currency": {
                      "type": "string",
                      "description": "Currency of this transaction"
                    }
                  }
                },
                "balance": {
                  "type": "object",
                  "properties": {
                    "amount": {
                      "type": "number",
                      "description": "Total balance of all transactions in
                          portfolio"
                    },
                    "currency": {
                      "type": "string",
                      "description": "Currency of the balance"
                    }
                  }
                },
                "combinedBalance": {
                  "type": "object",
                  "properties": {
                    "amount": {
                      "type": "number",
                      "description": "Total balance of all transactions accross all
                          portfolios"
                    },
                    "currency": {
                      "type": "string",
                      "description": "Currency of the combined balance"
                    }
                  }
                }
              }
            },
            "description": "List of all types of transactions"
          }
        }
      }
    }
```

```
}
```

11.2.23 /value-history GET

```
{
  "endpoint": "/v1/value-history",
  "method": "GET",
  "description": "Retrieves the complete value history of a user",
  "request": {
    "headers": [
      {
        "name": "Authorization",
        "type": "string",
        "required": true,
        "description": "Access token"
      },
      {
        "name": "Content-Type",
        "type": "string",
        "required": true,
        "description": "Request content type"
      }
    ],
    "queryParameters": [
      {
        "name": "startDate",
        "type": "string",
        "required": true,
        "description": "Start date of when to get value history"
      },
      {
        "name": "endDate",
        "type": "string",
        "required": true,
        "description": "End date of when to get value history"
      },
      {
        "name": "currency",
        "type": "string",
        "required": true,
        "description": "Currency to get value history in"
      }
    ],
    "body": { },
  },
  "response": {
    "body": {
      "type": "object",
      "properties": {
        "valueHistory": {
          "type": "object",
          "properties": {
            "valueHistory": {
              "type": "array",
              "items": {
```

```
"type": "object",
"properties": {
 "date": {
   "type": "string",
   "format": "date",
   "description": "Date of the total combined value"
 },
 "openPrice": {
   "type": "object",
   "properties": {
     "amount": {
       "type": "number",
       "description": "Open price amount of the total combined
           value"
     },
     "currency": {
       "type": "string",
       "description": "Open price currency of the total combined
           value"
     }
   }
 },
 "highPrice": {
   "type": "object",
   "properties": {
     "amount": {
       "type": "number",
       "description": "High price amount of the total combined
           value"
     },
     "currency": {
       "type": "string",
       "description": "High price currency of the total combined
           value"
     }
   }
 },
 "lowPrice": {
   "type": "object",
   "properties": {
     "amount": {
       "type": "number",
       "description": "Low price amount of the total combined
           value"
     },
     "currency": {
       "type": "string",
       "description": "Low price currency of the total combined
           value"
     }
   }
 },
 "closePrice": {
   "type": "object",
   "properties": {
     "amount": {
```

```
            "type": "number",
            "description": "Close price amount of the total combined
                value"
          },
          "currency": {
            "type": "string",
            "description": "Close price currency of the total combined
                value"
          }
        }
      }
    }
  },
  "description": "The value history, depicting the total value of all
      assets and transactions, accross all portfolios"
},
"dividendHistory": {
  "type": "array",
  "items": {
    "type": "object",
    "properties": {
      "date": {
        "type": "string",
        "format": "date",
        "description": "Date of the dividend payout"
      },
      "payout": {
        "type": "object",
        "properties": {
          "amount": {
            "type": "number",
            "description": "Dividend payout amount"
          },
          "currency": {
            "type": "string",
            "description": "Dividend payout currency"
          }
        }
      }
    }
  },
  "description": "Dividend payout history"
},
"portfolios": {
  "type": "array",
  "items": {
    "type": "object",
    "properties": {
      "id": {
        "type": "string",
        "description": "ID of the portfolio"
      },
      "name": {
        "type": "string",
        "description": "Name of the portfolio"
      },
```

```
                    "owner": {
                      "type": "string",
                      "description": "Owner of the portfolio"
                    },
                    "currency": {
                      "type": "string",
                      "description": "Currency of the portfolio"
                    }
                  }
                },
                "description": "List of all user owned portfolios"
              },
              "cashBalanceHistory": {
                "type": "array",
                "items": {
                  "type": "object",
                  "properties": {
                    "date": {
                      "type": "string",
                      "format": "date",
                      "description": "Date of the total cash balance registration"
                    },
                    "balance": {
                      "type": "object",
                      "properties": {
                        "amount": {
                          "type": "number",
                          "description": "Total cash balance amount"
                        },
                        "currency": {
                          "type": "string",
                          "description": "Total cash balance currency"
                        }
                      }
                    }
                  }
                },
                "description": "List of the cash balance history"
              }
            }
          }
        }
      }
    }
  }
}
```

11.2.24 /tokens POST

```
{
    "endpoint": "/v1/tokens",
    "method": "POST",
    "description": "Creates new token set, corresponds to logging in",
    "request": {
        "headers": [
            {
            "name": "Content-Type",
            "type": "string",
            "description": "Request content type"
            }
        ],
        queryParameters: [ ],
        "body": {
            "email": {
                "type": "string",
                "required": true,
                "description": "The email of the user"
            },
            "password": {
                "type": "string",
                "required": true,
                "description": "The password of the user
            }
        }
    },
    "response": {
        "body": {
            "refreshToken": {
                "type": "string",
                "description": "The generated refresh token"
            },
            "accessToken": {
                "type": "string",
                "description": "The generated access token"
            }
        }
    }
}
```

11.2.25 /tokens PUT

```
{
    "endpoint": "/v1/tokens",
    "method": "PUT",
    "description": "Refreshes the token set of a user",
    "request": {
        "headers": [
            {
                "name": "Content-Type",
                "type": "string",
                "required": true,
                "description": "Request content type"
            },
            {
                "name": "Authorization",
                "type": "string",
                "required": true,
                "description": "Refresh token"
            }
        ],
        queryParameters: [ ],
        "body": { }
    },
    "response": {
        "body": {
            "refreshToken": {
                "type": "string",
                "description": "The generated refresh token"
            },
            "accessToken": {
                "type": "string",
                "description": "The generated access token"
            }
        }
    }
}
```